



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Elektronikai Technológia Tanszék

Automatizált számlakezelés SAP Build Process Automation alkalmazásával

SZAKDOLGOZAT

Készítette
Csabuda Nóra

Konzulens
dr.Martinek Péter

2025. december 10.

Tartalomjegyzék

Kivonat	i
Abstract	ii
1. Bevezetés	1
1.1. Témafelvezetés: A Pénzügyi Funkció Stratégiai Átalakulása	1
1.1.1. A Szerepkör Forradalma: A Tranzakcióktól a Stratégiáig . . .	1
1.1.2. A Változás Gátja: A Manuális Munka Fogságában	1
1.1.3. A Technológia Mint Motor: Automatizáció, AI és Adatok . . .	2
1.1.4. Új Fókuszterületek: Tőkeáramlás és Szabályozói Megfelelés . .	2
1.1.5. Az Átalakuló Működési Modell és a „Tehetség-Válság”	3
1.2. Problémafelvetés: A Manuális Számlafeldolgozás Költségei és Techni-	
kai Korlátai	3
1.3. A Megoldás Iránya: Hyperautomation és az SAP Build Platform . . .	4
1.4. A Szakdolgozat Célkitűzései és Kutatási Kérdései	5
1.5. A Vizsgálat Fókusza és Korlátai	6
1.6. A Dolgozat Felépítése	6
2. Elméleti háttér és technológia alapok	7
2.1. Üzleti Folyamatmenedzsment (BPM) és Automatizálási Trendek . . .	7
2.1.1. Üzleti Folyamatmenedzsment (BPM) – A Stratégiai Alap . . .	7
2.1.2. Folyamatmodellezési Szabvány: BPMN 2.0	8
2.2. Robotikus Folyamatautomatizálás (RPA) – A Taktikai Eszköz és Kor-	
látai	9
2.3. Modern Integrációs Technológiák: REST és OData API-k	10
2.3.1. REST (Representational State Transfer)	10
2.3.2. OData (Open Data Protocol) – Az SAP Szabványa	10
2.4. A Hiperautomatizálás (Hyperautomation) – A Trendek Szintézise . .	10
2.5. Az SAP Stratégiai Válasza: A Business Technology Platform (BTP) .	11
2.6. Az SAP Build Platform: A BTP Hiperautomatizálási Eszköztára . . .	12
2.6.1. A „Motor”: SAP Build Process Automation (BPA) Eszköztára	13
2.6.2. A „Műszerfal”: SAP Build Work Zone	14
2.7. Fejezet Összegzése: A Platform Komponenseinek Szintézise	15
3. Standard és Vállalati Számlafeldolgozási Folyamat Elemzése	16
3.1. Bevezetés: Az „As-Is” Folyamat Megértése	16
3.1.1. A Standard SAP Számlafeldolgozási Folyamat („A Tankönyvi	
Modell”)	16

3.1.1.1.	Az Alapelv: A Háromoldalú Egyeztetés (Three-Way Match)	17
3.1.1.2.	A Folyamat Két Fő Típusa az SAP-ban	18
3.1.2.	Standard Automatizálási és Manuális Vezérlési Pontok	19
3.1.2.1.	Beépített Automatizációs Lehetőségek	20
3.1.2.2.	Tudatos Manuális Beavatkozási Pontok	21
3.1.3.	Egy Tipikus Vállalati Gyakorlat Elemzése („A Valós Világ”)	22
3.1.4.	Problémafeltárás: A Manuális Gyakorlat Hibái és Költségei	25
3.1.4.1.	Tipikus Manuális Hibák (Minőségi Kockázatok)	25
3.1.4.2.	Időigényes Lépések és Szűk Keresztmetszetek (Költség- és Időproblémák)	26
3.1.5.	Fejezet Összegzése	27
4.	Automatizált Számlafeldolgozási Prototípus Tervezése és Megvalósítása	29
4.1.	Bevezetés: A Prototípus Céljai és Keretrendszere	29
4.2.	A Prototípus Architektúrája és Komponensei	31
4.2.1.	A Folyamat Logikai Tervezése és BPMN Implementációja	33
4.2.2.	RPA Vezérlés és Helyi Erőforrás-kezelés	34
4.2.3.	Adatkinyerés Konfigurálása és API Kommunikáció	36
4.2.3.1.	Hitelesítés és Dokumentum Feltöltés	36
4.2.3.2.	Aszinkron Státuszfigyelés (Polling Mechanism)	37
4.2.3.3.	Adatkinyerés és Normalizálás	38
4.2.3.4.	Hitelesítés és Biztonságos Kulcskezelés	40
4.2.3.5.	Aszinkron Feldolgozás és Timeout Kezelés (Polling Pattern)	40
4.2.3.6.	Adatnormalizálás és Leképezés	42
4.2.4.	A Jóváhagyási Folyamat és Döntési Logika Tervezése	42
4.2.4.1.	Elsődleges Adatvalidáció (Gatekeeper Pattern)	42
4.2.4.2.	Üzleti Szabályok és Jóváhagyási Stratégia	43
4.2.5.	Üzleti Szabályok Implementációja (Technical Decisions)	44
4.2.6.	Felhasználói Felületek és Adatkötés (Forms & UI)	44
4.2.7.	Backend Integráció: Moduláris Felépítés és a Parkoló Bot	45
5.	A Prototípus Tesztelése és Eredmények Értékelése	48
5.1.	Bevezetés: A Tesztelés Módszertana	48
5.1.1.	A Tesztadat-generátor Felépítése és a Determinisztikus Megközelítés Indoklása	48
5.1.2.	Tesztelési Forgatókönyvek és Adatkészletek	49
5.2.	Tesztelési Eredmények	51
5.2.1.	Folyamatmonitorozás és Diagnosztika	51
5.2.2.	Eredmény: Sikeres „Touchless” Feldolgozás (TC_01)	52
5.2.3.	Eredmény: Elutasítási Folyamat Hiányos Adatoknál (TC_06)	53
5.2.4.	Eredmény: Emberi Beavatkozás és Jóváhagyás (TC_07)	54
5.2.5.	Összefoglaló Eredménytáblázat	55
5.3.	Eredmények Összehasonlítása az „As-Is” Folyamattal	56
5.3.1.	Feldolgozási Idő (Átfutási Idő) Becslése	56
5.3.2.	Hibakockázatok Csökkenése	57

5.3.3.	Átláthatóság és Nyomonkövethetőség	57
5.4.	Továbbfejlesztési Lehetőségek és Jövőkép	58
5.4.1.	Mesterséges Intelligencia Finomhangolása (AI Retraining)	58
5.4.2.	Process Mining és Valós Idejű Analitika Integrációja	58
5.4.3.	Teljes Pénzügyi Ciklus Automatizálása (End-to-End)	58
5.4.4.	Generatív AI Asszisztens (SAP Joule) Bevezetése	58
5.4.5.	Backend Üzleti Kivételkezelés Kiterjesztése	59
5.5.	Fejezet Összegzése	59
6.	Összefoglalás és Következtetések	60
6.1.	A Kutatási Eredmények Összegzése	60
6.2.	A Kutatási Kérdések Megválaszolása	60
6.3.	Záró Gondolatok és További Következtetések (ROI)	62
	Köszönetnyilvánítás	63
	Irodalomjegyzék	64
	Függelék	68
F.1.	A Tesztadat-generátor Forráskódja (Python)	68
F.2.	SAP Build Process Automation Szkriptek	69
F.2.1.	Adatfeltöltés és API Hívás (Prepare Upload)	69
F.2.2.	Adatkinyerés és Normalizálás (Extract Data)	69
F.3.	Tesztelési Jegyzőkönyv	70

HALLGATÓI NYILATKOZAT

Alulírott *Csabuda Nóra*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy autentikált felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2025. december 10.

Csabuda Nóra
hallgató

Kivonat

A pénzügyi szektor digitális transzformációjának egyik legnagyobb gátja a mai napig a manuális, papír- vagy PDF-alapú dokumentumfeldolgozás. A vállalati gyakorlatban a számlák jelentős része strukturálatlan formában érkezik, melyek feldolgozása a különböző rendszerek közötti manuális adatátvitellel („swivel-chair integration”) történik. Ez a folyamat nemcsak erőforrás-igényes és lassú, de magas hibakockázatot és átláthatatlanságot is eredményez a beszerzéstől a fizetésig (Purchase-to-Pay) tartó láncban.

Szakdolgozatomban ezen problémára kínálok megoldást a Hiperautomatizálás (Hyperautomation) eszköztárának alkalmazásával. A dolgozat célja egy olyan integrált, „end-to-end” számlafeldolgozó prototípus tervezése és megvalósítása az SAP Business Technology Platform (BTP) felhőalapú környezetében, amely minimalizálja az emberi beavatkozást, ugyanakkor biztosítja a folyamat kontrollálhatóságát.

A megvalósítás során az SAP Build Process Automation (SBPA) technológiát alkalmaztam, ötvözve a robotikus folyamatautomatizálást (RPA) és a mesterséges intelligenciát. A rendszer bemenetét egy saját fejlesztésű, determinisztikus Python szkripttel előállított szintetikus számlaállomány biztosítja, amely lehetővé tette a különböző hibatípusok (hiányos adat, formátumhiba) szisztematikus tesztelését. A strukturálatlan adatok kinyerését az SAP Document Information Extraction (DOX) szolgáltatás legújabb, Generative AI alapú modelljével valósítottam meg. A rendszer architektúrájában kiemelt szerepet kapott az egyedi JavaScript kódokkal vezérelt API kommunikáció és adattranszformáció, amely a technikai robot (Bot) és az üzleti folyamat (Process) közötti hidat képezi. A felhasználói interakciókat és a jóváhagyási döntéseket („Human-in-the-loop”) az SAP Build Work Zone egységesített felületén integráltam.

A prototípus tesztelése három különböző forgatókönyv alapján igazolta, hogy a megoldás képes a számlák teljesen automatikus („touchless”) feldolgozására, valamint a kivételes esetek intelligens kezelésére. Az eredmények azt mutatják, hogy az automatizált folyamat a manuális gyakorlathoz képest radikálisan csökkenti az átfutási időt és az adatrögzítési hibák számát, miközben valós idejű átláthatóságot biztosít a pénzügyi folyamatokban.

Abstract

One of the biggest obstacles to digital transformation in the financial sector remains manual, paper- or PDF-based document processing. In corporate practice, a significant portion of invoices arrives in unstructured formats, requiring manual data transfer between systems ("swivel-chair integration"). This process is not only resource-intensive and slow but also results in high error risks and a lack of transparency in the Purchase-to-Pay chain.

In my thesis, I propose a solution to this problem by applying the toolkit of Hyperautomation. The aim of the thesis is to design and implement an integrated, end-to-end invoice processing prototype within the cloud-based environment of the SAP Business Technology Platform (BTP), minimizing human intervention while ensuring process control.

For the implementation, I utilized SAP Build Process Automation (SBPA) technology, combining Robotic Process Automation (RPA) with Artificial Intelligence. The system input is provided by a synthetic invoice dataset generated by a self-developed deterministic Python script, which allowed for systematic testing of various error types (missing data, format errors). The extraction of unstructured data was achieved using the latest Generative AI-based model of the SAP Document Information Extraction (DOX) service. A key component of the system architecture is API communication and data transformation driven by custom JavaScript codes, bridging the technical bot and the business process. User interactions and approval decisions ("Human-in-the-loop") were integrated into the unified interface of SAP Build Work Zone.

Testing the prototype across three different scenarios confirmed that the solution is capable of fully automated ("touchless") invoice processing as well as intelligent handling of exception cases. The results demonstrate that the automated process radically reduces turnaround time and data entry errors compared to manual practices, while providing real-time transparency in financial processes.

1. fejezet

Bevezetés

1.1. Témafelvezetés: A Pénzügyi Funkció Stratégiai Átalakulása

A pénzügyi funkció (*Finance*) jelentős átalakuláson megy keresztül, amelyet a folyamatos költségsökkentési nyomás és a mélyebb üzleti betekintést igénylő stratégiai tanácsadói szerep iránti növekvő igény egyaránt vezérel. A hagyományos, reaktív és tranzakció-fókuszú „eredménykimutató” (*scorekeeper*) szerepkörből a pénzügy egyre inkább proaktív, „stratégiai értékteremtővé” (*strategic value creator*) válik.

1.1.1. A Szerepkör Forradalma: A Tranzakcióktól a Stratégiaiáig

A pénzügy jövőbeli alapvető szerepváltása a tranzakciók feldolgozásától a stratégiai partnerség felé mozdul el. Ezt a trendet a piacvezető tanácsadó cégek egyértelműen alátámasztják. A Deloitte kiemeli, hogy a pénzügy szerepe „a kontrollálástól a stratégiai tervezésig” (angolul: *from controlling to strategic planning*) mozdul el, és egy „előretekinthető üzleti funkcióvá válik, amely alakítja az üzleti irányt, ahelyett, hogy egyszerűen a múltbeli eredményekről számolna be”[28].

Ahogy a PwC elemzése fogalmaz, a „pénzügy a pénzügyért” (*finance for finance*) – vagyis a hagyományos funkciók hatékonyságának növelése – továbbra is fontos, de a jövő kulcsa a „pénzügy az üzletért” (*finance for business*), amely mélyebb betekintést nyújt az egész szervezetben[16].

1.1.2. A Változás Gátja: A Manuális Munka Fogságában

Ez az evolúció azonban nem könnyű. A stratégiai partnerség legnagyobb akadály, hogy a magasan képzett pénzügyi csapatok idejét felemésztik a manuális, ismétlődő feladatok. A Deloitte nyíltan kimondja, hogy „Sok pénzügyi csapat még mindig manuális folyamatokra támaszkodik... ami lassítja a döntéshozatalt, és korlátozza a pénzügyi vezető (CFO) képességét, hogy teljes mértékben stratégiai üzleti partnerként lépjen fel”[10]. Egy sokat idézett McKinsey tanulmány számszerűsíti a problémát: becslésük szerint a pénzügyi tevékenységek 42%-a már ma is teljesen automatizálható a meglévő technológiákkal, további 19% pedig nagyrészt az lenne[31]. Ezen akadály leküzdése miatt válik a technológia az átalakulás elsődleges hajtóerejévé.

1.1.3. A Technológia Mint Motor: Automatizáció, AI és Adatok

Ez a szerepváltás elképzelhetetlen a technológia és az adatok együttes forradalma nélkül.

A „Finance Factory” és az Automatizáció: A fókusz a Deloitte által „pénzügyi gyárnak” (*finance factory*) nevezett koncepcióban az operatív, back-office feladatokról egyértelműen a pénzügyi betekintést nyújtó front-office felé tolódik[10]. A tranzakcionális tevékenységek szinte teljes automatizálása várható, ami a Gartner szerint „a tranzakciós testreszabás végéhez” (*the end of transactional customization*) vezet[15]. A PwC szövege ezt úgy írja le, mint a „háromszög megfordítása”: a technológia (AI, ML) lehetővé teszi, hogy a korábban tranzakciókkal terhelt széles bázis helyett a fókusz a cselekvésre ösztönző üzleti intelligenciára kerüljön[16].

Számlafeldolgozás (AP) Példa: Ez a stratégiai törekvés már a legalapvetőbb „back-office” funkciókban is megjelenik. Az Ardent Partners jelentése szerint még a Számlafeldolgozási (Accounts Payable) osztályok is „helyet követelhetnek a stratégiai asztalnál”, mivel az automatizálás révén felszabadult idejüket már nem adatpötyögéssel, hanem készpénzmenedzsmenttel és szállítói kapcsolatok elemzésével tölthetik[38].

Mesterséges Intelligencia (AI) és Analitika: A jövő már nem csak az egyszerű automatizálásról szól. A Gartner „AI ügynökökből álló munkaerőt” (*a workforce of AI agents*) és „gép-dominálta döntéshozatalt” (*machine-dominated decision making*) vizionál[15]. A pénzügyi tervezési és elemzési funkció átalakításának középpontjában a prediktív analitika és az AI-eszközök állnak.

Az Adat Mint Alapfeltétel: Az összes jelentés egyetért abban, hogy a jövő pénzügyi funkciójának alapja a jó minőségű adat. Ahogy a Deloitte fogalmaz: a technológia nem lesz „csodaszer” a megfelelő adatarchitektúra nélkül[10]. A PwC kiemeli, hogy a vállalati adatmodellekbe és digitális képességekbe történő dedikált befektetés hiányában a pénzügyi vezetők nehezen tudnak majd valódi értéket teremteni[16].

1.1.4. Új Fókuszterületek: Tőkeáramlás és Szabályozói Megfelelés

A stratégiai tanácsadói szerepkör két új, kiemelt fókuszterületet hoz előtérbe, amelyeket a PwC elemzése részletesen tárgyal[16]:

Tőke és Cash Flow (*Capital and Cash Flow*): A bizonytalan gazdasági környezet miatt a készpénzbeszedésre és a működő tőkére irányuló figyelem megnövekedett. Kulcsfontosságúvá válik a „megrendeléstől a készpénzbeérkezésig” (*order-to-cash*) folyamat automatizálása. Mivel a cégfelvásárlások és egyéb tranzakciók 2025-ben várhatóan felpörögnek, a CFO-knak újra kell értékelnük, hogy képesek-e következetesen készpénzre váltani a nyereséget[16].

Szabályozói Jelentések (*Regulatory Reporting*): Az új, komplex szabályozási követelmények (mint az OECD Pillar Two, ESG) drámaian megnövelik az átláthatóság és a részletes adatszolgáltatás iránti igényt. A PwC felmérése szerint a pénzügyi vezetők 70%-a kockázatként tekint az amerikai szabályozói környezetre[16]. Sok vállalat integrált megoldások híján manuális, szigetszerű adatgyűjtéssel pazarolja az erőforrásait[16].

1.1.5. Az Átalakuló Működési Modell és a „Tehetség-Válság”

Az új feladatok új munkaszervezést és újfajta munkatársakat igényelnek.

A Tehetség Kérdése: A szükséges készségek lehetségesen megváltoznak. A hagyományos számviteli tudás helyett a jövő pénzügyi munkatársának erős kommunikációs készségekre, proaktivitásra és „digitális hozzáértésre” lesz szüksége. A Deloitte ezt „tehetségeként folyó háborúnak”[10], míg a Gartner egyenesen „pénzügyi tehetségválságnak” (*finance talent crash*) nevezi[15], rámutatva, hogy 2019 és 2021 között 300 000 könyvelő hagyta el a pályát. A PwC hangsúlyozza, hogy a képzett tehetségeként folyó verseny kemény, ezért a szervezetnek innovatív kultúrát kell kínálnia[16].

Új Működési Modellek: A költségek kordában tartása mellett a szervezeteknek új erőforrásokat is be kell vonniuk. A Deloitte kiemeli, hogy a COVID-19 által tesztelt távmunka és hibrid modellek tartósan megmaradnak[10]. Emellett a PwC elemzése rámutat, hogy sok szervezet harmadik fél (pl. menedzselt szolgáltatók) felé szervezi ki a nem alapvető tevékenységeket. Ennek célja már nem csupán a költségcsökkentés, hanem a szakemberhiány pótlása és a belső erőforrások felszabadítása az értéknövelő, stratégiai feladatok elvégzésére[16].

1.2. Problémafelvetés: A Manuális Számlafeldolgozás Költségei és Technikai Korlátai

Ahogy a témafelvezetésben láthattuk, a pénzügyi osztály stratégiai szerepvállalása és a manuális számlafeldolgozás valósága között mély szakadék tátong. A szakdolgozat által vizsgált probléma azonban kettős természetű: egyrészt üzleti, másrészt hangsúlyosan informatikai jellegű.

Üzleti problémák: A manuális számlafeldolgozásból eredő közvetlen és közvetett költségek jelentősek. A magasan képzett pénzügyi munkatársak idejüket azzal töltik, hogy PDF-ről vagy papírról adatokat gépelnek át a vállalati rendszerbe (ERP), mint például az SAP[38]. Ez a „forgószék” (*swivel-chair*) probléma – ahol a munkatárs egyik monitorról a másikra viszi át az adatokat – lassú, drága és alacsony hozzáadott értékű. Ehhez társul a pénzügyi veszteség (duplikált utalások, elbukott skontók), a határidők mulasztása, valamint az átláthatóság hiánya („black box effektus”), ami lehetetlenné teszi a hatékony készpénzmenedzsmentet[38].

Mérnöki-informatikai problémák: Mérnöki szempontból a folyamat alapvető hibája a vállalati rendszerek integrálatlansága és az adatok heterogén szerkezete. Je-

lenleg a levelezőrendszer (Microsoft Outlook) és a vállalatirányítási rendszer (SAP S/4HANA) egymástól elszigetelt technológiai szigetként (*silo*) működik. Hiányzik a két rendszer közötti közvetlen API-kapcsolat, amely lehetővé tenné az adatok automatikus áramlását. A legnagyobb technikai kihívást a strukturálatlan és a strukturált adatok közötti szakadék jelenti. A beérkező számlák (PDF, képfájl) strukturálatlan adatokat tartalmaznak – emberi szem számára értelmezhető vizuális információ –, míg az ERP rendszer szigorúan strukturált adatbázis-bejegyzéseket igényel. A jelenlegi gyakorlatban ezt a konverziót az ember végzi „humán interfészként”, ami technikai értelemben egy kényszermegoldás a rendszerintegráció hiányára. A probléma tehát nem csupán a munkaerő leterheltsége, hanem egy hiányzó technológiai láncszem az adatfolyam-architektúrában.

1.3. A Megoldás Iránya: Hyperautomation és az SAP Build Platform

A 1.2. pontban vázolt komplex problémákra – az adatrögzítéstől a jóváhagyási lánccon át az átláthatóság hiányáig – a válasz már nem egyetlen, izolált technológia. Míg az automatizálási hullám korai szakaszát a *Robotic Process Automation* (RPA) uralta, a piac felismerte, hogy ez önmagában kevés. A valódi megoldás a *Hyperautomation* (hiperautomatizálás) koncepciója, amely több technológia – AI, BPM és RPA – integrált alkalmazását jelenti.

A világ egyik vezető vállalatirányítási szoftvercége, az SAP válasza erre az SAP Business Technology Platform (BTP), és ezen belül az SAP Build Process Automation (BPA).

Technológiaválasztás indoklása: Bár a feladat technikailag megvalósítható lenne nyílt forráskódú eszközökkel (pl. Python szkriptekkel, Selenium alapú webes automatizációval) is, mérnöki és vállalati szempontból az SAP Build választása indokolt. Egy nagyvállalati környezetben a „barkácsolt” szkriptekkel szemben kritikus elvárás az adatbiztonság, a megfelelés (compliance) és a karbantarthatóság. Az SAP Build legnagyobb előnye a natív integráció: előre definiált, biztonságos konnektorokkal rendelkezik az S/4HANA rendszerhez (OData API-k kezelése), kezeli a hitelesítést, és biztosítja a folyamatok auditálhatóságát, amit egy egyedi fejlesztésű szkriptnél nulláról kellene felépíteni.

A BPA ereje abban rejlik, hogy egyetlen, vizuális felületen egyesíti a hiperautomatizáláshoz szükséges három kulcsképeséget, amelyek pontosan lefedik a számlafeldolgozás problémáit:

- **Mesterséges Intelligencia (AI):** Strukturálatlan adatok, például PDF vagy szkennelt számlák tartalmának automatikus kinyerése (*Document AI*).
- **Folyamatmenedzsment (BPM/Workflow):** Összetett, emberi beavatkozást igénylő jóváhagyási és ellenőrzési láncok grafikus modellezése és futtatása.
- **Robotika (RPA):** Automatizált botok futtatása, amelyek adatokat írhatnak be vagy olvashatnak ki más rendszerekből.

1.4. A Szakdolgozat Célkitűzései és Kutatási Kérdései

Az előzőekben felvázolt elméleti háttér és a beazonosított üzleti probléma (a manuális számlafeldolgozás költségei) alapján a jelen szakdolgozat a következő fő célkitűzést fogalmazza meg:

A szakdolgozat fő célja, hogy a manuális számlafeldolgozás fent vázolt problémáira választ adva, megtervezzen és megvalósítson egy automatizált számlakezelési prototípust az SAP Build Process Automation felhőalapú környezetében.

Ezen átfogó cél elérése érdekében a dolgozat a következő konkrét részcélokat, egyben kutatási kérdéseket tűzi ki:

Elméleti célkitűzés: A dolgozat célja részletesen bemutatni az SAP Build Process Automation (BPA) technológiai eszköztárát és az azt körülölelő hiperautomatizálási, valamint low-code/no-code (LCNC) koncepciókat.

Kutatási kérdés: Hogyan integrálható az SAP BPA architektúrája a meglévő heterogén vállalati környezetbe a technológiai szigetek („silók”) felszámolása érdekében, és milyen architekturális előnyöket biztosít a hiperautomatizálás megközelítése a hagyományos, izolált RPA megoldásokkal szemben?

Elemzési célkitűzés: A dolgozat célja elemezni egy tipikus vállalati számlafeldolgozási gyakorlatot, azonosítani annak manuális lépéseit, főbb hibalehetőségeit és szűk keresztmetszeteit.

Kutatási kérdés: Hol keletkeznek a legnagyobb költségek és a legtöbb hiba a manuális AP folyamat során, és melyek azok a pontok, amelyek automatizálással javíthatók?

Gyakorlati célkitűzés (Prototípus): A dolgozat fő gyakorlati célja egy működőképes prototípus (*Proof of Concept*) létrehozása. Ennek a megoldásnak képesnek kell lennie egy beérkező számladokumentum (pl. PDF) adatainak automatikus kinyerésére (AI), az adatok üzleti szabályok szerinti érvényességvizsgálatára, és egy előre definiált, többszintű jóváhagyási folyamat (*workflow*) elindítására.

Kutatási kérdés: Milyen integrációs korlátok és architekturális kompromisszumok mellett valósítható meg a heterogén rendszereket (Microsoft Outlook, SAP S/4HANA) összekapcsoló automatizáció egy felhőalapú, low-code környezetben?

Értékelési célkitűzés: Végül a dolgozat célja a megvalósított prototípus működésének tesztelése és hatékonyságának értékelése a korábban elemzett manuális folyamathoz képest.

Kutatási kérdés: Mennyivel csökkenti a prototípus a feldolgozási időt és a manuális hibák lehetőségét a hagyományos, e-mail és kézi adattrögzítés alapú módszerhez viszonyítva?

1.5. A Vizsgálat Fókusza és Korlátai

A szakdolgozat célkitűzéseinek (1.4. pont) reális teljesíthetősége érdekében elengedhetetlen a vizsgálat fókuszának és korlátainak egyértelmű meghatározása. Fontos hangsúlyozni, hogy a dolgozat gyakorlati része egy prototípus (*Proof of Concept*, PoC) elkészítésére vállalkozik. A cél nem egy teljeskörű, éles vállalati bevezetésre kész megoldás létrehozása, hanem annak koncepcionális bizonyítása, hogy az SAP Build Process Automation technológia alkalmas a beazonosított üzleti probléma (manuális számlafeldolgozás) hatékony kezelésére. A prototípus a teljes *Purchase-to-Pay* (P2P) folyamat egy szűk, de kritikusan fontos szakaszára fókuszál: a számla beérkezésétől a könyvelésre való előkészítéséig. A folyamat definiált végpontja a számla „parkolása” (*invoice parking*) az SAP rendszerben. Ez azt jelenti, hogy a számla adatstrukturálása, üzleti szabályok szerinti ellenőrzése és a felelősök általi jóváhagyása megtörtént, és a bizonylat előkészített státuszba került. A dolgozat nem terjed ki a tényleges főkönyvi könyvelés (*invoice posting*) automatizálására, sem pedig a pénzügyi teljesítés, azaz a kifizetési futtatás (*payment run*) lépéseire. Továbbá a megvalósítás az SAP Business Technology Platform (BTP) ingyenes próbaverziós (*Trial*) vagy fejlesztői környezetében történik, ami befolyásolhatja az elérhető funkciók körét és a rendszer teljesítményét egy éles vállalati környezethez képest.

1.6. A Dolgozat Felépítése

A szakdolgozat a Bevezetésben lefektetett célkitűzések elérése érdekében a következő logikai szerkezet szerint épül fel:

A **2. fejezet** bemutatja a dolgozat elméleti alapjait és a felhasznált technológiát. Részletesen tárgyalja a hiperautomatizálás koncepcióját, valamint az SAP Build Process Automation felhőalapú platform eszközkészletét, kitérve annak AI, workflow és RPA képességeire.

A **3. fejezet** a vizsgált üzleti folyamatot, a szállítói számlák kezelését elemzi. Először bemutatja a standard SAP „tankönyvi” folyamatát, majd azt összeveti egy tipikus vállalati gyakorlattal, feltárva annak manuális hibáit, költségeit és szűk keresztmetszeteit. Ez a fejezet alapozza meg a prototípus szükségességét.

A **4. fejezet** a dolgozat gyakorlati magja, amely részletesen bemutatja az automatizált prototípus tervezésének és megvalósításának lépéseit. Ez magában foglalja az adatkinyerés (AI) konfigurálását, a jóváhagyási logika (workflow) felépítését és az SAP rendszerrel való integrációt (a számla parkolását).

Az **5. fejezet** a prototípus tesztelésének eredményeit és a megoldás értékelését tartalmazza. A fejezet összehasonlítja az automatizált megoldás hatékonyságát (pl. feldolgozási idő, hibalehetőségek) a **3. fejezetben** azonosított manuális folyamattal, és javaslatokat fogalmaz meg a lehetséges továbbfejlesztésre.

Végül a **6. fejezet** összegzi a dolgozat kutatási eredményeit, válaszol a Bevezetésben feltett kutatási kérdésekre, és levonja a végső következtetéseket.

2. fejezet

Elméleti háttér és technológia alapok

Ahogy az **1. fejezetben** megállapítottuk, a pénzügyi osztály stratégiai átalakulását leginkább a manuális, ismétlődő feladatok terhe gátolja. A technológiai válasz erre a kihívásra nem egyetlen eszköz, hanem az automatizálási stratégiák fokozatos evolúciója. Ahhoz, hogy a dolgozatban vizsgált SAP Build Process Automation platform képességeit megértsük, először meg kell vizsgálnunk azt a három alapkoncepciót, amelyre épül: az Üzleti Folyamatmenedzsmentet (BPM), a Robotikus Folyamatautomatizálást (RPA) és az ezeket szintetizáló Hiperautomatizálást (*Hyperautomation*). Bár e fogalmakat gyakran összekeverik, eltérő megközelítést képviselnek: a BPM a stratégiai, felülről lefelé irányuló folyamat-szervezést biztosítja, míg az RPA egy taktikai, alulról építkező eszközt ad a konkrét feladatok végrehajtására.

2.1. Üzleti Folyamatmenedzsment (BPM) és Automatizálási Trendek

2.1.1. Üzleti Folyamatmenedzsment (BPM) – A Stratégiai Alap

Az automatizálási stratégiák elméleti alapja az Üzleti Folyamatmenedzsment (*Business Process Management*, BPM). A BPM, ahogy azt a Gartner iparági elemző definiálja, egy olyan menedzsment diszciplína, amely különböző módszereket alkalmaz a vállalat üzleti folyamatainak szisztematikus felfedezésére, modellezésére, elemzésére, mérésére, javítására és optimalizálására [7]. Nem egy szoftverről, hanem egy szemléletmódról van szó, amelynek célja, hogy összehangolja az emberek, rendszerek és információk viselkedését egy adott üzleti stratégia és a kívánt üzleti eredmények elérése érdekében. A BPM kulcsfontosságú az IT beruházások és a vállalati stratégia összehangolásában. A BPM a gyakorlatban nem egy egyszeri projekt, hanem egy folyamatos, iteratív életciklus, amely biztosítja a folyamatok állandó javítását [42]. Ez az életciklus jellemzően öt fő szakaszból áll:

Tervezés (*Design*): A meglévő („As-Is”) folyamatok azonosítása, feltérképezése és a szűk keresztmetszetek elemzése. Ebben a fázisban tervezik meg az ideális, javított („To-Be”) folyamatot.

Modellezés (*Model*): A megtervezett folyamat vizuális leképezése, tesztelése és szimulálása különböző forráskönyvek szerint. A modellezés szabványosított jelölésrendszereket, mint például a BPMN (*Business Process Model and Notation*), használ a folyamatábrák elkészítésére.

Végrehajtás (*Execute*): A megtervezett és modellezett munkafolyamat bevezetése és „élesítése”, gyakran egy BPM szoftver segítségével, amely irányítja a feladatokat az emberek és a rendszerek között.

Monitorozás (*Monitor*): A futó folyamatok teljesítményének valós idejű követése, kulcsfontosságú teljesítménymutatók (KPI-k), például átfutási idő vagy költségek mérése.

Optimalizálás (*Optimize*): A monitorozás során gyűjtött adatok alapján a folyamat folyamatos finomítása, a hibák javítása és a hatékonyság további növelése.

Látható tehát, hogy a BPM egy stratégiai, „felülről lefelé” (*top-down*) irányuló megközelítés, amely a teljes szervezet működését és folyamatainak egészségét tartja szem előtt. A BPM biztosítja azt a „karmesteri” szerepet, amely keretbe foglalja és irányítja az egyes automatizálási lépéseket.

2.1.2. Folyamatmodellezési Szabvány: BPMN 2.0

A folyamatok tervezéséhez és dokumentálásához elengedhetetlen egy egységes, szoftverfüggetlen jelölésrendszer használata. Az ipari szabványnak tekinthető **BPMN 2.0 (Business Process Model and Notation)** olyan grafikus nyelvet biztosít, amely mind az üzleti elemzők, mind a fejlesztők számára érthetővé teszi a komplex folyamatokat.

A BPMN szabvány egyik legfontosabb eleme a felelősségi körök vizuális elhatárolása, amelyet a *Pool* (medence) és *Lane* (sáv) fogalmak valósítanak meg. Egy medence jellemzően egy szervezetet vagy rendszert jelöl, míg a sávok az azon belüli szerepköröket (pl. "Pénzügyi Asszisztens", "ERP Rendszer"). Ez a struktúra, az úgynevezett *Swimlane diagram*, lehetővé teszi annak egyértelmű ábrázolását, hogy mikor történik átadás ember és gép, vagy két szervezeti egység között.

A folyamatok logikáját az alábbi alapelemek építik fel:

- **Események (Events):** A folyamat indítását (pl. e-mail érkezése), köztes állapotait vagy lezárását jelölik (kör alakzat).
- **Tevékenységek (Activities):** A konkrét elvégzendő feladatok, amelyek lehetnek emberi (User Task) vagy gépi (Service Task) lépések (lekerekített téglalap).
- **Döntési Kapuk (Gateways):** Az elágazási pontok, ahol a folyamat a definiált feltételek alapján különböző utakon haladhat tovább (rombusz alakzat).

A BPMN 2.0 használata nem csupán dokumentációs célokat szolgál; a modern folyamatmotorok (mint az SAP Build Process Automation) képesek ezeket a modelleket közvetlenül végrehajtható kóddá fordítani, biztosítva a terv és a megvalósítás összhangját.

2.2. Robotikus Folyamatautomatizálás (RPA) – A Taktikai Eszköz és Korlátai

Míg a BPM a folyamatok stratégiai „karmestere”, addig a Robotikus Folyamat-automatizálás (Robotic Process Automation, RPA) a „digitális munkás”, amely a konkrét, repetitív feladatokat végzi el. Az RPA olyan szoftvertechnológia, amely lehetővé teszi szoftverrobotok (botok) építését, amelyek az emberi felhasználói felületi (UI) interakciókat (kattintás, gépelés, adatbevitel) utánozzák [13].

Az RPA ideális taktikai megoldást kínál az **1. fejezetben** azonosított „manuális munka fogságában” lévő pénzügyi osztályok számára. A „forgószék” (swivel-chair) probléma – ahol az ügyintéző adatokat másol egy PDF-ből vagy Excelből egy SAP tranzakcióba – tökéletes célpontja az RPA-nak. A Gartner [2] rámutat, hogy az RPA értéke nem csupán a megtakarított munkaórákban rejlik, hanem a pontosság és a megfelelés javításában is.

Az RPA botoknak két fő típusa van [41, 5]:

- **Attended (Felügyelt) bot:** A felhasználó asztali gépén fut, mint egy „digitális asszisztens”. A felhasználó indítja el, és vele együttműködve végez el egy feladatot.
- **Unattended (Felügyelet nélküli) bot:** Szerveren fut, emberi beavatkozás nélkül. Jellemzően egy trigger (pl. API hívás vagy időzítés) indítja el a háttérben.

Fontos azonban objektíven látni a technológia hátrányait is. Bár az RPA gyors implementációt tesz lehetővé (mivel nem igényli a backend rendszerek átalakítását), ez az előny egyben a legnagyobb gyengesége is:

1. **UI-érzékenység és Törékenység:** Mivel a hagyományos RPA a grafikus felület (GUI) elemeire (gombok, mezők ID-ja, Xpath) támaszkodik, a rendszer **rendkívül sérülékeny a változásokra**. Egy apró SAP verziófrissítés, amely átrendezi a képernyőt vagy átnevez egy mezőt, a robot azonnali leállását okozhatja.
2. **Magas Karbantartási Igény:** Az említett törékenységi miatt az RPA botok folyamatos felügyeletet és karbantartást igényelnek, ami hosszú távon jelentős rejtett költséget (Technical Debt) generálhat.
3. **„Ragtapas” Megoldás:** Mérnöki szempontból az RPA gyakran csak tüneti kezelés: automatizál egy rosszul működő folyamatot anélkül, hogy kijavítaná a mögöttes strukturális problémát vagy integrációs hiányosságot.

Éppen ezen korlátok miatt hangsúlyozzák a szakértők [24, 33], hogy az RPA nem helyettesíti, hanem kiegészíti a BPM-et: az RPA a „hogyan”-t oldja meg gyorsan, de törékenyen, míg a BPM és a modern API-alapú integrációk a „miért”-et és a stabilitást biztosítják.

2.3. Modern Integrációs Technológiák: REST és OData API-k

Míg az RPA a felhasználói felületen keresztül imitálja az emberi munkavégzést, a modern rendszerintegráció gerincét az Alkalmazásprogramozási Interfészek (API-k) alkotják. Mérnöki szempontból az API-alapú integráció a **2.2 alfejezetben** tárgyalt RPA robusztusabb és stabilabb alternatívája.

2.3.1. REST (Representational State Transfer)

A REST egy szoftverarchitektúra-stílus, amely a HTTP protokoll szabványos metódusait (GET, POST, PUT, DELETE) használja az erőforrások kezelésére. A REST API-k előnye az RPA-val szemben, hogy közvetlenül a backend rendszer logikájával kommunikálnak, megkerülve a grafikus felületet (GUI). Ezáltal a megoldás érzékelté válik a vizuális változásokra (pl. gombok áthelyezése), és jelentősen gyorsabb adatfeldolgozást tesz lehetővé.

2.3.2. OData (Open Data Protocol) – Az SAP Szabványa

Az SAP ökoszisztémában kiemelt jelentőséggel bír az OData szabvány, amely a REST architektúrára épül, de azt további szemantikai réteggel egészíti ki. Az OData lehetővé teszi, hogy az adatokat (pl. számlák, partnerek) egységes, lekérdezhető és manipulálható erőforrásként kezeljük webes technológiákon keresztül. A gyakorlati megvalósítás során (pl. számla könyvelése S/4HANA rendszerbe) az OData V2/V4 szolgáltatások használata biztosítja a tranzakciós integritást és a szabványos hibakezelést, amit egy egyszerű képernyő-rögzítő robot nem tud garantálni. A modern SAP Build Process Automation platform ezért preferálja a *Service Task*-ok (API hívások) használatát a hagyományos RPA szkriptekkel szemben ott, ahol rendelkezésre áll megfelelő végpont.

2.4. A Hiperautomatizálás (Hyperautomation) – A Trendek Szintézise

Míg a BPM a stratégiai folyamattervezést, az RPA pedig a taktikai feladatvégrehajtást kínálja, mindkét technológia önmagában korlátokba ütközik. A BPM-ből hiányozhat a „digitális munkás” a feladatok elvégzésére, az RPA-ból pedig a „karmester”, amely a teljes folyamatot vezényli. Erre a kihívásra válaszul született meg a Hiperautomatizálás (*Hyperautomation*) koncepciója. A Gartner [20] definíciója szerint a hiperautomatizálás „egy üzletvezérelt, fegyelmezett megközelítés, amelyet a szervezetek arra használnak, hogy gyorsan azonosítsanak, megvizsgáltak és automatizáljanak annyi üzleti és IT folyamatot, amennyi csak lehetséges.” A kulcsszó a „fegyelmezett megközelítés”: a hiperautomatizálás nem egyetlen technológia, hanem egy üzleti stratégia. Nem csupán az automatizálásról szól, hanem az automatizálás lehetőségének folyamatos felfedezéséről és menedzseléséről [21]. A hiperautomatizálás és a hagyományos automatizálás közötti különbség a fókuszban rejlik [21]:

- **Hagyományos Automatizálás:** Egyedi, szabályalapú feladatokra összpontosít. Például: Egy RPA bot automatikusan beírja a számlaadatokat egy PDF-ből az SAP-ba.
- **Hiperautomatizálás:** Végponttól végpontig tartó, komplex folyamatokra összpontosít. Például (ami megegyezik e dolgozat céljával): A folyamat magában foglalja az RPA botot, de kiegészül AI-val (amely ellenőrzi a szállítói adatokat), low-code munkafolyamattal (amely jóváhagyásra küldi a számlát), és analitikával (amely valós időben követi a költségeket).

Ennek megfelelően a hiperautomatizálás egy „eszköztár”, amely a Gartner [20] szerint több technológia „hangszerelt használatát” (*orchestrated use*) jelenti. Ez az eszköztár pontosan lefedi az automatizálás evolúciójának lépéseit, és kiegészíti azokat:

BPM (Üzleti Folyamatmenedzsment): A „karmester”, amely a teljes, végponttól végpontig tartó munkafolyamatot tervezi meg.

RPA (Robotikus Folyamatautomatizálás): A „végrehajtó”, amely a repetitív, emberi feladatokat (kattintás, gépelés) végzi.

AI és ML (Mesterséges Intelligencia): Az „agy”, amely lehetővé teszi a strukturálatlan adatok (pl. PDF-ek, e-mailek) megértését és a komplex, korábban emberi ítéletet igénylő döntések meghozatalát.

Process Mining (Folyamatbányászat): A „szemek”, amelyek a rendszerek naplófájljait elemezve valós időben feltárják a meglévő folyamatokat, azonosítják a szűk keresztmetszeteket és javaslatot tesznek új automatizálási lehetőségekre.

Low-Code és Integrációs Platformok (iPaaS): A „ragasztó”, amely összeköti a különböző, izolált rendszereket [21].

Látható, hogy az IBM [22] megkülönböztetése szerint az „Intelligens Automatizálás” (RPA + AI) csupán egy része a tágabb hiperautomatizálási stratégiának. A hiperautomatizálás célja egy olyan intelligens, önmagát optimalizáló szervezet létrehozása, ahol a folyamatok valós időben képesek alkalmazkodni a változásokhoz. A Gartner becslése szerint azok a szervezetek, amelyek a hiperautomatizálást újratervezett működési folyamatokkal kombinálják, 30%-os működési költségcsökkenést érhetnek el[21].

2.5. Az SAP Stratégiai Válasza: A Business Technology Platform (BTP)

A 2.1 alfejezetben bemutatott iparági trendekre – különösen a hiperautomatizálás iránti igényre – az SAP stratégiai válasza az SAP Business Technology Platform (BTP). Az SAP BTP egy egységes, több-felhős (*multi-cloud*) „Platform as a Service” (PaaS) megoldás, amely egyfajta központi „operációs rendszerként” funkcionál a felhőben [34, 19]. A platform célja, hogy egyetlen, átfogó környezetben integrálja, automatizálja és kiterjessze a vállalat összes üzleti alkalmazását és folyamatát,

legyen az SAP vagy nem-SAP alapú [45]. A BTP bevezetésének elsődleges célja az SAP „Clean Core” stratégiájának támogatása. Ez a megközelítés azt jelenti, hogy a központi ERP rendszert (mint az S/4HANA) a lehető leginkább standard állapotban kell tartani, és minden egyedi fejlesztést, bővítést (*extension*) vagy integrációt a BTP platformon kell megvalósítani [40, 18]. Ahelyett, hogy a vállalatok a stabil központi rendszert módosítanák, a BTP biztosít számukra egy rugalmas, felhőalapú környezetet (mint a Cloud Foundry, ABAP vagy Kyma), hogy új alkalmazásokat építsenek vagy összekössék a meglévő felhőalapú és helyi (*on-premise*) rendszereiket [12]. Ez a megközelítés garantálja a rendszerek biztonságát, átjárhatóságát (*interoperability*) és a későbbi frissítések zökkenőmentességét. E feladatok ellátására az SAP BTP öt kulcsfontosságú pillérre épül, amelyek lefedik a modern vállalatirányítás teljes technológiai spektrumát [18]:

Application Development (Alkalmazásfejlesztés): Eszközök (Pro-code, Low-Code/No-Code) új üzleti alkalmazások építésére.

Automation (Automatizálás): Szolgáltatások az üzleti folyamatok automatizálására (pl. SAP Build Process Automation).

Integration (Integráció): Eszközkészlet a különböző SAP és nem-SAP rendszerek összekötésére (pl. SAP Integration Suite).

Data and Analytics (Adat és Analitika): Adatbázis-kezelés (pl. SAP HANA Cloud) és elemzési megoldások.

AI (Mesterséges Intelligencia): Beépített AI és Gépi Tanulási képességek (pl. AI Business Services).

Ez a szakdolgozat ezen pillérek közül kiemelten az „Alkalmazásfejlesztés” és az „Automatizálás” területeire fókuszál. Ezek azok a pillérek, ahol az SAP a *Low-Code/No-Code* (LCNC) filozófiáját a legerősebben érvényesíti. Ezt a LCNC eszközkészletet az SAP az SAP Build márkanév alatt fogja össze, amely a következő alfejezet tárgya. Az SAP Build biztosítja azokat a konkrét „építőköveket”, amelyekből a dolgozat prototípusa – a folyamatmotor (BPA) és a felhasználói felület (Work Zone) – felépül.

2.6. Az SAP Build Platform: A BTP Hiperautomatizálási Eszköztára

Ahogy a **2.5 alfejezetben** láthattuk, az SAP Business Technology Platform (BTP) biztosítja azt a stratégiai, felhőalapú „operációs rendszert”, amely a „Clean Core” elvet támogatja. Ezen a platformon belül az SAP konkrét, gyakorlati válasza a **2.4 alfejezetben** bemutatott hiperautomatizálási és *Low-Code/No-Code* (LCNC) trendekre az SAP Build termékcsalád. Az SAP Build egy egységesített, LCNC (low-code/no-code) ajánlat, amely lehetővé teszi, hogy a vállalatok „drag-and-drop” egyszerűséggel hozzanak létre és bővítsenek vállalati alkalmazásokat, automatizáljanak folyamatokat és tervezzenek üzleti webhelyeket [17]. Ennek a megközelítésnek a célja

az úgynevezett „Citizen Developer” (*amatőr fejlesztő*) bevonása. A *Citizen Developer* egy olyan üzleti felhasználó, aki kiváló üzleti és folyamatismerettel rendelkezik, de limitált vagy semmilyen programozási tudása nincs, mégis képes az IT által jóváhagyott eszközökkel saját alkalmazásokat építeni [17]. Ez a megközelítés tehermentesíti a professzionális IT-fejlesztőket is, akik az alapvető fejlesztési munka gyorsításával a bonyolultabb feladatokra fókuszálhatnak. Az SAP Build platform több komponensből áll, mint például az SAP Build Apps (webes és mobilalkalmazások vizuális építésére) vagy az SAP Build Code (AI-támogatott professzionális fejlesztői környezet). Ez a szakdolgozat ezen eszköztár két kulcselemére fókuszál, amelyek együttesen biztosítják a prototípus működését: az SAP Build Process Automation-re, mint a folyamatot vezérlő „motorra”, és az SAP Build Work Zone-ra, mint a felhasználói felületet biztosító „műszerfalra”. A következő alfejezetek ezeket az eszközöket mutatják be részletesen.

2.6.1. A „Motor”: SAP Build Process Automation (BPA) Eszköztára

Míg az SAP Build a hiperautomatizálási stratégia „márkája”, addig annak központi motorja, és e szakdolgozat prototípusának technológiai magja, az SAP Build Process Automation (BPA). A BPA egy egységesített, felhőalapú platform, amely az SAP korábbi, különálló szolgáltatásait – név szerint az SAP Workflow Management-et (amely a BPM képességeket biztosította) és az SAP Intelligent RPA-t (amely a bot-fejlesztésért felelt) – egyetlen, no-code/low-code környezetben egyesíti [13]. A platform célja, hogy a **2.1 alfejezetben** bemutatott hiperautomatizálási „eszköztárat” a „Citizen Developer”-ek (üzleti felhasználók) számára is elérhetővé tegye. A prototípus megépítéséhez a BPA eszköztárának következő öt kulcskomponensét alkalmazzuk:

Process Builder (Folyamattervező): A „Karmester” A BPA központi komponense a Process Builder, amely a folyamatorchestrációt végzi. Analógiával élve ez tekinthető a folyamat „karmesterének”, amely a teljes, végponttól végpontig tartó számlafeldolgozási folyamatot vezényli. Egy intuitív, „drag-and-drop” grafikus felületet biztosít [43], amely az iparági szabvány BPMN 2.0 (*Business Process Model and Notation*) jelölésrendszert használja. A prototípusban itt építjük fel a számla teljes útját, amely logikai sorrendbe fűzi az összes többi komponenst: meghívja az AI-t az adatok kinyerésére, elküldi az adatokat a döntési táblának, kiosztja a feladatot a jóváhagyónak, és végül elindítja a botot a parkoláshoz.

Automations (Automatizálások – RPA Botok): A „Digitális Munkás” Ez a platform RPA motorja, a „digitális munkás”, amely a ?? pontban definiált taktikai feladatvégrehajtást végzi. Lehetővé teszi „attended” (felügyelt) és „unattended” (felügyelet nélküli) szoftverrobotok fejlesztését, amelyek az emberi, repetitív feladatokat (pl. adatbevitel, másolás-illesztés) utánozzák [43]. Míg a Process Builder a folyamatot irányítja, az Automation a feladatot hajtja végre. A prototípusban ez a komponens felel a 8. lépésért: egy „unattended” bot szimulálja a felhasználót, aki belép az SAP rendszerbe és elvégzi a számla parkolását.

Forms (Űrlapok): Az Emberi Beavatkozás Felülete A hiperautomatizálás nem jelenti az ember teljes kiiktatását; a cél a hatékony ember-gép együttműködés. A Forms komponens biztosítja ehhez a no-code felhasználói felületet. Egy „drag-and-drop” űrlaptervezővel hozhatók létre azok az interaktív űrlapok, amelyek az emberi beavatkozást (*Human Task*) igénylő lépéseknél (pl. jóváhagyás vagy hibaellenőrzés) megjelennek a felhasználó számára [43]. A prototípusban ez az az űrlap, amelyet a jóváhagyó menedzser lát a Work Zone postafiókjában, és amelyen a kinyert számlaadatokat és a PDF képét látva meghozhatja a döntést.

Decisions (Döntések / Üzleti Szabályok): A „Logikai Központ” A prototípus egyik legfontosabb eleme a Decisions komponens. Ez a funkció lehetővé teszi a komplex üzleti logika és a szabályok (*Business Rules*) leválasztását a vizuális folyamatábráról. Ahelyett, hogy a BPMN ábrát bonyolult „ha-akkor” elágazásokkal terhelnénk, a szabályokat egy központosított, táblázatkezelőhöz hasonló felületen, úgynevezett Döntési Táblázatokban (*Decision Tables*) menedzselhetjük [43]. Ez lehetővé teszi, hogy akár az üzleti felhasználók (*Citizen Developerek*) is frissítsék a szabályokat (pl. egy új jóváhagyó felvétele) anélkül, hogy a folyamat logikájába bele kellene nyúlniuk. A prototípusban ez a komponens tárolja a jóváhagyási mátrixot (pl. „Ha az összeg > 500.000 HUF, a jóváhagyó Y”).

AI Képességek (Document AI - DOX): A „Prototípus Agya” Ez a komponens a prototípus „agya”, amely az AI-t (Mesterséges Intelligenciát) hozza a folyamatba, és közvetlenül megoldja a „swivel-chair” adatrögzítés problémáját. Bár technikailag egy különálló SAP BTP szolgáltatás (SAP AI Business Services), natívan integrálódik a BPA-ba [43]. A DOX technológia OCR (Optikai Karakterfelismerés) és Gépi Tanulás (ML) kombinációját használja arra, hogy strukturálatlan dokumentumokból, mint amilyen egy PDF számla, kinyerje a strukturált üzleti adatokat (pl. szállító, számlaszám, nettó összeg, dátum). Kulcsfontosságú funkciója, hogy minden kinyert adathoz egy *Confidence Score* (megbízhatósági pontszám) értéket rendel. Ez teszi lehetővé az intelligens folyamatvezérlést: a prototípusunkban a magas (pl. 99% feletti) pontszámú számlák „érintésmentesen” (*touchless*) mehetnek tovább, míg az alacsony pontszámúak (pl. elmosódott szkennelés) automatikusan egy emberi javító feladatot generálnak (a Forms komponens segítségével).

2.6.2. A „Műszerfal”: SAP Build Work Zone

Ha az SAP Build Process Automation a prototípusunk háttérben futó „motorja”, akkor az SAP Build Work Zone a felhasználói felületet biztosító „műszerfala”. Ez az SAP Build platform LCNC (low-code/no-code) eszköze, amelynek célja, hogy egyetlen, egységes „digitális munkaterületet” (*Digital Workspace*) és központi belépési pontot (*central entry point*) hozzon létre a vállalat összes felhasználója számára [8]. A Work Zone bevonása a prototípusba azért kritikusan fontos, mert közvetlenül megoldja az 1. és 3. fejezetben azonosított két legfőbb üzleti problémát: a kaotikus, e-mail alapú jóváhagyási „fekete lyukat” és az átláthatóság teljes hiányát. Ezt két fő funkcióval éri el:

Task Center / „My Inbox”: A Work Zone legfontosabb képessége a prototípus szempontjából a beépített „Task Center” (más néven „My Inbox”). Ez egy központosított, egységes feladatkezelő postafiók, amely egyetlen listában gyűjti össze a felhasználóra váró összes feladatot – függetlenül attól, hogy az melyik rendszerből érkezik [44]. Amikor az SAP BPA folyamatunk egy jóváhagyási (pl. TESZT_01) vagy egy hibajavítási (pl. TESZT_02) lépéshez ér, a feladatot nem egy e-mailbe, hanem közvetlenül a felelős felhasználó „My Inbox”-ába küldi. Ezzel megszünteti az e-mailek közötti keresgélést, és a feladatkezelést nyomon követhetővé teszi.

UI Integráció és Átláthatóság (Űrlapok és KPI Csempék): A „My Inbox” szorosan integrálódik a BPA Forms komponensével. Amikor a menedzser a feladatra kattint, a Work Zone felületén belül nyílik meg az a jóváhagyási űrlap, amelyet a **2.6.1** pontban terveztünk. Ez biztosítja az „intuitív és vonzó felhasználói élményt” [44]. Ezen felül a Work Zone lehetővé teszi egyedi „csempék” (*Cards*) és KPI mutatók létrehozását. Ezáltal egy pénzügyi ügyintéző számára egy olyan valós idejű műszerfalat építhetünk, amely mutatja a „Jóváhagyásra váró számlák számát” vagy az „Automatikus feldolgozás sikerességi rátáját”, megoldva ezzel az átláthatóság hiányának problémáját. Végül, a Work Zone stratégiai jelentőségét az adja, hogy (a BTP integrációs képességeire építve) nemcsak az SAP, hanem egyedi építésű és harmadik féltől származó (*third-party*) alkalmazások feladatait és adatait is képes egyetlen, perszonalizált és szerepkör-alapú (*role-based*) felületen megjeleníteni [8].

2.7. Fejezet Összegzése: A Platform Komponenseinek Szintézise

Látható tehát, hogyan áll össze a teljes kép: az SAP Business Technology Platform (BTP) biztosítja a stabil, „Clean Core” elvű, felhőalapú alapot és az „operációs rendszert”. Erre épül az SAP Build platform LCNC eszköztára, amely a dolgozat prototípusának két fő elemét adja. Az SAP Build Process Automation (BPA) funkcionál a megoldás „motorjaként”, amely egyetlen, integrált szolgáltatásban biztosítja a hiperautomatizáláshoz szükséges összes képességet: a folyamatvezérlést (BPM/-Process), az intelligens adatkinyerést (AI/DOX), a feladatvégrehajtást (RPA/Automation) és az üzleti logikát (Decisions). Ezt egészíti ki az SAP Build Work Zone, amely a „műszerfalként” szolgál, és a központi „My Inbox” (*Task Center*) révén egységes felhasználói felületet (UI) és hatékony feladatkezelést biztosít, megoldva az **1. fejezetben** felvázolt átláthatósági hiányosságokat és az e-mailek „fekete lyuk” problémát. Most, hogy a **2. fejezetben** részletesen megismertük az SAP modern hiperautomatizálási eszköztárát – azaz a potenciális megoldást –, a következő fejezetben azt elemezzük, hogy pontosan milyen üzleti problémákra és manuális folyamatokra nyújt ez a platform választ.

3. fejezet

Standard és Vállalati Számlafeldolgozási Folyamat Elemzése

3.1. Bevezetés: Az „As-Is” Folyamat Megértése

Ennek a fejezetnek a célja, hogy részletesen elemezze a szállítói számlafeldolgozás aktuális üzleti folyamatát, az úgynevezett „As-Is” állapotot. A probléma mélyebb megértése érdekében az elemzést két, egymásra épülő szintre bontjuk. Elsőként bemutatjuk a standard SAP folyamatot, vagyis azt a „tankönyvi modellt”, ahogyan a számlakezelés egy ideálisan konfigurált vállalatirányítási rendszerben működik. Ezt követően összevetjük az ideális állapotot a tipikus vállalati gyakorlattal: azzal a valósággal, ahogyan a folyamat a legtöbb szervezetnél zajlik, ahol a rendszer beépített képességeit gyakran manuális, e-mail alapú „foltokkal” és ad hoc megoldásokkal egészítik ki. A két modell – az elméleti és a valós – közötti eltérés, azaz a „rés” fogja pontosan kijelölni azokat a nehézségeket, költséges szűk keresztmetszeteket és hibaforrásokat, amelyekre a **4. fejezetben** tervezett és megvalósított SAP Build Process Automation prototípus célzott megoldást kínál.

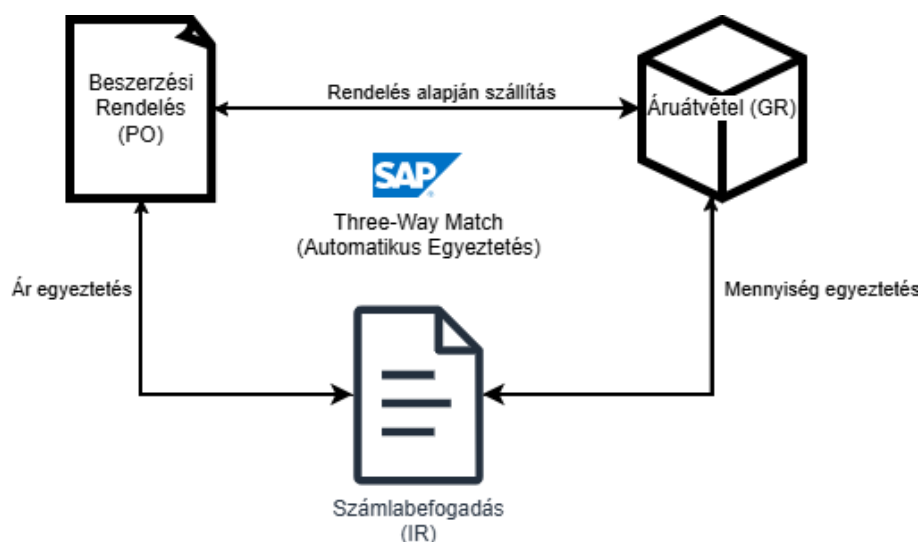
3.1.1. A Standard SAP Számlafeldolgozási Folyamat („A Tankönyvi Modell”)

A szállítói számlák kezelésének megértéséhez és értékeléséhez elengedhetetlen annak az ideális, elméleti modellnek a bemutatása, amelyet az SAP rendszer a maga teljes körűen konfigurált és integrált állapotában kínál. Ez az ún. „tankönyvi modell” szolgál majd később alapként, amellyel a valós vállalati gyakorlatot összevetjük. Ezen elméleti keret megalkotása mögött az SAP FI (Pénzügy) és MM (Anyaggazdálkodás) moduljainak zökkenőmentes együttműködése húzódik meg, mely lehetővé teszi a beszerzési folyamat teljes körű lezárását – a megrendeléstől a fizetésig. Ez az alfejezet ennek az integrált, standard folyamatnak a főbb elveit, dokumentumait és tranzakcióit mutatja be, kiemelve azokat a beépített automatizálási és ellenőrzési pontokat, amelyek a rendszer hatékonyságának és megbízhatóságának alapját képezik.

3.1.1.1. Az Alapelv: A Háromoldalú Egyeztetés (Three-Way Match)

A logisztikai számlaellenőrzés (LIV) és a teljes Procurement-to-Pay (P2P) folyamat központi alapelve a háromoldalú egyeztetés (Three-Way Match) [23, 29]. Ez a könyvelési ellenőrzési mechanizmus nem csupán technikai funkció, hanem alapvető pénzügyi biztonsági háló, melynek egyetlen célja van: biztosítani, hogy a vállalat kizárólag azért fizessen, amit előzetesen megrendelt és amit ténylegesen meg is kapott. A folyamat az alábbi három alapvető dokumentum egyidejű, automatikus összevetésén alapul, melyek együttesen lefedik a beszerzés ezen üzleti életciklusát:

- **Beszerzési Rendelés (Purchase Order - PO) - „A megállapodás”:** Ez a dokumentum (létrehozva az ME21N tranzakcióval) tartalmazza a vállalat szándékát és a szállítóval kötött formális megállapodást: mit rendeltünk, milyen áron és milyen feltételek mellett? A PO határozza meg az elvárásokat.
- **Anyagbevételezés (Goods Receipt - GR) - „A fizikai átvétel”:** Ez a bizonylat (rögzítve a MIGO tranzakcióval) igazolja a tényleges teljesítést: mit kaptunk meg a raktárban valójában? A GR nemcsak a készletnövekedést okozza, hanem egyben a szállítói kötelezettség előkészítését is jelenti a GR/IR (Goods Receipt/Invoice Receipt) számlán.
- **Számlabefogadás (Invoice Receipt - IR) - „A pénzügyi követelés”:** A szállító által küldött számla (feldolgozva a MIRO tranzakcióban) tartalmazza a fizetési igényt: miről és mekkora összegben kér fizetést a szállító?



3.1. ábra. A háromoldalú egyeztetés (Three-Way Match) elvi sémája: PO, GR és IR kapcsolat.

A háromoldalú egyeztetés folyamatának logikája a MIRO tranzakcióban materializálódik. Amikor a pénzügyi ügyintéző egy PO-hoz hivatkozó számlát rögzít, az SAP rendszer automatikusan és **szigorúan** összeveti a számla minden egyes tételének mennyiségét és árát a hivatkozott Beszerzési Rendelés (PO) és az arra történt Anyagbevételezés (GR) adataival. Ez a „három szintű” egyeztetés (Three Levels)

gyakorlatilag a GR/IR folyamat magja. Az egyezés hiánya esetén a rendszer a konfigurált toleranciahatárok alapján akár automatikusan blokkolhatja a számlát fizetésre, amelyet csak a felelős beszerző (Buyer) oldhat fel (például az MRBR jelentés segítségével), ezzel biztosítva a problémák azonosítását és megoldását. A folyamat stratégiai jelentőségét az adja, hogy hatékonyan megakadályozza a gyakori operatív kockázatokat, mint például a duplikált vagy hibás számlák, a nem rendelés alapján történt szállítások, vagy a nem átvett áruk kifizetése [23]. A háromoldalú egyeztetés tehát nem csupán egy technikai lépés, hanem a vállalat pénzügyi integritásának és a beszerzési folyamatok épségének záloga.

3.1.1.2. A Folyamat Két Fő Típusa az SAP-ban

Az SAP rendszerében a szállítói számlák feldolgozásának módját alapvetően az határozza meg, hogy a számla kapcsolódik-e egy létező beszerzési rendeléshez (Purchase Order - PO) [26, 9]. Ez a megkülönböztetés meghatározza a használandó tranzakciót, az ellenőrzési folyamatot és a feldolgozás automatizáltsági fokát. A két alapvető típus a PO-alapú (logisztikai) és a nem PO-alapú (pénzügyi) számlarögzítés.

1. PO-alapú (Logisztikai) Számlarögzítés (MIRO tranzakció) [26] Ez a folyamat a Logisztikai Számlaellenőrzés (LIV) magja, és az anyagok, szolgáltatások szabályozott beszerzésére jellemző.

- **Alkalmazási kör:** Akkor használatos, amikor a számla egy létező és érvényes beszerzési rendeléshez (PO) kapcsolódik. Ez jellemzően alapanyagok, késztermékek, alkatrészek beszerzésére, valamint olyan szolgáltatásokra vonatkozik, amelyeket előre leegyeztettek és PO-n rögzítettek (pl. külső szakértői munka, karbantartási szerződés).
- **Feldolgozás lépései:**
 1. Az ügyintéző a MIRO tranzakcióban megadja a beszerzési rendelés számát.
 2. A rendszer azonnal automatikusan feltölti a számla tételeit a PO és a hozzá tartozó áruátvételi (GR) adatok alapján. Megjelenik a rendelés összes nyitott tétele, a már átvett mennyiségek és a megállapodott árak.
 3. Az ügyintéző feladata lecsökken a fejadatok (számlaszám, dátum, fizetési feltételek) és a lehetséges eltérések (pl. minimális mennyiségi vagy árkülönbözet) kezelésére. A rendszer a „háromoldalú egyeztetés” elvével automatikusan összeveti a számla, a PO és a GR adatait.
- **Eredmény és kontroll:** A folyamat végeredménye egy automatikusan ellenőrzött és könyvelhető számla. A rendszer kiszámolja a különbözetet (ha van), és ha az a konfigurált toleranciahatárokon belül van, a számla fizetésre jóváhagyott státuszt kap. Ha az eltérés túl nagy, a rendszer fizetési blokkot állít be, amelyet csak a felelős beszerző v. vezető oldhat fel. Ez a folyamat biztosítja, hogy a vállalat kizárólag a megrendelt és átvett árukért vagy szolgáltatásokért fizet.

2. Nem PO-alapú (Pénzügyi) Számlarögzítés (FB60 tranzakció) [9] Ezt a folyamatot olyan kiadásokra használják, amelyeket nem előzték meg formális beszerzési eljárással és PO-kiállítással.

- **Alkalmazási kör:** Akkor kerül sor erre, amikor a számla nem köthető egyetlen beszerzési rendeléshez sem. Ilyen tipikusan a közüzemi számlák (villany, víz, gáz), bérleti díjak, általános szolgáltatások (pl. jogi tanácsadás, reklámkampány) vagy egyéb, előre nem tervezett üzleti kiadások.
- **Feldolgozás lépései:**
 1. Az ügyintéző az FB60 tranzakcióban közvetlenül rögzíti a számla adatait.
 2. Mivel nincs PO vagy GR, amire a rendszer hivatkozhatna, az automatikus adatfelület és egyeztetés hiányzik.
 3. **Kritikus manuális lépés:** Az ügyintézőnek teljes mértékben manuálisan kell elvégeznie a kontírozást. Ez azt jelenti, hogy minden egyes számlatételhez ki kell választania a megfelelő főkönyvi számlát (amely meghatározza a költségnemet, pl. „Villamos energia költsége”) és a költséghelyet (amely meghatározza, hogy melyik osztály vagy projekt viseli a költséget, pl. „Épületüzemeltetés” vagy „Marketing Osztály”).
- **Eredmény és kockázat:** A folyamat egy közvetlenül a főkönyvbe könyvelt számlával zárul. A feldolgozás jelentősen munkaigényesebb és hibákra érzékenyebb, mivel nem történik automatikus egyeztetés. A pontosság teljes mértékben az ügyintéző szakértelmén és figyelmességén múlik. Egy helytelen költséghely vagy főkönyvi számla megadása torzított költségelszámoláshoz és helytelen pénzügyi kimutatásokhoz vezet. Emiatt ezek a számlák gyakran több lépcsős belső jóváhagyási folyamaton esnek át, mielőtt kifizetésre kerülnének.

Szempont	PO-alapú (MIRO)	Nem PO-alapú (FB60)
Tranzakció	MIRO	FB60
Elv	Háromoldalú egyeztetés (PO-GR-Számla)	Közvetlen pénzügyi könyvelés
Automatizáció	Magas (rendszer javasol adatokat)	Alacsony (manuális kontírozás)
Kontroll	Erős (rendszer blokkol eltéréseket)	A folyamat és a jóváhagyás feladata
Kockázat	Alacsony	Magas (emberi hiba)
Példa	Alumínium öntvény beszerzése	Villanyszámla, Hirdetési költség

3.1. táblázat. Összefoglaló táblázat a PO-alapú és nem PO-alapú számlafeldolgozás összehasonlításához

3.1.2. Standard Automatizálási és Manuális Vezérlési Pontok

Az SAP rendszer kifinomultságát az is jelzi, hogy alapkiépítésében is tartalmaz jól meghatározott automatizálási mechanizmusokat és tudatosan kialakított manuális

beavatkozási pontokat. Ez a kettősség biztosítja a folyamatok hatékonyságát anélkül, hogy veszélyeztetné a pénzügyi kontrollok és a döntéshozatali lehetőség integritását. Az automatizmusok a rutin feladatok gyors és hibamentes elvégzését szolgálják, míg a manuális „fékek” lehetővé teszik a szükséges szakértői felülvizsgálatot kivételes helyzetekben.

3.1.2.1. Beépített Automatizációs Lehetőségek

Az SAP rendszer a számlafeldolgozás hatékonyságának és biztonságának növelése érdekében számos beépített automatizálási mechanizmust kínál, amelyek lehetővé teszik a manuális ellenőrzések mértékének csökkentését, miközben megtartják a pénzügyi kontrollok szilárd keretét. Ezek az automatizmusok a folyamatok gyorsításán túl a humán hibák kiküszöbölésére és a költségek csökkentésére is összpontosítanak.

Toleranciahatárok (Tolerances) [35, 36] A toleranciahatárok az SAP egyik legkifinomultabb kontrollmechanizmusa, amely kétjegyű kódok (toleranciakulcsok) formájában valósul meg, és amelyek a rendszer által tárolt előre meghatározott korlátok alapján automatikusan blokkolják a számlákat fizetésre, amennyiben azok ára, mennyisége vagy egyéb paraméterei túllépik a megengedett határértékeket. A mechanizmus nem csupán védelmet nyújt a túlfizetések ellen, hanem jelentős mértékben lecsökkenti a pénzügyi munkatársak terhelését is azáltal, hogy kiküszöböli a minden egyes számla egyedi manuális ellenőrzésének szükségességét. A konfiguráció a T169G táblában történik, és a különböző üzleti helyzetekre szabottan több tucat speciális toleranciakulcs áll rendelkezésre. A legfontosabb toleranciakulcsok közé tartozik a PP (Price Variance), amely a beszerzési rendelésen megállapított és a számlán szereplő egységár közötti eltérést ellenőrzi, vagy a DQ (Quantity Variance), amely a ténylegesen átvett és a számlázott mennyiség közötti különbséget monitorozza. Azonban a rendszer ennél jóval részletesebb kontrollokat is biztosít: a BD (Small Differences) kulcs például automatikusan elszámolja a minimális összegű eltéréseket egy előre meghatározott „kis különbségek” főkönyvi számlára, miközben a KW (Variance from Condition Value) specifikusan a szállítási és egyéb mellék költségek egyeztetésére specializálódott. A folyamat teljességét az MRBR tranzakció biztosítja, amely egy központosított felületet kínál az automatikusan blokkolt számlák áttekintéséhez, okának feltárásához és – adott esetben – manuális feloldásához.

ERS (Evaluated Receipt Settlement) [11] Az Evaluated Receipt Settlement (ERS) a számlafeldolgozási automatizáció csúcspontját jelentő, radikálisan innovatív megközelítés, amely alapvetően újradefiniálja a hagyományos számlázási folyamatot. Az ERS lényege, hogy a szállító és a vevő közötti előzetes megállapodás alapján a szállító egyáltalán nem küld fizikai vagy elektronikus számlát. Ehelyett az SAP rendszer kizárólag a beszerzési rendelés (PO) és a tényleges áruátvétel (MIGO) adatai alapján automatikusan generálja és könyveli a számlabizonylatot a megállapodott árak és feltételek figyelembevételével. Ennek a módszernek az alkalmazása számos stratégiai előnnyel jár: a beszerzési ciklus ideje jelentősen lecsökken, hiszen a számla kézbesítésére és feldolgozására várási idő teljesen megszűnik. Emellett a rendszer kiküszöböli a kézi adatrögzítésből adódó emberi hibákat, és megelőzi a mennyiségi vagy értékbeli eltérések kialakulását, mivel a számla mindig a PO és a GR pontos

adatai alapján készül. Az ERS különösen hatékonyan alkalmazható olyan szabványosított termékek és rutinszerű szolgáltatások esetében, ahol a szállítóival megbízható partneri kapcsolatot ápol, és a díjszabás stabil. Fontos megjegyezni, hogy az ERS nem alkalmazható olyan esetekben, ahol a rendszeresítés tervezett vagy nem tervezett szállítási költségeket tartalmaz.

EDI/IDoc (Electronic Data Interchange / Intermediate Document) [32]

Az elektronikus adatcsere (EDI) az üzleti kommunikáció modern standardja, amely strukturált formátumban történő, rendszer és rendszer közötti adatcserét tesz lehetővé, ezzel teljes mértékben kiküszöbölve a papír alapú vagy emberi beavatkozással járó adatátvitelt. Az SAP rendszerben ezt a folyamatot az úgynevezett Intermediate Document (IDoc) technológia valósítja meg, amely speciális üzenetformátumként szolgál a számlaadatok továbbítására. Amikor egy szállító EDI-n keresztül küld számlát, az azt reprezentáló IDoc üzenet közvetlenül az SAP rendszerbe érkezik, ahol a rendszer automatikusan megkísérli a számla könyvelését a beszerzési rendelés előzményei alapján meghatározott javasolt mennyiség és érték figyelembevételével. A folyamat során a rendszer a Customizing beállításainak megfelelően kezeli az esetleges eltéréseket: fizetési blokkolással, parkolással (invoice parking) vagy akár a rendszer által javasolt értékek használatával is történhet a könyvelés. Az EDI/IDoc integráció nemcsak a feldolgozási sebességet növeli meg és csökkenti a adminisztratív terheket, hanem az adatpontosságot is maximalizálja, miközben teljes körű audit nyomvonalat biztosít a bejövő dokumentumokról.

3.1.2.2. Tudatos Manuális Beavatkozási Pontok

Míg az SAP rendszer kiterjedt automatizálási lehetőségeket kínál, a gyakorlatban számos olyan stratégiai pont van, ahol a tudatos manuális beavatkozás elkerülhetetlen és egyben értéket teremtő. Ezek a pontok nem a rendszer hiányosságait jelzik, hanem éppen ellenőleg, olyan tervezett „fékeket” és ellenőrzési mechanizmusokat tesztítenek meg, amelyek lehetővé teszik a szükséges szakértői értékelést és döntéshozatalt a pénzügyi folyamatok kulcspontjain. Ezek a manuális intervenciók biztosítják, hogy a rendszer rugalmassága megmaradjon a valós üzleti helyzetek kezelésében, miközben a pénzügyi kontrollok integritása sértetlen marad.

Manuális Számlarögzítés: Az Adatátvitel Kritikus Szűk keresztmetszete

A számlafeldolgozási lánc legjelentősebb manuális teherpróbája akkor következik be, amikor a számla nem elektronikus adatcsere (EDI) útján, hanem hagyományos papír alapon vagy strukturálatlan PDF formátumban érkezik. Ebben az esetben az ügyintézőnek teljes mértékben manuálisan kell átvinnie a számla összes releváns adatát - számlaszám, dátum, szállítói adatok, tételek, árak, adók - az SAP rendszerbe a MIRO (beszerzési rendeléshez kapcsolódó) vagy FB60 (közvetlen könyvelésű) tranzakciók segítségével. Ez a folyamat, amelyet gyakran „forgószék-adaptációként” („swivel-chair integration”) ismernek, nem csupán időigényes és költséges, hanem kiemelten ki van téve az emberi hibáknak is. Az elgépelések, számtranszpozíciók vagy hiányzó adatok komoly pénzügyi következményekkel járhatnak, beleértve a duplikált fizetéseket, a helytelen költségelszámolást vagy a meg nem érdemelt kötbérek kifizetését. Jóllehet, ez a manuális lépés jelenleg még mindig elkerülhetetlen a legtöbb

szervezetnél, és egyben a legfontosabb indoklást szolgáltatja a hiperautomatizálási megoldások, mint például az intelligens adatkinyerés (Document AI) bevezetésére.

Számlaparkolás: A Kontrollált „Félkész” Állapot Művészete [30, 25, 37]

A számlaparkolás (MIR7 vagy FV60 tranzakciók) egy kifinomult köztes állapot, amely lehetővé teszi a számla adatainak biztonságos elmentését anélkül, hogy az végleges főkönyvi könyvelést generálna. Ezt a funkcionalitást leginkább egy „fizetésre előkészített” vagy „függőben lévő” státuszként lehet leírni, ahol a dokumentum minden adata rendelkezésre áll, de a tényleges könyvelés és fizetés még nem történt meg. A parkolás stratégiai jelentősége abban rejlik, hogy rugalmasságot biztosít a belső ellenőrzési folyamatok számára. Egy számlát parkolni szokás, amikor az adatai bizonytalanok vagy hiányosak, további információkra van szükség a költséghelyes elosztáshoz, belső jóváhagyási körök függenek a véglegesítéstől, vagy egyszerűen csak a fizetési időzítést szeretnék optimalizálni. A parkolt számla később bármikor elővehető, akár egy másik, magasabb szintű felhasználó (például egy költséghely-vezető vagy pénzügyi kontroller) által, aki átnézheti, kiegészítheti, korrigálhatja, és végül „élesre” könyvelheti azt. Ez a folyamat biztosítja, hogy csak a teljes körűen ellenőrzött és hivatalosan jóváhagyott számlák kerüljenek a vállalat főkönyvébe, ezzel erősítve a belső kontrollrendszer megbízhatóságát.

Blokkolt Számlák Feloldása: Az Eltérések Menedzsmentje [27] Az SAP rendszer intelligens kontrollmechanizmusa, különösen a toleranciahatárok, gyakran automatikus fizetési blokkot alkalmaz azokon a számlákon, amelyek ára, mennyisége vagy egyéb paraméterei túllépik az előre meghatározott korlátokat. Amikor egy ilyen blokk létrejön a MIRO tranzakció során, a rendszer ugyan létrehozza a számlabizonylatot, de azt egy speciális „fizetésre blokkolt” státuszba helyezi, megakadályozva ezzel a pénztári futtatás (payment run) során történő kifizetését. A blokk feloldása nem automatizált, hanem egy tudatos, felelősségteljes manuális lépés, amelyet általában a MRBR tranzakció (Feladáslistája a számláknak, blokk feloldással) segítségével hajtanak végre. Ez a folyamat feltétlenül megköveteli, hogy egy előre meghatározott, illetékes felelős (legtöbbször a beszerző, aki ismeri a megrendelés hátterét, vagy egy pénzügyi vezető) személyesen felülvizsgálja az eltérés okát. Ennek során értékeli, hogy az eltérés indokolt-e (például egy egyeztetett áremelkedés, szállítási problémából adódó többletköltség), dokumentálja a döntés indoklását, és végül manuálisan jóváhagyja a fizetést. Ez a fék mechanizmus rendkívül értékes, mivel megakadályozza a jelentős vagy gyanús eltérésekkel rendelkező számlák automatikus kifizetését, és lehetővé teszi, hogy a megfelelő szakértői tudás bevonásával szolgáltatassák ki a végső döntést. Egyes toleranciakulcsok (például DQ vagy DW) esetén a blokk automatikusan is feloldódhat, ha a mögöttes probléma megszűnik (például a hiányzó áruátvétel rögzítésre kerül), de a legtöbb kritikusabb blokk (például PP áreltérés) kifejezetten emberi beavatkozást igényel a feloldásához.

3.1.3. Egy Tipikus Vállalati Gyakorlat Elemzése („A Valós Világ”)

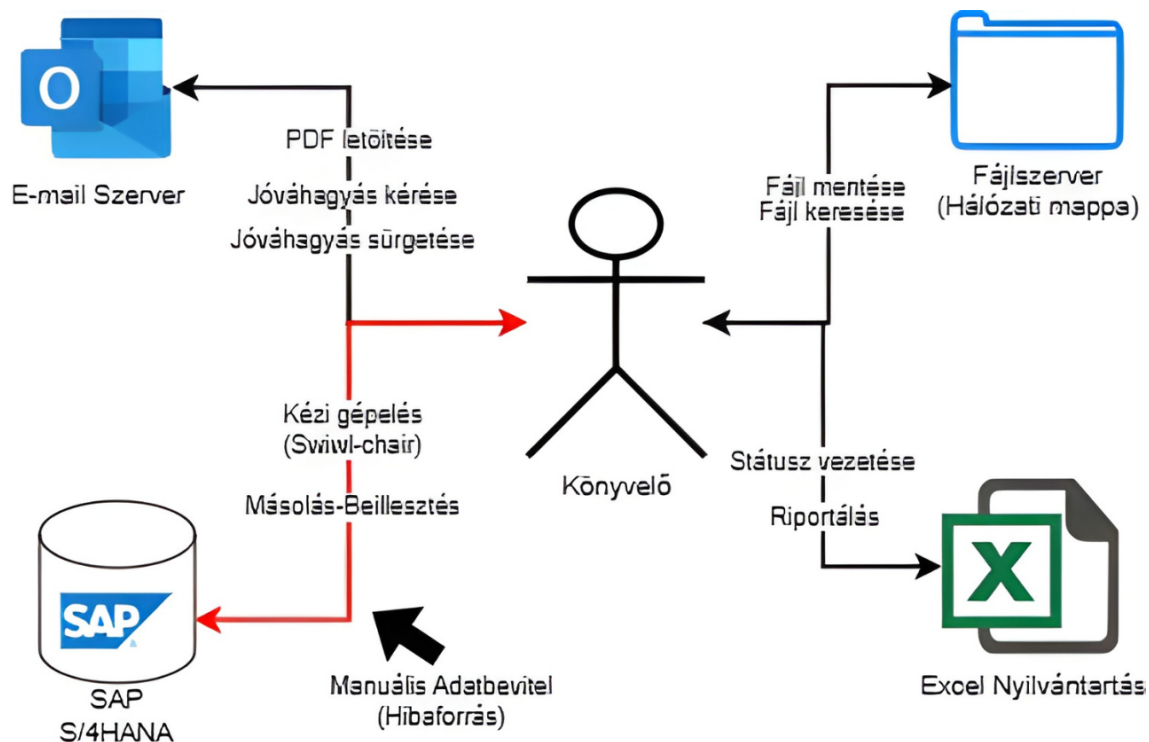
Az SAP rendszer elméleti keretei és beépített automatizálási lehetőségei (3.1.2.1) dacára a valós vállalati gyakorlat gyakran jelentős eltéréseket mutat az ideális „tan-

könyvi modelltől”. Ebben a fejezetben egy fiktív, de a gyakorlatban jól megfigyelhető tipikus vállalati környezetet („As-Is” állapot) elemzünk, amely bár technikailag rendelkezik SAP licenccel és alapvető konfigurációval, a fejlett automatizálási eszközök - mint például az SAP Build Process Automation - hiánya, valamint a történelmi teherként jelentkező elavult munkafolyamatok miatt nem képes kihasználni a rendszer teljes potencialitását. Ez az ellentmondás a technológia és a valós üzleti gyakorlat közötti szakadékot testesíti meg, amely számos operatív és stratégiai kihívást generál. A vizsgált modell alapvető jellemzője a digitális és analóg folyamatok hibrid együttélése. Míg a végső könyvelési lépések az SAP rendszerében történnek, addig a számlafeldolgozás előkészítő, ellenőrzési és jóváhagyási fázisai túlnyomórészt az SAP-on kívüli, manuális csatornákon keresztül zajlanak. Ennek a hibrid modellnek a működését egy jól körülhatárolt, ismétlődő ciklus jellemzi, amely a következő kritikus lépésekből áll:

1. **Beérkezés: A Digitális Postafiók Kaotikus Világa** A folyamat kezdete egy dedikált, de gyakran túlterhelt e-mail címre történik (pl. szamlak@vallalat.hu), ahová a számlák mintegy 90%-a PDF formátumban érkezik. Ez a központi gyűjtőpont azonban gyakran átláthatatlan „digitális fekete lyukká” válik, ahol a dokumentumok könnyen elveszhetnek a nagy mennyiségű beérkező levél között. A probléma mélyebb gyökere a PDF formátum használatában rejlik - bár digitális, ez a formátum alapvetően strukturálatlan, és nem tartalmaz géppel olvasható adatokat, ami lehetetlenné teszi a közvetlen rendszerintegrációt. Ahogyan Anichshuk (2025) is rámutat [4], „a PDF egy statikus dokumentum, nem pedig strukturált adatokat tartalmazó fájl, amelyet a rendszerek képesek lennének olvasni”, ami elkerülhetetlenül manuális adatátvitelhez vezet.
2. **Manuális Szelektálás és Mentés: Az Adminisztratív Teher Első Szintje** Egy pénzügyi adminisztrátornak (AP ügyintéző) folyamatosan monitoroznia kell a postafiókot, manuálisan ki kell válogatnia a számlákat, letöltenie a PDF fájlokat, és azokat egy megosztott hálózati mappa előre meghatározott, de gyakran bonyolult mappaszerkezetében elhelyeznie. Ez a látszólag egyszerű lépés rendkívül időigényes, és jelentős kockázati pontot jelent: az elveszett vagy rossz helyre mentett számlák hetekig, akár hónapokig is „eltűnhetnek” a rendszerből, ami késedelmi kamatokhoz és megszakadt ellátási lánchoz vezethet. A Parseur (2025) által közölt benchmark adatok szerint a manuális feldolgozás költsége 12-20 dollár számlánként terjed, ami ezen lépések összetettségét is jelzi [3].
3. **„Swivel-Chair” Adatrögzítés: A Hibák Fő Forrása** A valódi operatív teher ekkor jelentkezik, amikor a pénzügyi ügyintéző párhuzamosan kell, hogy működjön az SAP felülete és a számla PDF-je között. Az ügyintéző egyik monitoron megnyitja a letöltött számlát, a másikon pedig az MIRO (beszerzési rendeléshez kapcsolódó) vagy FB60 (közvetlen könyvelésű) tranzakciót, majd kézzel, manuálisan átviszi az összes szükséges adatot. Ez a „forgószék-integráció” („swivel-chair integration”) nemcsak rendkívül lassú (átlagosan 10-30 perc számlánként), de kiemelten ki van téve az emberi hibáknak is. Az elgépelt számok, a rosszul másolt dátumok vagy a téves költséghely kijelölések

komoly pénzügyi következményekkel járhatnak, beleértve a duplikált fizetéseket és a helytelen költségelszámolást.

4. **Jóváhagyási Kör: A Kommunikációs „Fekete Lyuk”** Amikor egy számla belső jóváhagyást igényel - legyen szó nem PO-alapú kiadásról (FB60) vagy a toleranciahatárok túllépéséről (MIRO) - a rendszer nem egy integrált workflow-ot indít el. Ehelyett a felelős ügyintézőnek manuálisan kell parkolnia a számlát (FV60), majd e-mailben megkeresnie a megfelelő jóváhagyót (osztályvezetőt vagy beszerzőt), csatolva a parkolt bizonylat számát és a PDF számlát. Ez a folyamat azonban gyakran összeomlik a valóságban: a vezetők, akik naponta több tucat hasonló kérést kapnak, nem tudják prioritizálni a számlajóváhagyásokat. Ahogyan Kumar Mishra és munkatársai (2024) megállapították, a manuális intervenció igénybevétele jelentős időráfordítással jár, és a folyamat átláthatatlansága tovább rontja a helyzetet [27].
5. **Követés és Várakozás: Az Utánkövetés Ciklikus Folyamata** A vezetők elfoglaltsága és az e-mailek eltévedése miatt az AP ügyintézőnek napokkal, hetekkel később telefonon vagy újabb e-mailben kell „üldöznie” („chasing”) a jóváhagyást. Ez az úgynevezett „e-mail üldözéses” folyamat nemcsak további adminisztratív terhet jelent, hanem komoly üzleti kockázatokhoz is vezethet. A lassú átfutási idő miatt a vállalat gyakran nem tud élni a korai fizetési kedvezményekkel (skontó), ami jelentős pénzügyi veszteséget okoz, valamint késedelmi kamatok kifizetésére kényszerülhet. A folyamat átláthatatlansága miatt a menedzsment nem rendelkezik valós idejű betekintéssel sem a folyamatban lévő számlák állapotába, sem a vállalat aktuális kötelezettségállományába.
6. **Manuális Véglegesítés: A Bizonytalan Lezárás** A folyamat végén a vezető informális válasza (pl. rövid e-mail jóváhagyás) alapján - amely önmagában is audit kockázatot jelent - az AP ügyintéző megkeresi és megnyitja a parkolt bizonylatot, majd azt „élesre” könyveli. Ez a végső manuális lépés azonban további hibalehetőségeket hordoz: a rossz bizonylatszám megadása, a téves összeg beírása vagy egyszerűen csak az elavult információkon alapuló döntések további problémákhoz vezethetnek.



3.2. ábra. A manuális, hibrid számlafeldolgozási folyamat „Spaghetti Diagramja”, amely szemlélteti a rendszerek közötti kaotikus adatmozgást.

3.1.4. Problémafeltárás: A Manuális Gyakorlat Hibái és Költségei

Ez a fejezet a szakdolgozat egyik legfontosabb elemzése, amely közvetlenül megindokolja a **4. fejezetben** bemutatásra kerülő automatizált prototípus szükségességét. A manuális számlafeldolgozás nem csupán hatékonysági problémákat okoz, hanem jelentős pénzügyi kockázatokat és stratégiai akadályokat is jelent a vállalatok számára. Az alábbiakban részletesen feltárjuk a hagyományos megközelítés minőségi és mennyiségi korlátait.

3.1.4.1. Tipikus Manuális Hibák (Minőségi Kockázatok)

A manuális folyamatok elkerülhetetlenül vezetnek olyan minőségi problémákhoz, amelyek közvetlen pénzügyi veszteségeket és működési kockázatokat eredményeznek.

Adatrögzítési hibák (Elgépelések) A „forgószék” feladat, ahol az ügyintéző párhuzamosan dolgozik a PDF számla és az SAP felület között, kiemelten ki van téve az emberi hibáknak. Az elgépelések - mint például 100.000 helyett 1.000.000 forint beírása, vagy dátumok felcserélése - nem csupán adminisztratív problémát jelentenek. A Parseur kutatása szerint a manuálisan feldolgozott számláknál 1-2%-os hibaaránya teljesen általános, ami nagy volumenű feldolgozás esetén komoly pénzügyi következményekkel jár [3]. A ResolvePay által közölt adatok még aggasztóbb képet festenek: a számlák ellenőrzése során az AP szakemberek ezeknek csupán 39%-át képesek észlelni[1].

Duplikált Számlafeldolgozás A gyakorlatban előfordul, hogy a szállító ugyanazt a számlát több csatornán (e-mail, posta) is elküldi, vagy különböző ügyintézők dolgozzák fel ugyanazt a dokumentumot. Amikor a számlaszámot kissé eltérően rögzítik (pl. „SZ-100” helyett „SZ100”), a standard SAP szűrési mechanizmus nem képes felismerni a duplikációt. Ennek eredményeképpen a vállalat ugyanazt a számlát kétszer fizeti ki, ami jelentős pénzügyi veszteséget okoz. Tamaro szerint a vállalatok mintegy egyharmada küzd duplikált fizetések problémájával, ami a manuális folyamatok alacsony megbízhatóságára utal [39].

Elveszett Számlák Az e-mail alapú feldolgozás során a számlák könnyen elveszhetnek: a spam mappába kerülhetnek, a hálózati meghajtó rossz mappájába menthetik őket, vagy egyszerűen „eltűnhetnek” a túlterhelt e-mail fiókokban. Ferro rámutat, hogy az e-mailes jóváhagyási rendszerben „az AP részleg soha nem lehet biztos abban, hogy egy számla hol tart a folyamatban” [14]. Az elveszett számlák nemcsak késedelmi kamatokhoz vezetnek, hanem megingatják a szállítói kapcsolatokat is, hiszen a szállítók idővel elégedetlenné válnak az állandó fizetési késések miatt.

3.1.4.2. Időigényes Lépések és Szűk Keresztmetszetek (Költség- és Időproblémák)

A minőségi problémák mellett a manuális folyamatok jelentős mennyiségi és pénzügyi terheket is rónak a vállalatokra.

1. **Manuális Adatrögzítés (FTE Költség)** A manuális adatrögzítés jelentése az egyik legnagyobb költségtényező. Bár számlánként csupán 3-5 percnyi gépelésről van szó, ez nagy volumenű feldolgozás esetén teljes munkaidős pozíciók (FTE) megtartását igényli. Baran számítása szerint egy évente 40.000 számlát feldolgozó vállalatnál a számlánkénti 5 perc több mint 3.300 óra munkaidőt jelent évente [6]. Összehasonlításképp: egy AI-alapú megoldás (mint az SAP Document AI) ugyanezt a feladatot másodpercek alatt elvégezheti. Parseur adatai szerint a manuális feldolgozás költsége 10-15 dollár számlánként terjed, míg az automatizált megoldások ezt 2-3 dollárra csökkenthetik [3].
2. **Jóváhagyóra Várakozás (Átfutási Idő)** Az e-mail alapú jóváhagyási folyamat a leggyakrabban előforduló szűk keresztmetszet. Ferro kiemeli, hogy a vezetők naponta átlagosan 100 e-mailt kapnak, amelyek között az aprobálási kérések könnyen elvesznek [14]. Az e-mailes folyamat „fekete lyukként” működik: nincs átláthatóság arról, hogy ki látta a számlát, mikor vette át, és mennyi időre van szüksége a döntéshez. A várakozási idő tovább növekszik, amikor többszörös jóváhagyási körök szükségesek, és a felelősök nem ismerik a vállalati szabályokat. Tamaro szerint a manuális feldolgozás átlagos átfutási ideje 14.6 nap, ami kritikusan lassú a modern üzleti környezetben [39].
3. **Elvesztett Skontó (Pénzügyi Veszteség)** A lassú átfutási idő közvetlen pénzügyi következménye az elvesztett korai fizetési kedvezmények (skontók). Amikor egy számla hetekig várakozik a jóváhagyásra, a vállalat nem tud élni a 2-5%-os árengedményekkel, amelyek jelentős összegeket jelentenek éves szinten. Parseur egy esettanulmányában egy kiskereskedelmi vállalat évi 50.000 dollárt fizetett késedelmi bírságokként, részben az elvesztett skontók miatt [3].

Baran is hangsúlyozza, hogy a manuális folyamatok csökkentik a korai fizetési kedvezmények igénybevételének lehetőségét, ami közvetlen veszteséget jelent a vállalat számára [6].

4. **Átláthatóság Hiánya (Stratégiai Gát)** A manuális folyamatok talán leg-súlyosabb stratégiai hátránya az átláthatóság teljes hiánya. A menedzsment nem látja valós időben, hogy mennyi a vállalat aktuális kötelezettségállománya (liability), mivel a számlák tucatjai „keringenek” az e-mail postafiókokban és a megosztott mappákban. Ferro pontosan fogalmaz: „Az AP részleg soha nem lehet biztos abban, hogy egy számla hol tart a folyamatban” [14]. Ez az átláthatatlanság lehetetlenné teszi a pontos pénzügyi tervezést, megnehezíti a cash flow menedzsmentet, és akadályozza a pénzügyi osztályt abban, hogy stratégiai partnerként funkcionáljon az üzleti döntéshozatalban.

Összefoglalóan, a manuális számlafeldolgozás nem csupán hatékonysági problémát jelent, hanem komoly pénzügyi kockázatokat és stratégiai akadályokat is. A fenti problémák együttesen indokolják a **4. fejezetben** bemutatásra kerülő hiperautomatizálási megoldás megvalósításának szükségességét, amely célzottan ad választ ezekre a kihívásokra.

3.1.5. Fejezet Összegzése

A harmadik fejezet átfogó elemzése egyértelműen rámutat a szállítói számlafeldolgozás modern vállalati gyakorlatában rejlő fundamentális szakadéokra. Egyrészt bemutattuk az SAP rendszer által kínált ideális, „tankönyvi” folyamatot, amely a **háromoldalú egyeztetés** elvén, a **PO-alapú (MIRO)** és **nem PO-alapú (FB60)** számlák szisztematikus feldolgozásán, valamint a **beépített automatizálási eszközökön** – mint például a **toleranciahatárok**, az **ERS** és az **EDI/IDoc** – keresztül magas fokú hatékonyságot, kontrollt és biztonságot kínál. Ez a modell elméletileg képes a humán hibák minimalizálására és a folyamatok optimális lebonyolítására. Másrészt, a valóságot vizsgálva – a tipikus vállalati gyakorlatot („As-Is”) – egy lényegesen kevésbé hatékony és kockázatosabb kép rajzolódott ki. Itt a technológiai keretrendszer és a tényleges működés között óriási rés tátong. A folyamat kritikus szakaszai – a számlák **beérkeztetése (intake)**, a manuális **adatrögzítés** („swivel-chair integration”), valamint a belső **jóváhagyási körök** – túlnyomórészt az SAP-on kívül, e-mail alapú, strukturálatlan és erőforrás-igényes manuális tevékenységekbe torkollnak. Ez a hibrid modell számos súlyos problémát generál, melyeket a **3.1.4.** pont részletesen feltárt:

- **Minőségi kockázatok:** Megbízhatatlan adatátvitel, **adathibák**, **duplikált fizetések** és **elveszett számlák**, melyek közvetlen pénzügyi veszteségekhez vezetnek.
- **Operatív és pénzügyi költségek:** A jelentős **manuális munkaerő-felhasználás (FTE költség)**, a jóváhagyási csúcsok miatti extrém **átfutási idők**, az **elvesztett korai fizetési kedvezmények (skontók)** és a késedelmi kamatok.

- **Stratégiai akadályok:** A folyamat teljes **átláthatatlansága**, ami lehetetlenné teszi a valós idejű kötelezettségállomány nyomon követését és alátámasztja a pénzügyi tervezés megbízhatóságát.

4. fejezet

Automatizált Számlafeldolgozási Prototípus Tervezése és Megvalósítása

4.1. Bevezetés: A Prototípus Céljai és Keretrendszere

A szakdolgozat gyakorlati részének célja egy olyan „end-to-end” automatizált megoldás létrehozása volt, amely képes a számlák beérkezésétől azok előkönyveléséig (parkolásáig) terjedő folyamatot emberi beavatkozás nélkül, vagy minimális felügyelettel elvégezni. A megvalósítás alapfeltétele a megfelelő felhőalapú infrastruktúra (*Infrastructure-as-a-Service* / *Platform-as-a-Service*) kiépítése volt, amely biztosítja a szükséges számítási kapacitást és a mikroszolgáltatások elérhetőségét.

A fejlesztéshez az **SAP Business Technology Platform (BTP)** környezetét konfiguráltam. Ez nem csupán a szolgáltatások egyszerű aktiválását jelentette, hanem egy teljeskörű rendszeradminisztrációs feladatot, melynek során egy dedikált, multi-environment típusú alkörnyezetet (*Subaccount*) hoztam létre „AI Sandbox” elnevezéssel. Az infrastruktúra mögöttes szolgáltatójaként (*Provider*) az **Amazon Web Services (AWS)** került kiválasztásra, a *Frankfurt (Europe)* régióban. Ezen régió választását szakmai szempontok indokolták: egyrészt biztosítja az optimális hálózati válaszidőt (alacsony látencia), másrészt megfelel a szigorú európai adatvédelmi irányelveknek (GDPR), ami pénzügyi adatok kezelésekor kritikus szempont.

A környezet inicializálása során a következő jogosultságokat (*Entitlements*) és szolgáltatáspéldányokat (*Service Instances*) konfiguráltam a fejlesztői (*dev*) hatókörben, standard csomagok alkalmazásával:

Subaccount és Jogosultságok: A fejlesztés számára dedikált alkörnyezet létrehozása után a szükséges szolgáltatási kvóták (*Service Quotas*) hozzárendelése történt meg, biztosítva az erőforrásokat a futtatókönyezet számára.

Szolgáltatások példányosítása (Instantiation): A rendszer gerincét alkotó szolgáltatások konkrét példányainak létrehozása az alábbi technikai specifikációk szerint:

- doc-ai-srv: Az SAP Document AI szolgáltatás példánya (*standard plan*), amely a számlák intelligens, gépi tanulás alapú feldolgozásáért felel.
- sap_process_automation: Az SAP Build Process Automation szolgáltatás (*standard plan*), amely a folyamatvezérlés és a robotok futtatását végzi.
- sbwz_instance: Az SAP Build Work Zone (*standard edition*), amely a felhasználói interakciókat biztosító központi felületet (*Central Entry Point*) adja.

Megjegyzés a költségekről: A fejlesztés során az SAP BTP „Free Tier” konstrukcióját vettem igénybe, amely fejlesztési és tanulási célokra ingyenes hozzáférést biztosít a szolgáltatásokhoz. Fontos megjegyezni, hogy éles üzemi környezetben a Document AI szolgáltatás használata jellemzően tranzakció-alapú (blokkonkénti) elszámolás alá esik, ami a méretezésnél tervezendő költségtényező.

Subscriptions (10)

Applications to which your subaccount is currently subscribed

Application	Plan	Changed On	Status	
SAP AI Launchpad	standard	2 Apr 2025	Subscribed	...
SAP Build Code	standard	2 Apr 2025	Subscribed	...
Cloud Integration Automation	standard	13 Mar 2025	Subscribed	...
Joule	standard	3 Oct 2025	Subscribed	...
SAP Document AI	default	9 Oct 2025	Subscribed	...
SAP HANA Cloud	tools	1 Apr 2025	Subscribed	...
SAP Build Process Automation	build-default	3 Oct 2025	Subscribed	...
Cloud Identity Services	connectivity	25 Sept 2025	Subscribed	...
SAP Business Application Studio	build-code	2 Apr 2025	Subscribed	...
SAP Build Work Zone, standard edition	standard	29 Sept 2025	Subscribed	...

4.1. ábra. A konfigurált BTP szolgáltatáspéldányok és feliratkozások áttekintése

Biztonság és Hozzáférés (Service Keys): A szolgáltatások külső eléréséhez és az egyedi fejlesztésű szkriptek API kommunikációjához minden példány esetében *Service Key*-eket generáltam. Ezek a kulcsok tartalmazzák a hitelesítéshez elengedhetetlen OAuth 2.0 kredenciákat (*Client ID*, *Client Secret*, *Token URL*), amelyek lehetővé teszik a biztonságos, tokenszintű azonosítást.

Kapcsolatok (Connectivity): A rendszerkomponensek közötti biztonságos kommunikációt az **SAP BTP Connectivity** szolgáltatásán belül, úgynevezett *Destination* konfigurációkkal valósítottam meg. A prototípus működéséhez két kritikus kapcsolat definiálása volt szükséges (lásd 4.1. kódblokk):

S4HANA_CLOUD: Ez a kapcsolat biztosítja a hidat a folyamatautomatizálás és a vállalatirányítási backend rendszer között. A prototípus környezetben itt *Basic Authentication* hitelesítést alkalmaztam egy dedikált technikai felhasználó (SBPA_BOT_USER) segítségével.

- Kiemelten fontos a sap.processautomation.enabled tulajdonság beállítása, amely lehetővé teszi, hogy az SAP Build Process Automation közvetlenül, OData hívásokon keresztül érje el a rendszer API végpontjait.

sap_process_automation_service_workzone: Ez a konfiguráció a felhasználói felület (Work Zone) és a folyamatmotor közötti belső adatcseréért felel. Itt a magasabb biztonsági szintet képviselő *OAuth2ClientCredentials* hitelesítési folyamatot implementáltam.

- Ez a beállítás biztosítja, hogy a Work Zone felületén megjelenő feladatok (*Task*-ok) és a folyamatindító űrlapok (*Trigger Forms*) megfelelően szinkronizálódjanak a háttérben futó instanciákkal.

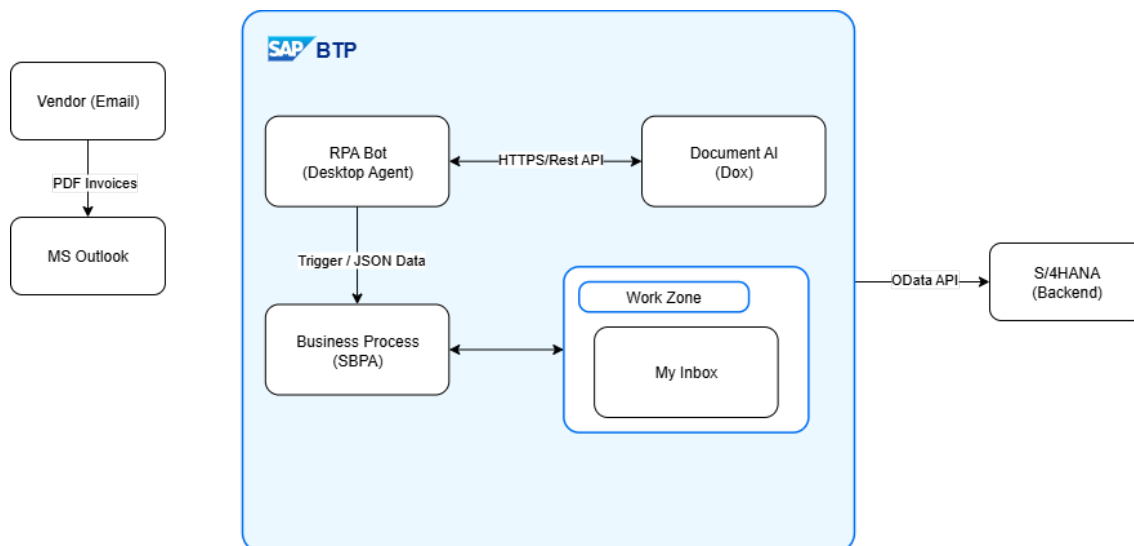
```
1 [
2   {
3     "Name": "S4HANA_CLOUD",
4     "Type": "HTTP",
5     "URL": "https://my30xxxx-api.s4hana.ondemand.com",
6     "Authentication": "BasicAuthentication",
7     "User": "SBPA_BOT_USER",
8     "sap.processautomation.enabled": "true"
9   },
10  {
11    "Name": "sap_process_automation_service_workzone",
12    "Type": "HTTP",
13    "Authentication": "OAuth2ClientCredentials",
14    "TokenServiceURL": "https://<subaccount>.authentication.eu10.hana.ondemand.com/oauth/token",
15    "ClientId": "sb-clone-12345!b12345",
16    "ClientSecret": "*****",
17    "sap.processautomation.enabled": "true",
18    "HTML5.DynamicDestination": "true",
19    "WebIDEEnabled": "true"
20  }
21 ]
22
```

4.1. lista. A konfigurált BTP Destination-ök reprezentatív JSON specifikációja

A prototípus teszteléséhez és a robusztusság bizonyításához elengedhetetlen volt a megfelelő minőségű és mennyiségű tesztadat. Mivel a valós vállalati számlák gyakran szenzitív adatokat tartalmaznak, egy egyedi **Python alapú tesztadat-generátor szkriptet** fejlesztettem. Ellentétben a véletlenszerű generátorokkal, ez a megoldás determinisztikus adatbevitelt tesz lehetővé: a fejlesztő által előre definiált paraméterek alapján állít elő PDF dokumentumokat. Ez a megközelítés lehetővé tette a „szabályozott tesztelést” (*Controlled Testing*): célzottan hoztam létre „tökéletes” (*Happy Path*), valamint hiányos vagy formailag hibás (*Exception Path*) számlákat három eltérő sablon (*layout*) alapján, így a rendszer hibatűrése többféle lehetséges bemenetelre tesztelhetővé vált.

4.2. A Prototípus Architektúrája és Komponensei

A rendszer architektúráját úgy terveztem meg, hogy elválasszam egymástól az adatgyűjtésért felelős technikai robotot (*Unattended Bot*) és az üzleti döntéseket kezelő folyamatot (*Process*).



4.2. ábra. A prototípus architektúrájának és komponenseinek áttekintése

Ahogy az infrastruktúra áttekintő ábrán (4.2. ábra) is látható, a megoldás három fő pillérre épül:

- **SAP Build Process Automation (SBPA):** A folyamatvezérlés és az RPA botok futtatásáért felelős szolgáltatás.
- **SAP Document AI (DOX):** A mesterséges intelligencia alapú OCR és adatkinyerő modul.
- **SAP Build Work Zone:** Az üzleti felhasználók számára biztosított egységes felület (*Central Entry Point*).

A technikai réteg (*Bot*) felelős a levelezőrendszer (Outlook) figyeléséért, a csatolmányok kezeléséért és a DOX API-val való szinkron kommunikációért. Ezt a logikát az SAP Build „Desktop Agent” komponense valósítja meg. Az üzleti réteg (*Process*) kizárólag a már strukturált adatokkal dolgozik, elvégzi a validációt, kezeli a jóváhagyási lépéseket, és kommunikál a felhasználókkal a Work Zone felületén keresztül.

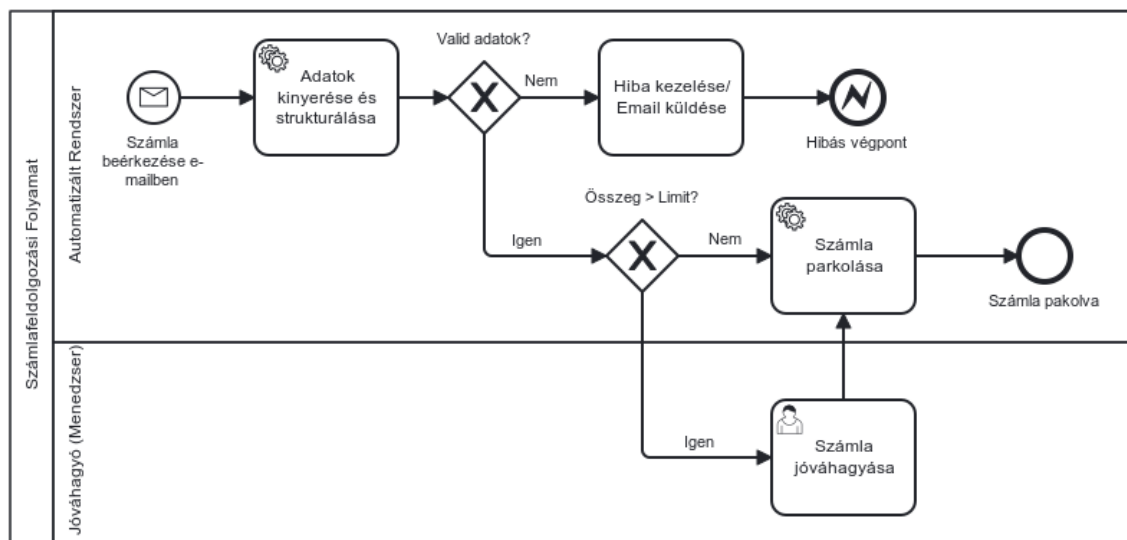
A rendszerkomponensek közötti integrációt *RESTful* architektúra alapokon valósítottam meg. A technikai robot és a felhőalapú szolgáltatások (DOX, BTP Services) közötti kommunikáció HTTPS csatornán zajlik, JSON formátumú adatcsere alkalmazásával. A biztonságos hitelesítést (*Authentication*) és a jogosultságkezelést (*Authorization*) az OAuth 2.0 iparági szabvány biztosítja: a robot a munkamenet elején az XSUAA szolgáltatástól dinamikusan igényel hozzáférési tokent (*Bearer Token*), amelyet minden későbbi API hívás fejlécében továbbít.

Az architektúra kulcsfontosságú eleme a transzformációs réteg: a bemeneti oldalon keletkező strukturálatlan adatokat (nyers PDF bájtfolyam) és a DOX szolgáltatás által visszaadott technikai JSON válaszokat a rendszer egy köztes, normalizált adatmodellre (*Canonical Data Model*) képezi le. Ez biztosítja, hogy az üzleti folyamat (*Process*) már független legyen a bemeneti forrás technikai sajátosságaitól, megvalósítva ezzel a laza csatolást (*Loose Coupling*) elvét.

4.2.1. A Folyamat Logikai Tervezése és BPMN Implementációja

A rendszerarchitektúra (4.2. pont) definiálása után a következő mérnöki lépés a folyamatvezérlés logikai modelljének megalkotása volt. A tervezéshez az ipari szabványnak tekinthető **BPMN 2.0 (Business Process Model and Notation)** jelölésrendszert alkalmaztam, amely biztosítja a platformfüggetlen dokumentálhatóságot és az üzleti logika tiszta szétválasztását a technikai megvalósítástól.

A 4.3. ábra szemlélteti a megtervezett „To-Be” folyamat logikai struktúráját *Swimlane* (sávós) elrendezésben, amely vizuálisan is elkülöníti a teljesen automatizált rendszertevékenységeket az emberi beavatkozást igénylő lépésektől.



4.3. ábra. A tervezett automatizált (To-Be) folyamat szoftverfüggetlen BPMN 2.0 modellje.

A modell az alábbi négy kritikus vezérlési szakaszt definiálja, amelyek később közvetlenül leképezésre kerültek az SAP Build Process Automation folyamatszerkesztőjében:

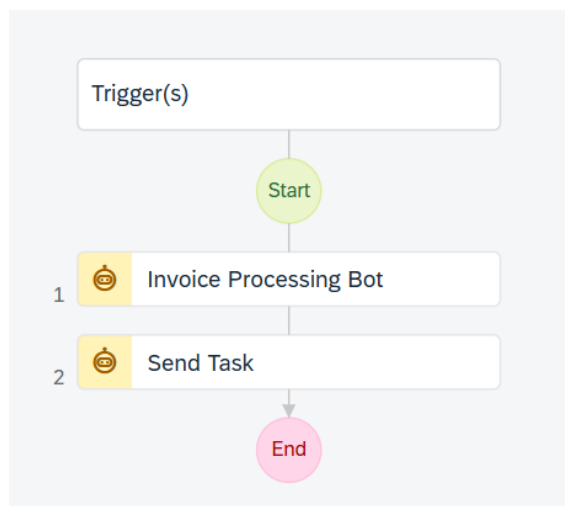
- Ütemezett indítás (Scheduled Trigger / Polling):** A kliensoldali eseményvezérlés (pl. Outlook makrók) szigorú vállalati biztonsági korlátozásai miatt a folyamat egy időzített lekérdezési mechanizmust alkalmaz. A rendszer a háttérben automatikusan, 5 perces időközönként fut le, és ellenőrzi az új számlák jelenlétét a postafiókban, ezzel megvalósítva a kvázi valós idejű (*near real-time*) feldolgozást.
- Automatikus Adatfeldolgozás (Data Processing):** Az első szakaszban (Automatizált Rendszer sáv) a rendszer autonóm módon, emberi beavatkozás nélkül végzi el az adatok strukturálását és normalizálását.
- Döntési Kapuk (Gateways):** A folyamat intelligenciáját a rombusz alakzatokkal jelölt elágazások adják. A modell kétlépcsős szűrést alkalmaz:
 - Technikai validáció (Valid Adatok?):* Ellenőrzi az adatok teljességét (pl. hiányzó szállító).

- *Üzleti szabály (Összeg > Limit?)*: A számla végösszege alapján dönti el a jóváhagyás szükségességét.
4. **Hibrid Végrehajtás (Orchestration)**: A folyamat dinamikusan vált a teljesen automatikus („Touchless”) és az emberi döntéshozatalt igénylő („Manager Approval”) ágak között, biztosítva az erőforrások optimális elosztását.

Ez a logikai terv szolgáltatta az alapot a következő alfejezetekben részletezett technikai komponensek (Bot, API hívások, UI űrlapok) konfigurációjához és integrációjához.

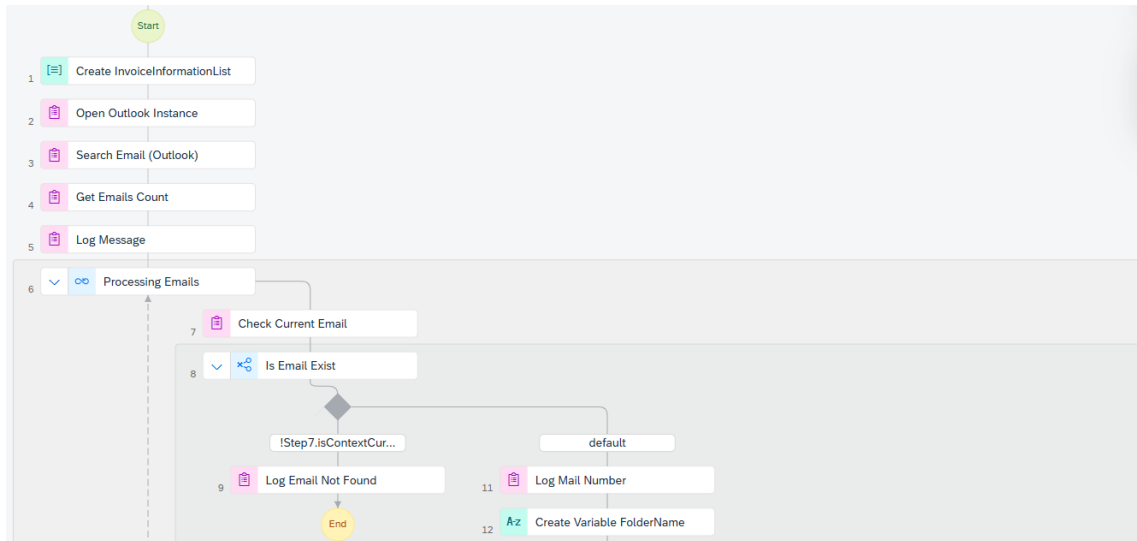
4.2.2. RPA Vezérlés és Helyi Erőforrás-kezelés

A folyamat technikai belépési pontja nem közvetlenül a feldolgozó logika, hanem egy fő vezérlő automatizáció (*Main Wrapper Automation*), amelyet a Desktop Agent inicializál az időzített trigger alapján. Ahogyan a 4.4. ábrán látható, ez a réteg felelős az alfolyamatok orchestrációjáért: első lépésben meghívja a tényleges feldolgozást végző robotot (Invoice Processing Bot), majd a sikeres futás után kezdeményezi az adatok továbbítását (Send Task).



4.4. ábra. A fő vezérlő automatizáció (Wrapper), amely elindítja a feldolgozó robotot.

A feldolgozó robot (*Invoice Processing Bot*) elsődleges feladata a lokális környezettel való interakció. A folyamat a levelezőrendszerrel való kapcsolat felépítésével indul. A 4.5. ábra szemlélteti az Outlook integráció lépéseit: a robot csatlakozik a futó Outlook példányhoz (Open Outlook Instance), majd a Search Email aktivitás segítségével, előre definiált szűrőfeltételek (Tárgy mező, Olvasatlanság) alapján leválogatja a feldolgozandó üzeneteket.

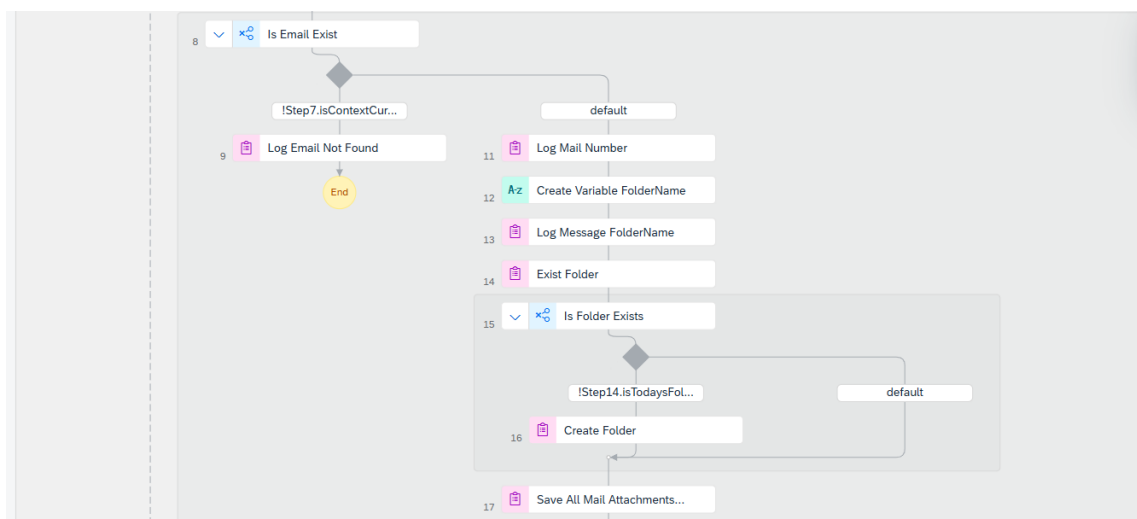


4.5. ábra. Az Outlook integráció és a levelek szűrésének folyamata.

A levelek feldolgozása egy ciklusban (*Loop*) történik. Mielőtt az adatkinyerés megkezdődne, a rendszer egy strukturált helyi archívumot képez a csatolmányokból. Ez a lépés kritikus a nyomonkövethetőség (*Digital Audit Trail*) szempontjából, biztosítva, hogy az eredeti dokumentumok offline is rendelkezésre álljanak.

A 4.6. ábrán látható logika dinamikus mappakezelést valósít meg:

1. A rendszer minden futáskor generál egy változót az aktuális dátum alapján (Create Variable FolderName).
2. Ellenőrzi, hogy létezik-e már a napi mappa a fájlrendszerben (Is Folder Exists).
3. Amennyiben nem, a robot automatikusan létrehozza azt (Create Folder).
4. Végül a Save All Mail Attachments aktivitás menti le a PDF állományokat a dedikált helyre.



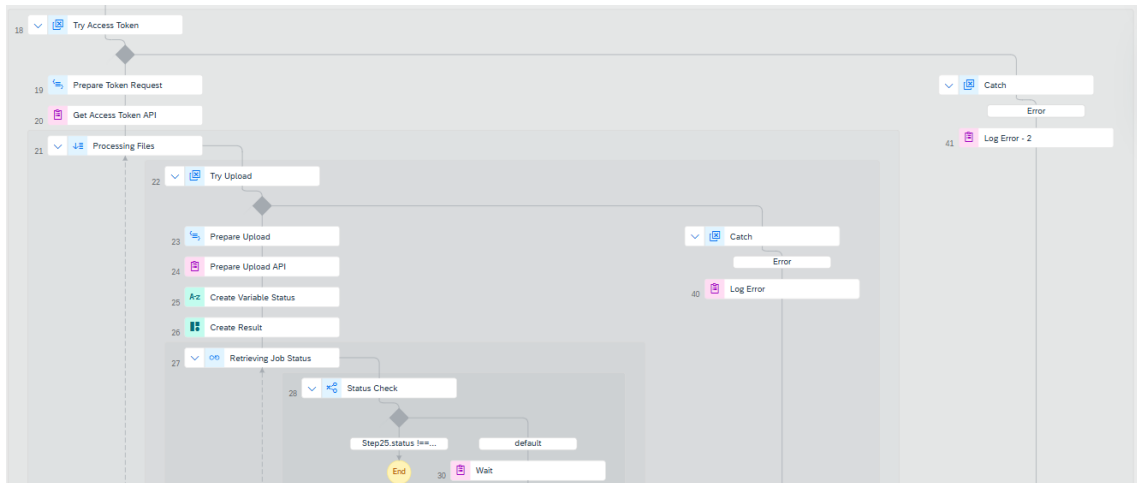
4.6. ábra. Dinamikus mappakezelés és a csatolmányok helyi mentése.

4.2.3. Adatkinyerés Konfigurálása és API Kommunikáció

A dokumentumok helyi archiválása után a folyamat átlép a felhőalapú feldolgozási szakaszba. Mivel az SAP Document Information Extraction (DOX) szolgáltatás aszinkron működésű és REST API alapú, a kommunikációt három logikai lépésre bontottam: hitelesítés, dokumentumfeltöltés, és az eredmények lekérdezése (Polling). A fejlesztés során a robusztusság érdekében a standard SAP Build könyvtárak mellett az `irpa_core` modult használtam a részletes naplózás és a hibakezelés (Error Handling) támogatására.

4.2.3.1. Hitelesítés és Dokumentum Feltöltés

A kommunikáció első lépése a biztonságos csatorna kiépítése. A 4.7 ábrán látható folyamatrészlet elején a robot a Try Access Token blokkban kezdeményezi a hitelesítést.



4.7. ábra. A hitelesítési (Token) és feltöltési (Upload) folyamat logikai ábrája.

A hitelesítéshez az OAuth 2.0 *Client Credentials* folyamatot alkalmaztam. A Prepare Token Request szkript dinamikusan állítja össze a kérést a környezeti változók alapján, biztosítva, hogy a robot mindig érvényes Bearer tokennel rendelkezzen.

```
1 // Input variables: uaaUrl, clientInfo
2 return {
3   method: 'GET',
4   url: uaaUrl + '/oauth/token?grant_type=client_credentials',
5   responseType: 'json',
6   resolveBodyOnly: true,
7   headers: {
8     Authorization: 'Basic ' + clientInfo
9   }
10 };
```

4.2. lista. A hitelesítési kérést előkészítő kód

A sikeres hitelesítést követően történik a PDF állományok tényleges feltöltése a `/document/jobs` végpontra. Itt kritikus fontosságú a `optionsPayload` objektum konfigurációja (lásd 4.3. kódblokk). A `schemaName: "SAP_invoice_schema"` pa-

paraméter megadásával utasítom a rendszert, hogy ne a hagyományos, sablon-alapú felismerést, hanem a Generatív AI modellel támogatott sémafelismerést alkalmazza.

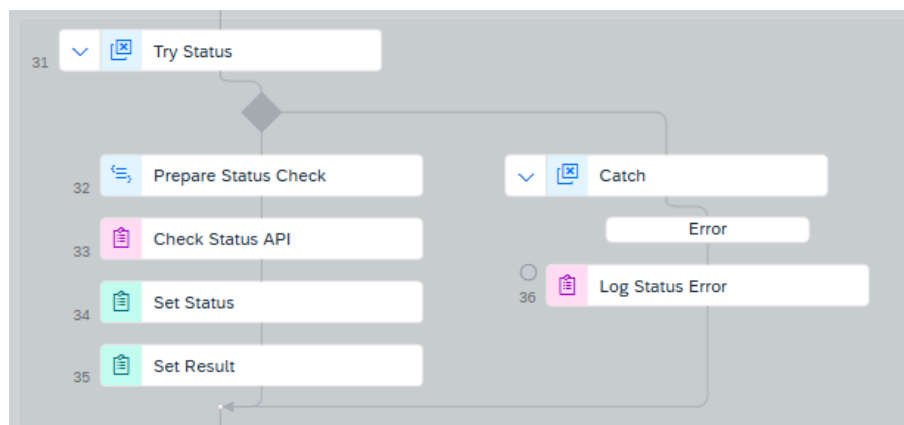
Technikai megjegyzés: A prototípus implementációja során az URL címet explicit módon definiáltam (*hard-coded URL*). Éles, vállalati környezetben (*Production*) ezt a „Clean Core” elveknek megfelelően BTP Destination konfiguráción keresztül ajánlott megvalósítani, amely központosítottan kezeli a végpontokat és a tanúsítványokat, függetlenül a kódot a fizikai régiótól.

```
1 // Input változók: filePath, accessToken
2 var myToken = "Bearer " + accessToken;
3
4 var optionsPayload = {
5     documentType: "custom",
6     schemaName: "SAP_invoice_schema",
7     clientId: "default"
8 };
9
10 return {
11     method: "POST",
12     // Demonstráció: célból direkt URL hívás:
13     url: "https://aiservices-dox.cfapps.eu10.hana.ondemand.com/document-
14         information-extraction/v1/document/jobs",
15     responseType: "json",
16     resolveBodyOnly: true,
17     metadataType: irpa_core.enums.request.metadataType.formData,
18     headers: {
19         Authorization: myToken
20     },
21     metadata: [
22         {
23             name: "file",
24             file: filePath
25         },
26         {
27             name: "options",
28             value: JSON.stringify(optionsPayload)
29         }
30     ]
31 };
```

4.3. lista. A multipart/form-data feltöltést konfiguráló szkript

4.2.3.2. Aszinkron Státuszfigyelés (Polling Mechanism)

Mivel a gépi látás számításigényes feladat, a feltöltés válasza csak egy jobId-t tartalmaz, nem magát az adatot. A feldolgozás állapotát a robotnak aktívan figyelnie kell. Ezt a feladatot a 4.8 ábrán látható Try Status hurok végzi.



4.8. ábra. A feldolgozási státuszt figyelő ciklus (Polling Loop) szerkezete.

A ciklus belsejében futó Prepare Status Check szkript periodikusan (a beállított várakozási idővel) kérdezi le az adott feladat státuszát. Fontos technikai részlet a Cache-Control: no-cache fejléc alkalmazása, amely megakadályozza, hogy a rendszer gyorsítótárazott, elavult státusz-információt kapjon vissza.

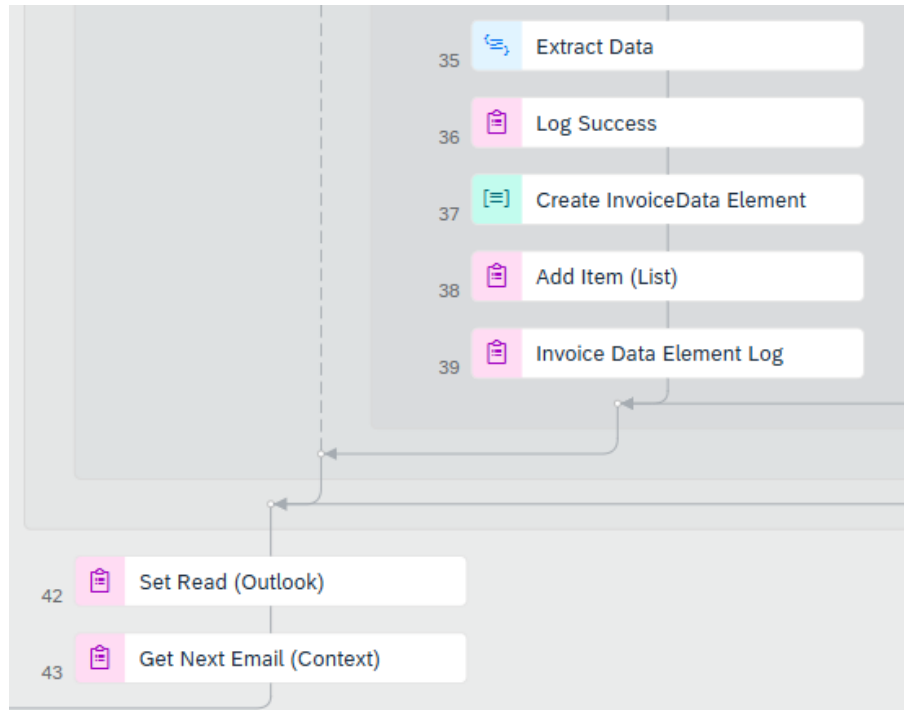
```

1 // Input változók: jobId, accessToken
2 return {
3   method: "GET",
4   url: "https://aiservices-dox.cfapps.eu10.hana.ondemand.com/document-
5     information-extraction/v1/document/jobs/" + jobId,
6   responseType: "json",
7   resolveBodyOnly: true,
8   headers: {
9     Authorization: "Bearer " + accessToken,
10    'Cache-Control': 'no-cache'
11  }
12 };
  
```

4.4. lista. A státuszlekérdezés kódja

4.2.3.3. Adatkinyerés és Normalizálás

Amikor a státuszlekérdezés DONE eredménnyel tér vissza, a folyamat a befejező szakaszba lép (lásd 4.9 ábra). A robot lekéri a feldolgozott JSON eredményt, naplózza a sikeres futást, és elvégzi az email olvasottnak jelölését (Set Read).



4.9. ábra. Az adatkinyerés lezárása és az e-mail státuszának frissítése.

A folyamat "agya" az Extract Data szkript, amely transzformációs réteggént funkcionál a nyers API válasz és az üzleti folyamat között. A kód (lásd 4.5. kód-blokk) nemcsak egyszerűsíti az adatstruktúrát, hanem egy részletes diagnosztikai naplózást is végez. A confidence (magabiztosság) értékek vizsgálatával a rendszer figyelmeztetést (WARNING) generál a bizonytalan adatok esetén, ami a tesztelés során kulcsfontosságú visszajelzést biztosított.

```

1 // Input változók: apiResult, currentFilePath
2
3 var extracted = { /* ... Alapértelmezett értékek inicializálása ... */ };
4
5 // Konfigurálható biztonsági küszöbérték (85%)
6 var CONFIDENCE_THRESHOLD = 0.85;
7
8 if (apiResult && apiResult.extraction && apiResult.extraction.headerFields) {
9     var fields = apiResult.extraction.headerFields;
10
11     for (var i = 0; i < fields.length; i++) {
12         var fieldName = fields[i].name;
13         var fieldValue = fields[i].value;
14         var conf = fields[i].confidence || 0.0;
15
16         // A pontosság ellenőrzése a küszöbértékhez képest
17         var isConfident = conf >= CONFIDENCE_THRESHOLD;
18
19         // 1. Példa: Szállító név leképezése (Vendor Mapping)
20         if (fieldName === "senderName" || fieldName === "vendorName") {
21             if (isConfident) {
22                 extracted.VendorName = fieldValue;
23             } else {
24                 irpa_core.core.log("WARNING: Szállító név elutasítva (Biztonság:
25 " + (conf*100).toFixed(1) + "%)");

```

```

25     }
26 }
27
28 // ... (Hasonló logika ismétlődik a többi mezőre: Számlaszám, Összeg, Dá-
29 tum) ...
30 }
31
32 return extracted;

```

4.5. lista. Az adatokat normalizáló algoritmus konfidencia-ellenőrzéssel (Részlet)

4.2.3.4. Hitelesítés és Biztonságos Kulcskezelés

A kommunikáció biztonságát az **OAuth 2.0** iparági szabvány garantálja. A rendszer a `client_credentials` engedélyezési folyamatot (Grant Type) valósítja meg, amely során a robot a *Client ID* és *Client Secret* birtokában dinamikusan kérényez hozzáférési tokent (*Bearer Token*) az SAP XSUAA (*Extended Services for User Account and Authentication*) szolgáltatástól.

Kiemelt biztonsági szempont volt, hogy a hitelesítéshez szükséges érzékeny adatokat (különösen a *Client Secret*-et) **soha ne tároljam forráskódba égetett értékként**. Ehelyett az SAP Build Process Automation beépített biztonsági funkcióját, az **Environment Variables** (Környezeti Változók) szolgáltatást alkalmaztam.

- **Titkosított Tárolás:** A fejlesztési környezetben (*Control Tower*) létrehoztam egy titkosított változót a hitelesítési adatok számára. Ezt az értéket a rendszer az adatbázisban kódolva tárolja, és a fejlesztői felületen sem olvasható vissza.
- **Futásidejű Injektálás:** A `Prepare-Token-Request` szkript bemeneti paraméterként (`clientInfo`) kapja meg a hitelesítési adatokat. A robot futásidőben (*Runtime*) olvassa ki a biztonságos tárolóból az értéket, és csak a memóriában, az API hívás erejéig használja fel azt.

Ez a megoldás biztosítja a *Security by Design* elv érvényesülését: a forráskód bárki számára olvasható lehet anélkül, hogy a rendszer biztonsága sérülne. A megszerzett tokent a szkript a HTTP kérés fejlécébe (`Authorization: Bearer <Token>`) illeszti be a DOX szolgáltatás hívásához.

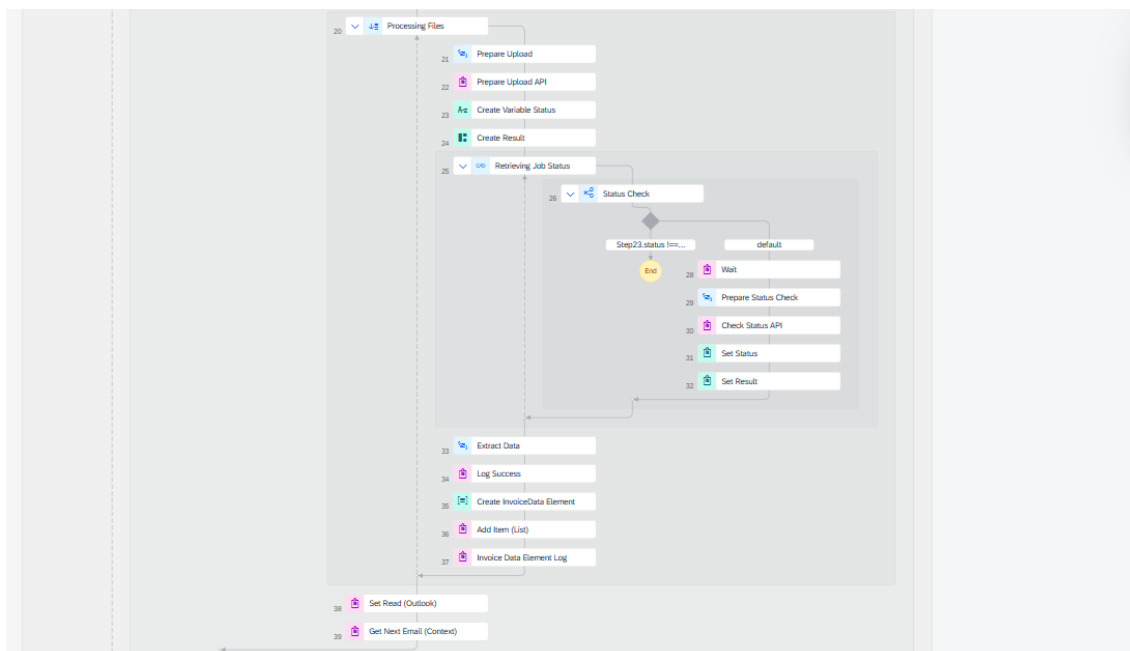
4.2.3.5. Aszinkron Feldolgozás és Timeout Kezelés (Polling Pattern)

Mivel a gépi látás és az adatkinyerés számításigényes, aszinkron folyamat, a DOX szolgáltatás a feltöltéskor azonnal visszatér egy `Job ID`-val, miközben a feldolgozás a háttérben zajlik. Ennek kezelésére a robotban egy **Polling (lekérdezési) ciklust** implementáltam (lásd 4.10. ábra).

A végtelen ciklusok (Infinite Loop) elkerülése érdekében egy mérnöki biztonsági mechanizmust, úgynevezett **Timeout-számlálót** építettem a logikába:

1. **Ciklusváltozók:** A folyamat elején definiáltam egy `retryCount` változót (0) és egy `maxRetries` határértéket (20 alkalom).
2. **Lekérdezés:** A ciklus belsejében a `Prepare-Status-Check` szkript periodikusan (5 másodpercenként) GET kérést küld a szervernek.

3. **Megszakítási feltétel:** Minden iteráció növeli a számlálót. Ha a számláló eléri a határértéket ($20 \times 5s = 100s$) anélkül, hogy a státusz DONE-ra váltana, a robot **kivételt dob** (TimeoutException) és kilép a ciklusból.



4.10. ábra. Az aszinkron feldolgozás státuszát figyelő ciklus (Polling Loop) implementációja a Timeout védelemmel.

Ez a megoldás garantálja, hogy szolgáltatáskiesés esetén a robot nem ragad "zombi" állapotban, hanem kontrollált hibaüzenettel leállítja a folyamatot és értesíti az adminisztrátort.

Hibakezelés és Kivételkezelés (Try-Catch Pattern) A robusztus működés érdekében a kritikus műveleteket – különösen a külső API hívásokat (Upload, Status Check, Get Token) – strukturált hibakezelési blokkokkal láttam el. Az SAP Build **Try-Catch** vezérlőelemének alkalmazásával elkerülhető, hogy egy váratlan hiba (pl. 500 Internal Server Error vagy hálózati szakadás) a teljes folyamat kontrollálatlan leállítását okozza.

A megvalósított logika szerint minden API interakció egy Try ágba került. Amennyiben a hívás során hiba lép fel, a vezérlés azonnal átkerül a Catch (Error) ágra, ahol a rendszer végrehajtja a diagnosztikai lépéseket:

1. **Hibaazonosítás:** A rendszer kinyeri a hibaobjektumból a pontos státuszkódot (pl. 401) és a hibaüzenetet (error.message).
2. **Naplózás (Logging):** A Log Message aktivitás segítségével a hiba részletei rögzítésre kerülnek a konzolon, ami elengedhetetlen a későbbi hibakereséshez (Troubleshooting).
3. **Folyamatvezérlés:** A kritikus hibák esetén a bot szabályozottan leállítja az adott tranzakciót, és a „Failed” státusz beállításával értesíti a felügyeletet, megakadályozva a hibás adatok továbbterjedését a folyamatban.

Ez a többrétegű védelmi mechanizmus (Timeout számláló + Try-Catch blokkok) biztosítja, hogy a robot ipari környezetben is stabilan, felügyelet nélkül (*Unattended*) üzemeltethető legyen.

4.2.3.6. Adatnormalizálás és Leképezés

A sikeres feldolgozást követően a rendszer egy összetett JSON objektumot ad vissza, amely tartalmazza a felismert mezőket és azok konfidencia-értékeit. Ezen nyers adatok transzformációját az `Extract_Data` szkript végzi (lásd 4.5. kódblokk).

A script feladata a kanonikus adatmodell előállítás: a szolgáltatótól függő, változatos mezőelnevezéseket (pl. a JSON-ben érkező `senderName`, `vendorName` vagy `supplier`) egy egységes, belső objektumstruktúrára (`extracted.VendorName`) képezi le.

Ez a réteg (*Data Abstraction Layer*) biztosítja, hogy a folyamat további lépései függetlenek legyenek a bemeneti dokumentum technikai sajátosságaitól, jelentősen növelve ezzel a rendszer hibatűrését.

4.2.4. A Jóváhagyási Folyamat és Döntési Logika Tervezése

Az adatkinyerési szakasz lezárultával a Desktop Agent a strukturált és normalizált adatokat egy API híváson (*API Trigger*) keresztül adja át a fő üzleti folyamatnak (*Business Process*). A folyamat bemeneti változói (*Process Inputs*) fogadják a JSON objektumot, amely innentől kezdve a folyamatkontextus (*Process Context*) részét képezi, és elérhetővé válik a döntési pontok és a felhasználói űrlapok számára.

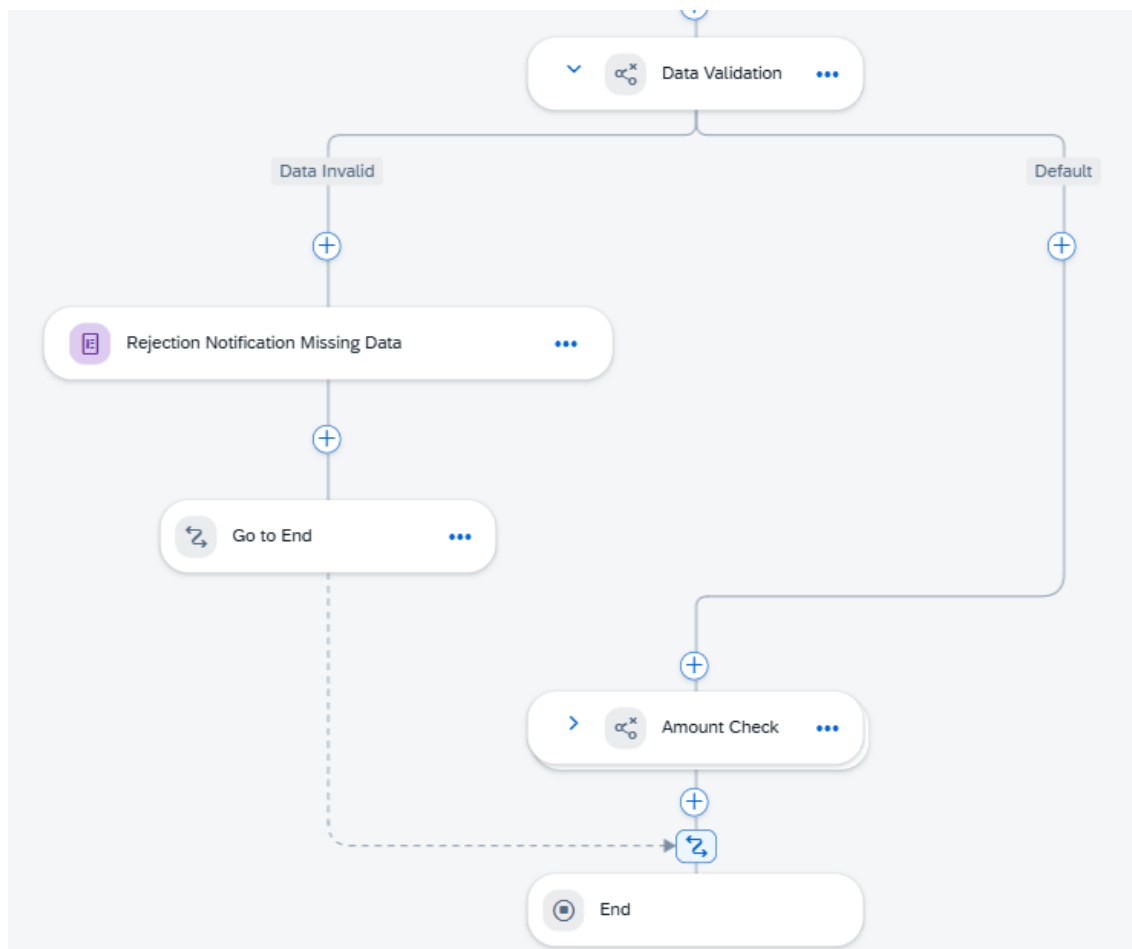
A folyamatvezérlés logikai felépítése a **BPMN 2.0** szabványt követi. A központi vezérlő elem a Form Notifications Handling alfolyamat, amely exkluzív logikai kapuk (*Exclusive Gateways*) sorozatával valósítja meg a döntési fát.

4.2.4.1. Elsődleges Adatvalidáció (Gatekeeper Pattern)

A folyamat első védvonala a Data Validation döntési pont (lásd 4.11. ábra). A rendszer itt vizsgálja meg programozott feltételekkel, hogy a kritikus mezők (pl. Számlaszám, Dátum, Szállító) rendelkeznek-e érvényes értékkel.

Hibaág (*Exception Path*): Amennyiben a rendszer hiányosságot észlel (pl. `invoiceNumber == null`), a folyamat a Rejection Notification ágra fut. Itt a rendszer automatikusan generál egy értesítést a felhasználónak a hiba pontos megjelölésével, megakadályozva, hogy hibás adat kerüljön a könyvelésbe.

Sikeres ág (*Happy Path*): Valid adat esetén a vezérlés tovább lép a második szintű, üzleti ellenőrzésre.



4.11. ábra. Az adatvalidációs logikai kapu és a hibakezelési ág a folyamattervezőben

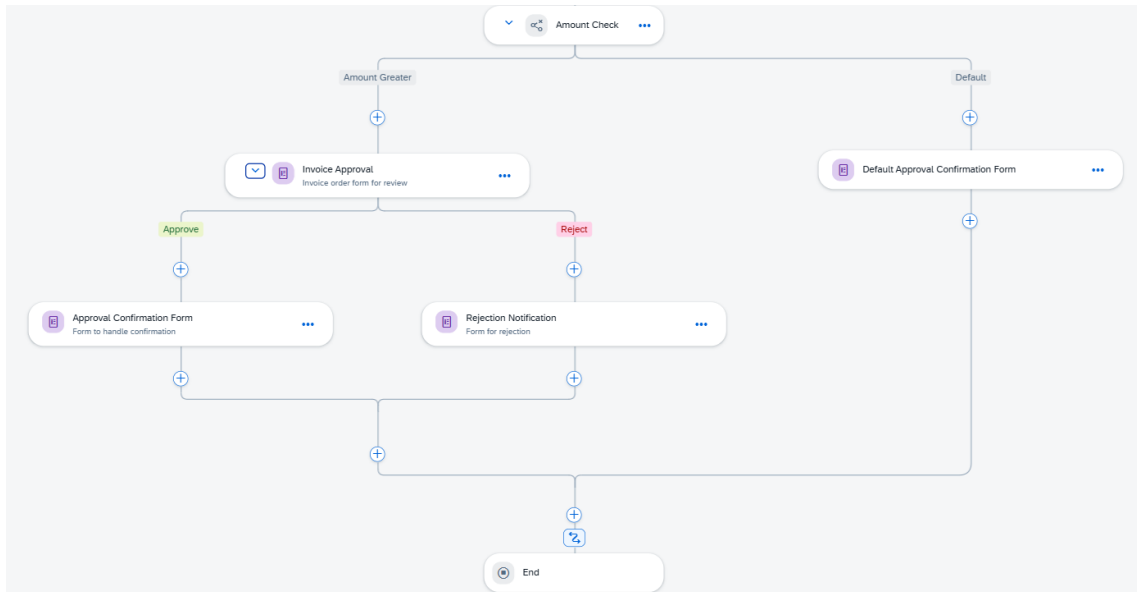
4.2.4.2. Üzleti Szabályok és Jóváhagyási Stratégia

A validált számlák a prototípus legfontosabb döntési pontjához, az Amount Check (Összegellenőrzés) lépéshez érkeznek. Itt valósul meg a vállalat automatizálási stratégiája: a rendszer a számla bruttó végösszege alapján dönti el a szükséges beavatkozás mértékét (lásd 4.12. ábra).

A prototípusban implementált szabályrendszer a következő:

Nagy értékű számlák (*Amount Greater*): Ha az összeg meghaladja a konfigurált küszöbértéket, a folyamat az Invoice Approval ágra lép. Ekkor a rendszer egy „User Task” (Felhasználói feladat) típust generál, amely megállítja a folyamat futását mindaddig, amíg egy jogosult jóváhagyó (*Approver*) a Work Zone felületén döntést nem hoz.

Kis értékű számlák (*Default/Auto-approve*): Küszöbérték alatti összeg esetén a folyamat a Default Approval ágra fut. Ez demonstrálja a „Touchless Processing” (érintésmentes feldolgozás) koncepcióját, ahol az alacsony kockázatú tételek emberi interakció nélkül kerülhetnek továbbításra a backend rendszer felé.



4.12. ábra. Az összeghatár alapú döntési logika (Business Rule) implementációja

Ez a többszintű, feltételes elágazásokra épülő struktúra biztosítja, hogy az emberi erőforrásokat csak a valóban döntést igénylő esetekhez rendeljük hozzá, maximalizálva ezzel a folyamat hatékonyságát.

4.2.5. Üzleti Szabályok Implementációja (Technical Decisions)

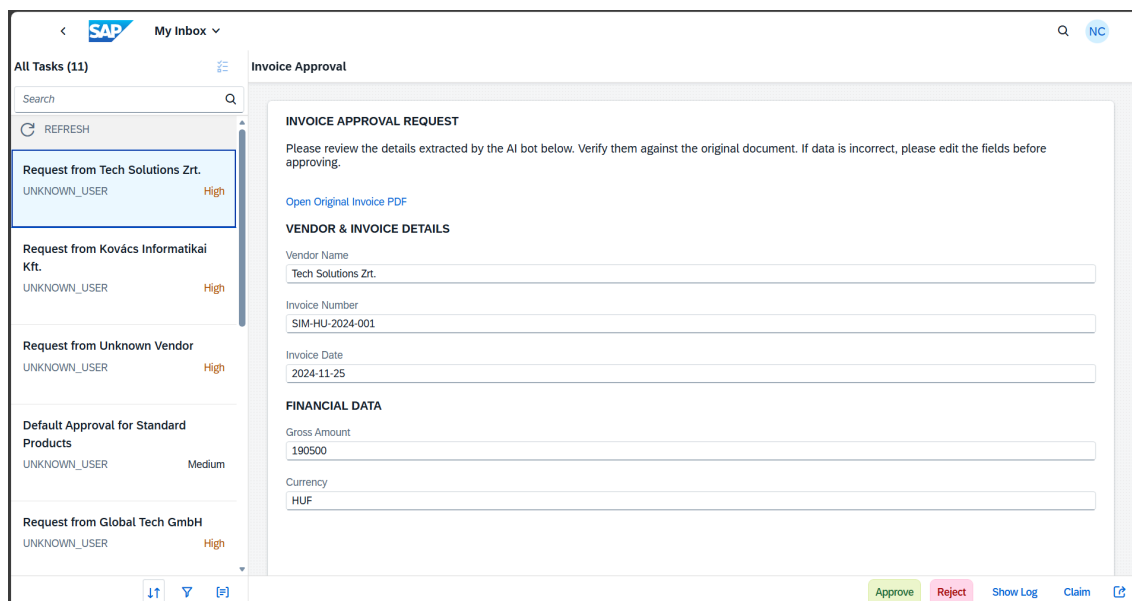
Az üzleti döntések technikai megvalósításához a **Döntési Tábla** (*Decision Table*) artefaktumot alkalmaztam. Ez az architektúrális megoldás megvalósítja az üzleti logika és a folyamatvezérlés szétválasztását (*Separation of Concerns*).

A komponens bemenetként (*Input*) fogadja a normalizált számlaadatokat – kiemelten a GrossAmount és Currency változókat –, kimenetként (*Output*) pedig egy vezérlési döntést ad vissza a folyamat számára. A megoldás nagy előnye a karbantarthatóság: az összeghatárok és szabályok egy konfigurációs felületen módosíthatók anélkül, hogy a folyamatdiagramot újra kellene tervezni vagy a forráskódot módosítani kellene.

4.2.6. Felhasználói Felületek és Adatkötés (Forms & UI)

A felhasználói interakciók központja az **SAP Build Work Zone** „My Inbox” alkalmazása. A jóváhagyási űrlapokat a *No-Code Form Editor* segítségével terveztem meg, ahol a mezők értékeit közvetlenül a folyamat kontextusából (*Context Binding*) nyertem ki.

Ahogy a 4.13. ábrán látható, a jóváhagyó nemcsak a robot által kinyert strukturált adatokat (Szállító, Összeg, Dátum) látja, hanem egy közvetlen linket is kap az eredeti PDF dokumentum megnyitásához (Open Original Invoice PDF). Ez a funkció kiküszöböli a „swivel-chair” problémát, mivel az ellenőrzés és a döntés egyetlen felületen, kontextusváltás nélkül elvégezhető.

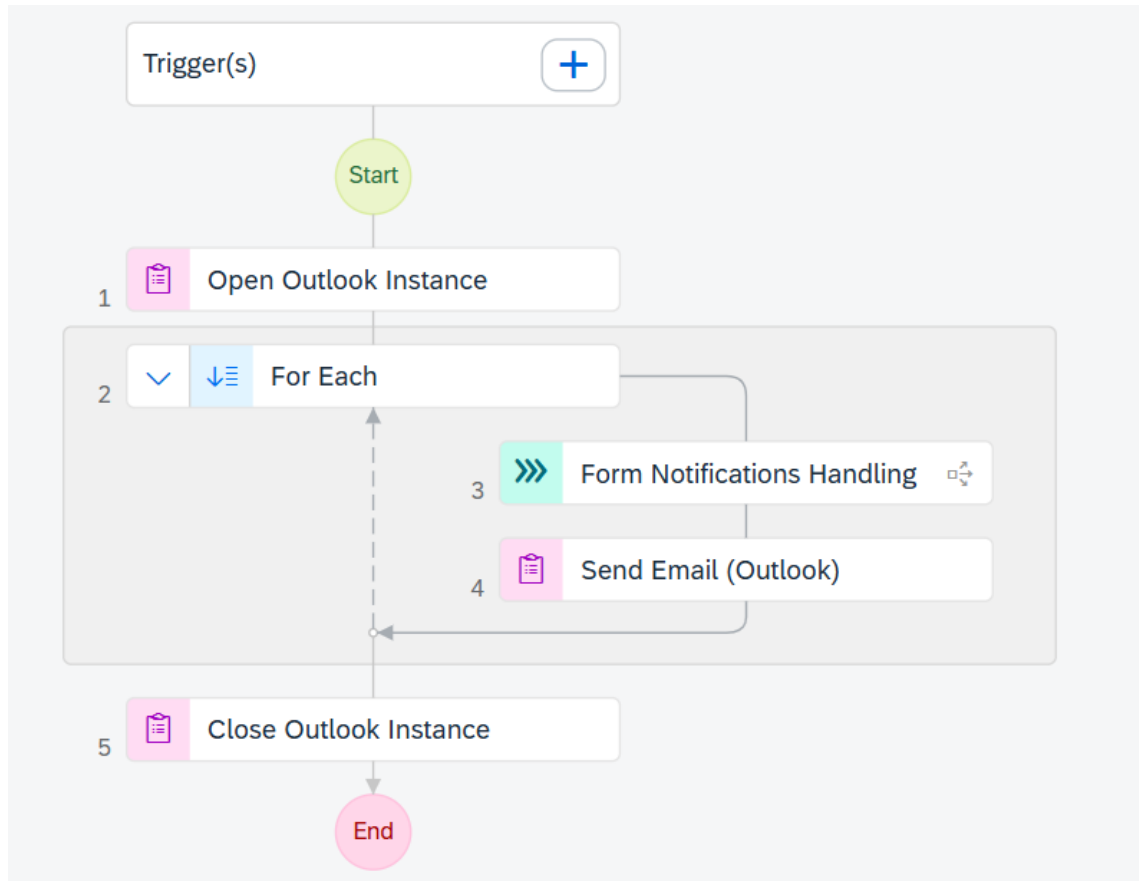


4.13. ábra. A jóváhagyói felület a Work Zone „My Inbox” alkalmazásában

4.2.7. Backend Integráció: Moduláris Felépítés és a Parkoló Bot

A folyamat lezáró szakaszában a jóváhagyott adatok SAP S/4HANA rendszerbe történő továbbítását valósítottam meg. A tervezés során a monolitikus megközelítés helyett a moduláris architektúrát választottam: szétválasztottam a folyamatvezérlést (*Orchestration*) és a technikai végrehajtást (*Execution*).

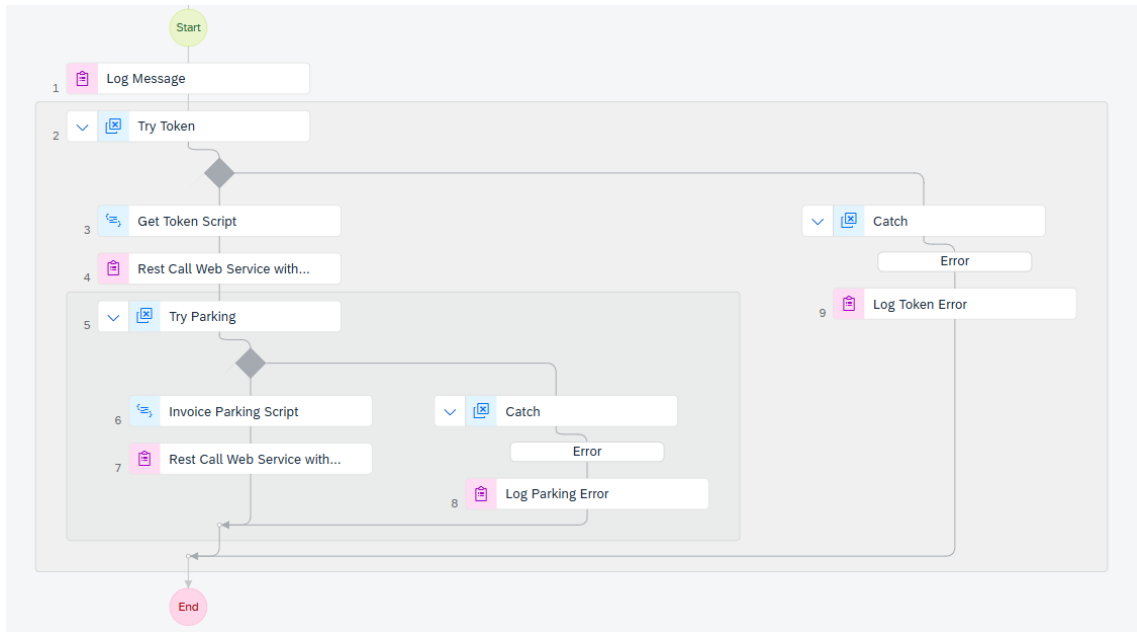
A folyamat legfelső szintjét az Automation Task vezérli, amely meghívja a Send Task Automation komponenst (lásd 4.14. ábra). Ez az automatizáció felelős a Form Notification Handling alfolyamat kimenetének kezeléséért: iterál a jóváhagyott elemeken (*For Each loop*), és minden egyes tételhez meghívja a dedikált, felügyelet nélküli (*Unattended*) robotot.



4.14. ábra. A vezérlő folyamat (Send Task Automation), amely delegálja a feladatot a technikai robotnak.

A tényleges rendszerintegrációt a dedikált Parking Bot Automation végzi (lásd 4.15. ábra), amely egy szekvenciális, tranzakcionális logikát hajt végre:

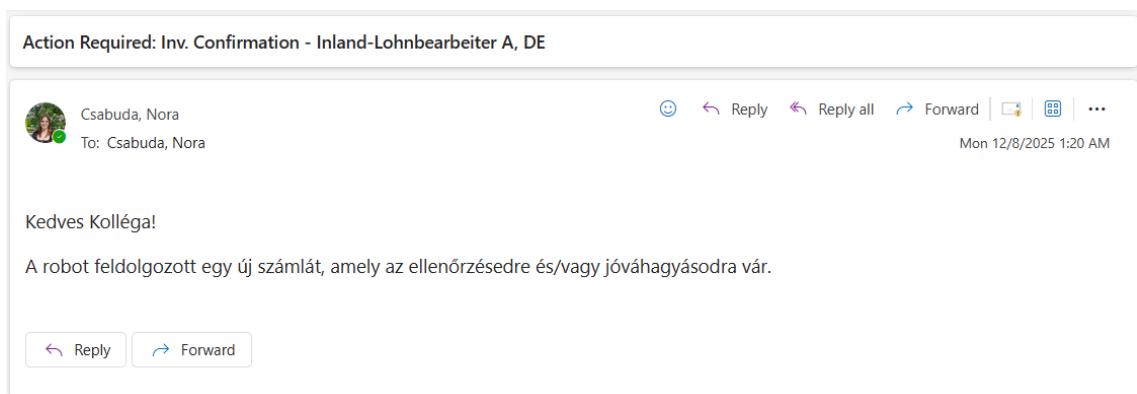
1. **Triggerelés és Adatátadás:** A fő folyamat átadja a normalizált számlaadatokat (*Input Parameters*) a robotnak.
2. **Hitelesítés (Token Fetching):** A robot első lépésként a Try Token blokkban a Get Token Script segítségével meghívja az SAP OData \$metadata végpontját (GET kérés), és megszerzi az érvényes X-CSRF-Token-t a munkamenet biztonságos indításához.
3. **Adattranszformáció (Payload Script):** A Try Parking blokkban futó Invoice Parking Script a bemeneti adatokból (pl. Szállító neve, Dátumok) és a megszerzett tokenből összeállítja a végleges, SAP-konform JSON üzenetet. Itt történik a szállító-azonosítás (Mapping) és a kötelező mezők (pl. TaxDeterminationDate) beillesztése is.
4. **Végrehajtás (API POST):** A Rest Call Web Service aktivitás elküldi az összeállított csomagot az S/4HANA A_SupplierInvoice végpontjára. A kérés fejlécében a Parked státuszkódot állítja be, így a számla előkönyvelt állapotban jön létre.



4.15. ábra. A dedikált Parkoló Bot (Parking Bot Automation) belső felépítése: A Try-Catch blokkokba szervezett Token kérés és API hívás szekvenciája.

Ez a rétegzett megközelítés biztosítja a rendszer stabilitását: ha az API hívás során technikai hiba lép fel (pl. hálózati szakadás vagy érvénytelen token), a Parking Bot a beépített Try-Catch blokkok segítségével lokálisan kezeli a kivételt. A rendszer ilyenkor strukturált hibaüzenettel (Log Token Error, Log Parking Error) tér vissza a vezérlő folyamathoz anélkül, hogy a teljes rendszerleállást okozza.

Végül, sikeres végrehajtás esetén a folyamat e-mail értesítést küld az érintetteknek (lásd 4.16. ábra).



4.16. ábra. A rendszer által generált automatikus értesítés a sikeres API hívásról és a parkolásról.

5. fejezet

A Prototípus Tesztelése és Eredmények Értékelése

5.1. Bevezetés: A Tesztelés Módszertana

A prototípus működésének validálására egy szisztematikus tesztelési tervet dolgoztam ki, amely a „Happy Path” (ideális működés) mellett a kivételes eseteket („Exception Path”) is lefedi. A validáció alapfeltétele a kontrollált bemeneti adatállomány (*Ground Truth*) megléte volt. Mivel a valós vállalati számlák használata adatvédelmi okokból aggályos (GDPR), és nem garantálja minden hibatípus előfordulását, egy saját fejlesztésű, parametrikus tesztadat-generátor motort készítettem **Python** nyelven.

5.1.1. A Tesztadat-generátor Felépítése és a Determinisztikus Megközelítés Indoklása

A validáció alapfeltétele a kontrollált bemeneti adatállomány (*Ground Truth*) megléte volt. A fejlesztés során tudatos mérnöki döntés volt a **determinisztikus adatgenerálás** választása a modern Generatív AI (LLM) megoldásokkal szemben.

- **Miért nem LLM?** A Nagy Nyelvi Modellek (pl. ChatGPT) sztochasztikus, valószínűségi alapon működnek. Tesztadat-generáláskor fennáll a „hallucináció” veszélye: ha arra utasítjuk a modellt, hogy generáljon egy hibás számlát, a modell a tanult mintázatok alapján véletlenszerűen mégis kiegészítheti a hiányzó adatot (autocorrection), ami megghiúsítaná a negatív teszteket.
- **A Szkript Előnye:** Ezzel szemben a saját fejlesztésű Python modul imperatív logikája garantálja a **reprodukálhatóságot**. Ha a kódban üres sztringet rendelünk a számlaszám változóhoz, a generált PDF-en garantáltan, bit-szinten nem fog szerepelni az adat. Ez biztosítja a „Reference Truth” 100%-os pontosságát az összehasonlító mérésekhez.

A szoftvermodul az FPDF könyvtárra épül, amely lehetővé teszi a PDF dokumentumok alacsonyszintű, koordináta-alapú (`set_xy`) manipulációját. A generátor architektúrája szétválasztja az adatot és a megjelenítést (*Data-Presentation Separation*):

- **Adatréteg:** Egy JSON-szerű dictionary objektum (`base_data`) tartalmazza a számla szemantikai tartalmát (pl. `bill_to_text`, `inv_number`, `items`).
- **Megjelenítési réteg:** A `ComplexInvoice` osztály felelős azért, hogy ezeket az adatokat három különböző, a gyakorlatban elterjedt sablon (*Modern*, *Simple*, *General*) szerint renderelje a PDF-re, beleértve a táblázatok dinamikus felépítését és a betűtípusok kezelését.

Ez a megoldás lehetővé tette az **adatinjektálásos tesztelést**: a bázis adathalmaz célzott módosításával („mutációjával”) állítottam elő a hibás teszteseteket.

```

1 # 1. BASE CASE (Valid Data Definition)
2 base_data = {
3     "inv_number": "174221",
4     "inv_date": "2/18/2019",
5     "total_gross": "$ 220.00",
6     "items": [
7         ["2001", "Labor Service", "2.00", "HR", "$ 55.00", "$ 110.00"]
8     ]
9     # ... további adatok ...
10 }
11
12 # Generalas: Valid számla
13 create_complex_invoice("general_invoice_01_valid.pdf", base_data)
14
15 # 2. TEST CASE: MISSING DATA (Controlled Mutation)
16 # A bázisadatok masolása és a kötelező mező törlése
17 missing_id_data = base_data.copy()
18 missing_id_data["inv_number"] = "" # Üres sztring injektálása
19 missing_id_data["po_number"] = ""
20
21 # Generalas: Hibás számla a validáció teszteléséhez
22 create_complex_invoice("general_invoice_02_missing_id.pdf", missing_id_data)
23

```

5.1. lista. Részlet a Python generátorkódból: a bázisadatok definíciója és a hibás eset (Missing ID) előállítása adatinjektálással

5.1.2. Tesztelési Forgatókönyvek és Adatkészletek

A rendszer működésének átfogó validálásához a Python generátor segítségével összesen 8 különböző tesztesetet (*Test Case*) hoztam létre. A tesztelés célja kettős volt: egyrészt igazolni az AI modell (DOX) képességeit különböző vizuális elrendezések (*Layouts*) kezelésében, másrészt ellenőrizni az üzleti logika (*Business Rules*) és a hibakezelés (*Exception Handling*) működését a 4. fejezetben implementált szabályok alapján.

A tesztadatokat három eltérő számlasablon („General”, „Modern”, „Simple”) felhasználásával állítottam elő, biztosítva, hogy a rendszer ne csak egyetlen formátumra legyen optimalizálva.

1. Csoport: „General” Sablon (Hagyományos elrendezés) Ez a csoport a leggyakoribb, szabványos vállalati számlaképet reprezentálja.

TC_01 (General - Valid): Formailag és tartalmilag helyes számla, a limitösszeg alatti értékkel.

- **Elvárt eredmény:** Sikeres, emberi beavatkozás nélküli (*Touchless*) feldolgozás és automatikus parkolás.

TC_02 (General - Missing ID): A számláról hiányzik a kötelező számlaszám (*Invoice Number*).

- **Elvárt eredmény:** A Data Validation kapu megállítja a folyamatot, és a rendszer értesítést küld a hiányzó adatról (Technikai hibaág).

TC_03 (General - Foreign Currency): Helyes számla, de a devizanem eltér az alapértelmezett EUR-tól (pl. USD).

- **Elvárt eredmény:** Az adatkinyerés pontosságának ellenőrzése; a rendszernek helyesen kell átadnia a devizakódot az SAP API felé.

2. Csoport: „Modern” Sablon (Grafikus elrendezés) Ez a sablon bonyolultabb, grafikus elemekkel tarkított elrendezést használ, tesztelve az OCR motor képességeit.

TC_04 (Modern - Valid): Helyes adattartalmú, modern designnal rendelkező számla.

- **Elvárt eredmény:** Annak igazolása, hogy a Document AI sablonfüggetlenül képes a mezők azonosítására.

TC_05 (Modern - Duplicate): Olyan számla, amelynek számlaszáma megegyezik egy már korábban rögzített tétellel (pl. a TC_04-gyel).

- **Elvárt eredmény:** Backend oldali validáció tesztelése. Bár a folyamat elindul, az S/4HANA API-nak vissza kell utasítania a kérést 400 Bad Request vagy üzleti hibaüzenettel a duplikáció miatt.

TC_06 (Modern - Missing Vendor): A számlán nem szerepel (vagy olvashatatlan) a szállító neve.

- **Elvárt eredmény:** Az intelligens szállító-felismerés (*Vendor Mapping*) hibája. A rendszernek „Unknown Vendor”-ként kell azonosítania, és a validációs lépésnél meg kell állnia.

3. Csoport: „Simple” Sablon (Minimalista elrendezés) Kevés szöveget és egyszerű struktúrát tartalmazó dokumentumok.

TC_07 (Simple - Valid): Minimális adattartalmú, de érvényes számla.

- **Elvárt eredmény:** Sikeres feldolgozás, bizonyítva, hogy a rendszer nem igényel felesleges (zaj) adatokat a működéshez.

TC_08 (Simple - Missing Mandatory): Egyéb kötelező adat (pl. Dátum vagy Bruttó összeg) hiánya.

- **Elvárt eredmény:** A validációs szkript (*Gatekeeper*) általi elutasítás és hibaüzenet generálása.

ID	Sablon	Típus	Tesztelt Funkció
TC__01	General	Valid	Happy Path (Touchless)
TC__02	General	Invalid	Validáció (Hiányzó ID)
TC__03	General	Valid	Adatkinyerés (Deviza)
TC__04	Modern	Valid	OCR Robustness
TC__05	Modern	Invalid	Backend Validáció (Duplikáció)
TC__06	Modern	Invalid	Vendor Mapping Logic
TC__07	Simple	Valid	Minimal Data Set
TC__08	Simple	Invalid	Validáció (Hiányzó Mező)

5.1. táblázat. A definiált tesztesetek összefoglaló táblázata

5.2. Tesztelési Eredmények

A definiált forgatókönyvek lefuttatása során a rendszer monitorozó felületén (Monitoring) és a végfelhasználói felületeken (Work Zone) keletkezett bizonyítékokat rögzítettem. A tesztelés sikerességének egyik legfontosabb mérőszáma nem csupán a végeredmény helyessége, hanem a folyamat teljes átláthatósága és nyomkövet-
hetősége volt.

5.2.1. Folyamatmonitorozás és Diagnosztika

Az SAP Build Process Automation beépített **Monitoring Dashboard**-ja kritikus szerepet játszik az üzemeltetésben. Ez a felület teszi lehetővé az egyes automatizációk (Botok) és üzleti folyamatok (Processes) valós idejű megfigyelését. Ahogyan az 5.1. ábrán látható, a rendszer minden egyes futtatásról részletes naplót (*Execution Log*) vezet.

Monitoring / Automation Jobs / Invoice Processing Automation			
	Invoice Processing Automation	Completed	
Creation date:	Project	Environment	Trigger
Nov 30, 2025, 04:37 PM	Invoice_Processing_Task Version 1.4.0	Public	Scheduled_Invoice_Processing
Status History			
Status History			
Time Stamp	Status	Message	
11/30/2025, 04:37:05 PM	Ready	> Invoice Processing Automation	
11/30/2025, 04:37:23 PM	Running	> Invoice Processing Automation	
11/30/2025, 04:39:46 PM	Completed	> Invoice Processing Automation	

5.1. ábra. Az SAP Build Monitoring felülete, amely mutatja az Invoice Processing Automation sikeres (Completed) lefutását és az időbélyegeket.

A felület legfontosabb diagnosztikai képességei, amelyek a tesztelés során kiemelten hasznosnak bizonyultak:

- **Státuszkövetés:** Azonnal látható a folyamatok állapota (Running, Completed, Failed). A sikeres futásokat zöld címke jelzi, míg hiba esetén a rendszer piros jelzéssel hívja fel a figyelmet a beavatkozás szükségességére.
- **Időbeli lefutás:** A *Status History* táblázat pontos időbélyegekkel (*Timestamp*) dokumentálja az egyes lépések kezdetét és végét. Ez lehetővé teszi a teljesítményelemzést: a példában látható, hogy a teljes feldolgozás 4:37:05-kor indult és 4:39:46-kor zárult, ami igazolja a gyors, 3 percen belüli átfutási időt.
- **Hibakeresés (Root Cause Analysis):** Hiba esetén a naplóbejegyzésekre kattintva a fejlesztő közvetlenül hozzáférhet a 4. fejezetben implementált Log Message aktivitások kimenetéhez (pl. "Mező: [invoiceNumber] -> Érték: " (Pontosság: 0.0%)). Ez drasztikusan csökkenti a hibafeltárás idejét a hagyományos, naplófájlokat böngésző módszerekhez képest.

Ez a központosított felügyeleti eszköz garantálja, hogy a rendszer ne "Fekete Dobozként" működjön, hanem minden tranzakció auditálható és ellenőrizhető legyen.

5.2.2. Eredmény: Sikeres „Touchless” Feldolgozás (TC_01)

A teszt során kis összegű, rutin számlák tömeges feldolgozását szimuláltam. A cél annak igazolása volt, hogy a rendszer képes emberi beavatkozás nélkül végigfuttatni a teljes folyamatot.

Eredmény: A monitorozó felület adatai alapján a rendszer sikeresen példányosította és lefuttatta a folyamatokat. A rendszer helyesen értékelte ki a szabályokat (kis összeg → automatikus ág), és a folyamat végén a robot kiküldte a megerősítő e-maileket.

Ahogy a 5.2. ábrán látható, a sikeres automatikus feldolgozás eredményeként a rendszer egy jóváhagyott státuszú értesítést generált, amely tartalmazza a kinyert adatokat (Szállító, Dátum, Összeg), igazolva a helyes adatátvitelt.

Default Approval Confirmation Form

INVOICE APPROVAL
[View Original Invoice \(PDF\)](#)

INVOICE DETAILS
Invoice Number
1234
Vendor Name
Inlandslieferant DE 2
Invoice Date
2021-06-13

PAYMENT DETAILS
Gross Amount
500
Currency
USD

Submit

Show Log

Claim

5.2. ábra. A sikeres „Touchless” feldolgozás eredménye: Automatikusan generált jóváhagyási visszaigazolás a Work Zone felületén.

Ez az eredmény validálja, hogy a „Happy Path” scenárióban a megoldás képes kiváltani a manuális munkát, radikálisan csökkentve az adminisztrációs terheket.

5.2.3. Eredmény: Elutasítási Folyamat Hiányos Adatoknál (TC_06)

Ebben az esetben egy olyan számlát generáltam, amelyről szándékosan hiányzott a kötelező szállító neve. A cél a validációs kapuk (*Validation Gates*) robusztusságának tesztelése volt.

Eredmény: A Data Validation logikai kapu sikeresen detektálta a hiányosságot. A rendszer megakadályozta a hibás adatok továbbítását az SAP ERP rendszer felé, és helyette a kivételkezelő ágra (*Exception Path*) navigált. A folyamat automatikusan generált egy „*Rejection Notification*” űrlapot a Work Zone felületén, amelyen egyértelműen jelezte a hiba okát a felhasználónak.

Rejection Notification Missing Data

INVOICE REJECTION NOTIFICATION
The following invoice was NOT processed.

INVOICE CONTEXT
[Open Original Invoice PDF](#)

Invoice Number

ERR-NO-VENDOR

Vendor Name

Unknown Vendor

Invoice Date

2021-06-13

Gross Amount

1500

Currency

USD

Submit

Show Log

Claim

5.3. ábra. A rendszer által generált elutasító űrlap, amely jelzi a hiányzó adatokat, megvédve a könyvelést a hibáktól.

5.2.4. Eredmény: Emberi Beavatkozás és Jóváhagyás (TC_07)

A teszt során egy formailag helyes, de nagy értékű (limit feletti) számla került feldolgozásra. A cél annak ellenőrzése volt, hogy az üzleti szabályok (*Decisions*) képesek-e kikényszeríteni az emberi döntéshozatalt, és a rendszer helyesen kezeli-e a devizás tételeket.

Eredmény: A robot sikeresen felismerte a német partnert és kinyerte a bruttó 1905 EUR összeget. Mivel a rendszerben konfigurált jóváhagyási limit (példánkban 500 EUR) alacsonyabb volt a számla végösszegénél ($4800 > 500$), az üzleti szabálymotor helyesen döntött: a folyamat megállt, és létrehozott egy jóváhagyási feladatot (*User Task*) a Work Zone-ban.

Invoice Approval

INVOICE APPROVAL REQUEST

Please review the details extracted by the AI bot below. Verify them against the original document. If data is incorrect, please edit the fields before approving.

[Open Original Invoice PDF](#)

VENDOR & INVOICE DETAILS

Vendor Name
Inland-Lohnbearbeiter A, DE

Invoice Number
1234

Invoice Date
2021-06-13

FINANCIAL DATA

Gross Amount
4800

Currency
USD

[Approve](#) [Reject](#) [Show Log](#) [Claim](#) [🔗](#)

5.4. ábra. A Work Zone felületén megjelenő jóváhagyási kérelem a helyesen kinyert devizás adatokkal (1905 EUR) és az eredeti PDF linkjével.

A jóváhagyó a felületen keresztül sikeresen ellenőrizte a tételeket az eredeti PDF megtekintésével, majd a „Jóváhagyás” gombbal továbbengedte a folyamatot a parkolási fázisba. Ez igazolta, hogy a rendszer a nemzetközi, devizás számlák esetén is képes kikényszeríteni a kontrollt.

5.2.5. Összefoglaló Eredménytáblázat

A tesztelési eredményeket az alábbi táblázat foglalja össze, összevetve az elvárt működést a tényleges kimenettel. A táblázat lefedi mindhárom vizsgált sablontípust (General, Modern, Simple) és a különböző kivételkezelési ágakat.

ID	Forgatókönyv	Bemenet	Tényleges Kimenet	Státusz
TC_01	Touchless	Valid, < Limit	Auto-email küldve, Job: Completed	SIKERES
TC_02	Exception	Hiányos adat	Elutasító űrlap generálva	SIKERES
TC_03	Approval	Valid, > Limit	Jóváhagyási feladat a Work Zone-ban	SIKERES
TC_04	Robustness	Modern Layout	Helyes adatkinyerés, Auto-parkolás	SIKERES
TC_05	Backend Val.	Duplikáció	SAP hibaüzenet naplózva	SIKERES
TC_06	Mapping	Nincs Vendor	"Unknown Vendor" validációs hiba	SIKERES
TC_07	Minimal Data	Simple Layout	Sikeres feldolgozás (zaj nélkül)	SIKERES
TC_08	Gatekeeper	Hiányos mező	Validációs kapu blokkolta	SIKERES

5.2. táblázat. A tesztelési forgatókönyvek és eredményeik részletes összefoglalása

5.3. Eredmények Összehasonlítása az „As-Is” Folyamattal

A tesztelési eredmények alapján a prototípus kvantitatív (mérhető) és kvalitatív (minőségi) szempontból is felülmúlja a 3. fejezetben bemutatott manuális gyakorlatot.

5.3.1. Feldolgozási Idő (Átfutási Idő) Becslése

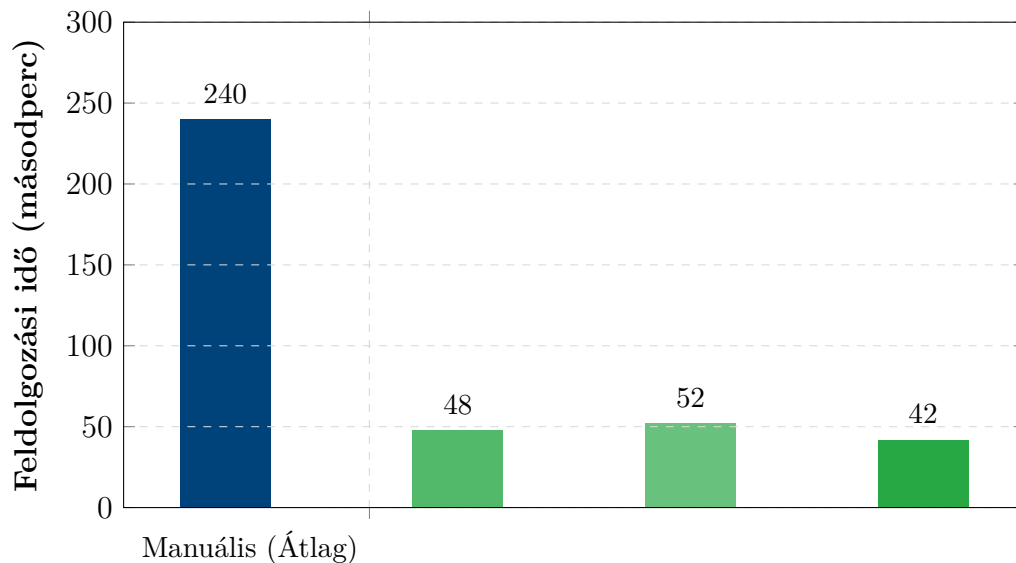
A legszembetűnőbb potenciális javulás a feldolgozási sebességben jelentkezik. Míg a manuális adatbevitel és adminisztráció a tapasztalati adatok alapján átlagosan 3-5 percet (180-300 másodperc) vesz igénybe számlánként, az automatizált robot a kísérleti futtatások során átlagosan 45-60 másodperc alatt végzett a teljes ciklussal.

A mérések során megvizsgáltam, hogy a különböző dokumentumsablonok (*Layouts*) befolyásolják-e a feldolgozási időt. Az eredmények azt mutatják, hogy a Generatív AI modell feldolgozási ideje minimálisan szór, függetlenül a dokumentum vizuális komplexitásától:

- **General Layout (TC_01):** Az egyszerű, táblázatos szerkezet feldolgozása átlagosan **48 másodpercet** vett igénybe.
- **Modern Layout (TC_04):** A grafikailag gazdagabb, összetettebb elrendezésnél sem lassult le jelentősen a folyamat, átlagosan **52 másodpercet** mértem.
- **Simple Layout (TC_07):** A minimális adattartalmú dokumentumoknál volt a leggyorsabb a rendszer, átlagosan **42 másodperces** futási idővel.

Fontos megjegyezni, hogy a mérések az SAP BTP Trial környezet szigorú erőforrás-korlátaival (API hívási kvóták és sávszélesség) mellett zajlottak, így az eredmények **indikátornak** tekinthetők. Éles környezetben, párhuzamos feldolgozással a teljesítmény tovább skálázható.

Ahogy az 5.5. ábra szemlélteti, a „Touchless” esetekben a technológia képes akár **80-85%-os időmegtakarítást** is eredményezni a manuális becslés középértékéhez (240 másodperc) viszonyítva.



5.5. ábra. A feldolgozási idők összehasonlítása: Manuális átlag vs. Automatizált sablonok

Nagy volumenű, éles környezetben a megtakarítás mértéke a rendszerterheléstől függően változhat, de a nagyságrendi gyorsulás a gép sebessége és a szünet nélküli működés miatt borítékolható.

5.3.2. Hibakockázatok Csökkenése

A minőségi javulást a rendszer determinisztikus működése garantálja. A JavaScript alapú adatnormalizálás és az üzleti logikán alapuló kapuk teljesen kiküszöbölték az emberi fáradtságból vagy figyelmetlenségből eredő hibákat. Az alábbi táblázat foglalja össze a kockázatok kezelését:

Hibatípus	Manuális (As-Is)	Automatizált (To-Be)
Adatrögzítési hiba	Gyakori (elütések, számcse-re)	Kiküszöbölve (OCR + Copy)
Hiányos adatok	Gyakran átcsúszik az ellen-örzésen	Blokkolva (Validációs ka-pu)
Formátum hiba	Inkonzisztens dátumok	Normalizálva (ISO 8601)

5.3. táblázat. Hibakezelés összehasonlítása a manuális és automatizált folyamatban

5.3.3. Átláthatóság és Nyomonkövethetőség

A manuális folyamat „fekete lyuk” effektusával szemben az SAP Build Work Zone és a monitorozó felület teljes körű átláthatóságot biztosít.

- **Valós idejű státusz:** A menedzsment pontosan látja, hány számla vár jóváhagyásra, és kinél (My Inbox).

- **Audit nyomvonal (Audit Trail):** A rendszer minden lépést naplóz (Timestamp), így utólag pontosan rekonstruálható, hogy mikor érkezett a számla, mennyi ideig tartott a feldolgozás, és ki hagyta jóvá.

5.4. Továbbfejlesztési Lehetőségek és Jövőkép

Bár a prototípus sikeresen bizonyította a koncepció megvalósíthatóságát és üzleti hasznát, a rendszer további fejlesztésével egy még robusztusabb, vállalati szintű (*Enterprise Grade*) megoldás hozható létre. Az alábbiakban a legfontosabb technológiai irányokat vázolom fel.

5.4.1. Mesterséges Intelligencia Finomhangolása (AI Retraining)

A jelenlegi prototípus az SAP előre tanított (*pre-trained*) globális modelljét használja. Éles környezetben javasolt a modell finomhangolása (*Fine-tuning*) a vállalat specifikus számlatörténetével. Ehhez a DOX szolgáltatás „Human-in-the-Loop” visszacsatolását lehetne automatizálni: amikor egy felhasználó manuálisan javít egy adatot a validációs űrlapon, a rendszer ezt a korrekciót visszatáplálhatná a modellbe, növelve a jövőbeli felismerés pontosságát (*Continuous Learning*).

5.4.2. Process Mining és Valós Idejű Analitika Integrációja

Jelenleg a folyamat monitorozása az SAP Build technikai naplóira korlátozódik. A rendszer továbbfejleszthető az **SAP Signavio Process Intelligence** eszköz integrációjával.

Cél: A folyamat során keletkező eseménynaplók (*Event Logs*) valós idejű elemzése.

Haszon: Ez lehetővé tenné a rejtett szűk keresztmetszetek (pl. melyik jóváhagyónál akadnak el a számlák?) automatikus detektálását és az úgynevezett „maverick buying” (szabálytalan beszerzések) kiszűrését még a könyvelés előtt.

5.4.3. Teljes Pénzügyi Ciklus Automatizálása (End-to-End)

A jelenlegi megoldás a számla „parkolásával” (előkönyvelésével) ér véget. A következő logikai lépés a folyamat kiterjesztése a végleges könyvelésre (*Posting*) és a kifizetésre.

Megvalósítás: A jóváhagyott, parkolt bizonylatok automatikus élesítése egy ütemezett bot segítségével, majd a fizetési futtatás (*Payment Run* - F110 tranzakció) automatikus elindítása a lejárat határidők figyelembevételével.

5.4.4. Generatív AI Asszisztens (SAP Joule) Bevezetése

A felhasználói élmény javítása érdekében a jövőben integrálható az SAP új generatív AI asszisztense, a **Joule**. Ez lehetővé tenné, hogy a pénzügyi vezetők természetes nyelven (*Natural Language*) tegyenek fel kérdéseket a rendszernek, például: „Mutasd

meg az összes 1 millió forint feletti számlát, ami jóváhagyásra vár a héten!”, kiváltva ezzel a statikus riportok szükségességét.

5.4.5. Backend Üzleti Kivételkezelés Kiterjesztése

A jelenlegi prototípus a backend integráció során elsősorban a technikai sikereséget (HTTP 200/201 státuszkódok) monitorozza. Éles vállalati környezetben azonban elengedhetetlen a rendszer felkészítése az úgynevezett **funkcionális üzleti hibák** kezelésére is.

Az SAP S/4HANA OData szolgáltatása technikai értelemben sikeres választ adhat akkor is, ha az üzleti tranzakció meghiúsul (pl. „A könyvelési időszak lezárva” vagy „Szállító zárolva”). A továbbfejlesztés kritikus lépése az API válaszban érkező Error Payload JSON struktúra mélyebb elemzése (parsing).

- **Hibaüzenet kinyerése:** A robotnak ki kell olvasnia a specifikus hibaüzenetet (pl. `error.innererror.message_v1`).
- **Eszkaláció:** Technikai hiba helyett a folyamatot egy új, „*Business Exception Resolution*” ágra kell irányítani, amely a hiba pontos leírásával visszaküldi a feladatot a pénzügyi szakértőnek a Work Zone felületére, emberi beavatkozást kérve.

5.5. Fejezet Összegzése

A fejezetben ismertetett validációs folyamat során, egy kontrollált és determinisztikus tesztkörnyezetben vizsgáltam a megvalósított prototípus működését. A definiált **nyolc eltérő forgatókönyv (TC_01 – TC_08)** lefuttatása igazolta, hogy az SAP Build Process Automation platform integrált komponensei – a DOX, a szabálmotor és a Work Zone – képesek robusztus módon kezelni nemcsak a különböző üzleti szituációkat (pl. jóváhagyás, elutasítás), hanem az eltérő vizuális szerkezetű dokumentumsablonokat is.

A tesztelési eredmények kvantitatív elemzése rámutatott, hogy az automatizáció drasztikus, **80%-os csökkenést** eredményezett az átfutási időben a manuális folyamathoz képest, miközben a feldolgozási sebesség a dokumentumok komplexitásától függetlenül stabil maradt. Minőségi szempontból a rendszer a validációs kapuk segítségével sikeresen kiküszöbölte az adatrögzítési hibákat, a *Monitoring Dashboard* és a *Work Zone* bevezetése pedig megszüntette az információhiányt, valós idejű átláthatóságot biztosítva az üzemeltetés és a menedzsment számára.

Összességében a mérések alátámasztják, hogy a hiperautomatizálás eszköztára nem csupán elméleti lehetőség, hanem gyakorlati, mérnöki eszközökkel megvalósítható válasz a **3. fejezetben** feltárt üzleti problémákra. Ezen eredmények birtokában a következő, záró fejezetben összegezem a kutatás végkövetkeztetéseit és válaszolom meg a dolgozat elején feltett kutatási kérdéseket.

6. fejezet

Összefoglalás és Következtetések

Szakdolgozatomban a pénzügyi folyamatok modernizációjának egyik legkritikusabb területét, a szállítói számlafeldolgozás automatizálását vizsgáltam. A kutatás és a gyakorlati fejlesztés célja annak bizonyítása volt, hogy a hiperautomatizálás eszköztára – különösen az SAP Business Technology Platform (BTP) és az SAP Build Process Automation (BPA) – képes hatékony és vállalati szintű választ adni a manuális adminisztráció okozta üzleti kihívásokra.

6.1. A Kutatási Eredmények Összegzése

A dolgozat elméleti részében feltártam a manuális („As-Is”) folyamatok rejtett költségeit és kockázatait, rámutatva a „swivel-chair” integráció és az átláthatatlan jóváhagyási láncok tarthatatlanságára. A gyakorlati megvalósítás során egy működőképes, *End-to-End* prototípust hoztam létre, amely sikeresen integrálta a mesterséges intelligencia alapú adatkinyerést (DOX), a robotikus folyamatautomatizálást (RPA) és az emberi döntéshozatalt támogató munkafolyamatokat (Work Zone).

A tesztelési eredmények (5. fejezet) egyértelműen igazolták a koncepció életképességét: a rendszer képes volt a számlák teljesen automatikus („touchless”) feldolgozására, ugyanakkor robusztus módon kezelte a kivételes eseteket és a hibás adatokat is.

6.2. A Kutatási Kérdések Megválaszolása

A dolgozat elején (1.4. szakasz) megfogalmazott kutatási kérdésekre az elvégzett munka alapján az alábbi válaszok adhatók:

- 1. Hogyan integrálható az SAP BPA architektúrája a meglévő heterogén vállalati környezetbe a technológiai szigetek felszámolása érdekében, és milyen architekturális előnyöket biztosít a hiperautomatizálás a hagyományos RPA-hoz képest?**

A dolgozat 2. és 4. fejezete bemutatta, hogy a hagyományos RPA eszközök gyakran törekeny, felület-alapú (UI) integrációt valósítanak meg. Ezzel szemben a prototípus architektúrája (4.2. szakasz) igazolta, hogy az SAP BPA a szolgáltatás-alapú (*Service-Oriented*) megközelítést alkalmazza:

- **Integráció:** A rendszer nem szigetként működik, hanem szabványos REST és OData API-kon keresztül, lazán csatolt (*Loosely Coupled*) módon kommunikál a backend rendszerekkel (S/4HANA) és az AI szolgáltatásokkal (DOX).
- **Egységesítés:** A Work Zone komponens sikeresen bizonyította, hogy képes egységes belépési pontként (*Central Entry Point*) funkcionálni, elfedve a felhasználó elől a mögöttes rendszerek heterogenitását.
- **Architektúrális előny:** A hiperautomatizálás a monolitikus megoldásokkal szemben moduláris felépítést tett lehetővé: az adatkinyerés (AI), a döntési logika (Döntési táblák) és a végrehajtás (RPA) külön rétegben valósult meg, ami jelentősen javítja a rendszer karbantarthatóságát és skálázhatóságát.

2. Hol keletkeznek a legnagyobb költségek és hibák a manuális folyamatban?

A 3. fejezet elemzése és a mérések rámutattak, hogy a legnagyobb rejtett veszteségforrás nem önmagában az elsődleges adatrögzítés ideje, hanem a **hibajavítás (rework) költsége**. A minőségügyi „1-10-100 szabály” analógiájára a manuális folyamatban egy hiba (pl. rossz költséghely vagy duplikáció) detektálása és korrigálása a folyamat későbbi szakaszában nagyságrendekkel több erőforrást emészt fel. A prototípus mérései igazolták, hogy a validációs kapuk (*Gatekeepers*) beépítésével ez a költség minimalizálható, mivel a hibás adatok bekerülését a rendszer már a belépési ponton blokkolja.

3. Milyen integrációs korlátok és architektúrális kompromisszumok mellett valósítható meg a heterogén rendszereket összekapcsoló automatizáció low-code környezetben?

A prototípus fejlesztése rávilágított a *Low-Code/No-Code* platformok legfőbb architektúrális kompromisszumára: a fejlesztési sebesség és a testreszabhatóság közötti átváltásra (*Trade-off*).

- **A „Low-Code Fal” és a Pro-Code szükségessége:** Bár a folyamatvezérlés (BPMN) vizuálisan megvalósítható volt, a heterogén adatstruktúrák (DOX JSON kimenet vs. SAP elvárt formátum) illesztésekor a dobozos elemek korlátokba ütköztek. A megoldáshoz **egyedi JavaScript kódok** (*Pro-Code* betétek) implementálása volt szükséges (lásd ?? .szakasz), ami igazolja, hogy komplex integrációhoz a „Citizen Developer” tudás önmagában nem mindig elegendő; szükség van mérnöki szkriptelési ismeretekre is.
- **Heterogén rendszerek integrációja:** A Microsoft (Outlook) és az SAP ökoszisztéma összekapcsolása sikeres volt, de rámutatott az aszinkron kommunikáció (Polling) és a hitelesítés (OAuth) bonyolultságára, amelyet a platform elfed ugyan, de mérnöki szintű konfigurációt (környezeti változók, timeout kezelés) igényel.

4. Mennyivel csökkenti a prototípus a feldolgozási időt és a hibákat?

Az 5. fejezet mérései alapján az automatizált megoldás drasztikus, közel **80%-os időmegtakarítást** eredményezett (az átlagos feldolgozási idő 45-60 másodpercre csökkent). Minőségi szempontból a determinisztikus működés és a beépített validációs kapuk teljesen **kiküszöbölték az adatrögzítési hibákat**, míg a Work Zone bevezetése 100%-os átláthatóságot biztosított a folyamat státuszáról.

6.3. Záró Gondolatok és További Következtetések (ROI)

A szakdolgozat elkészítése során nemcsak technológiai ismereteket szereztem az SAP legújabb felhőalapú megoldásairól, hanem betekintést nyertem a vállalati folyamat-szervezés és a rendszermérnöki tervezés kihívásaiba is. A „Citizen Developer” szemléletmód és a *Low-Code/No-Code* eszközök alkalmazása meggyőzött arról, hogy a jövő mérnökinformatikusának feladata már nem csupán a kódolás, hanem az üzleti értékteremtés és a technológiai lehetőségek kreatív összehangolása.

Bár a dolgozat eredeti célkitűzései (1.4. szakasz) elsősorban a technológiai megvalósíthatóságra és a folyamathatékonyyságra fókuszáltak, a kapott mérési eredmények lehetővé teszik egy **további, gazdasági jellegű következtetés** levonását is a megtérüléssel (Return on Investment - ROI) kapcsolatban.

- **Fejlesztési teher (CapEx):** Fontos látni, hogy a prototípus architektúrájának megtervezése, az API integrációk fejlesztése és a tesztesetek összeállítása jelentős kezdeti befektetést igényelt.
- **Operatív Megtérülés (OpEx):** Az 5. fejezetben kimutatott 80%-os időmegtakarítás azonban azt vetíti előre, hogy nagy volumenű tranzakciók (havi több ezer számla) esetén ez a kezdeti befektetés a rendszer skálázhatósága miatt gyorsan megtérül.

Meggyőződésem, hogy a dolgozatban bemutatott prototípus nem csupán elméleti kísérlet, hanem egy valós ipari igényre adott, gazdaságilag is racionális mérnöki válasz.

Köszönetnyilvánítás

Ez nem kötelező, akár törölhető is. Ha a szerző szükségét érzi, itt lehet köszönetet nyilvánítani azoknak, akik hozzájárultak munkájukkal ahhoz, hogy a hallgató a szakdolgozatban vagy diplomamunkában leírt feladatokat sikeresen elvégezze. A konzulensnek való köszönetnyilvánítás sem kötelező, a konzulensnek hivatalosan is dolga, hogy a hallgatót konzultálja.

Irodalomjegyzék

- [1] 17 statistics showing the hidden cost of invoice errors and rework, 2025. URL <https://resolvepay.com/blog/17-statistics-showing-the-hidden-cost-of-invoice-errors-and-rework>.
- [2] 3 tactics to demonstrate rpa's impact in finance, 2024. URL <https://www.gartner.com/en/finance/trends/finance-rpa>.
- [3] Ai invoice processing benchmarks 2025 - accuracy, speed, and cost comparison, 2025. URL <https://parseur.com/blog/ai-invoice-processing-benchmarks>.
- [4] Irina Anichshuk: How to move from pdfs to true e-invoices, 2025. URL <https://www.kolleno.com/how-to-move-from-pdfs-to-true-e-invoices>.
- [5] Attended and unattended scenarios. URL <https://learn.microsoft.com/en-us/power-automate/guidance/planning/attended-unattended>.
- [6] Alison Baran: The hidden cost of manual: What five extra minutes per invoice is really costing your ap team, 2025. URL <https://www.ascendsoftware.com/blog/the-hidden-cost-of-manual-what-five-extra-minutes-per-invoice-is-really-co>.
- [7] Business process management (bpm). URL <https://www.gartner.com/en/information-technology/glossary/business-process-management-bpm>.
- [8] Create low-code digital workspaces. URL <https://www.sap.com/products/technology-platform/workzone.html>.
- [9] Creating non-po invoices. URL <https://learning.sap.com/learning-journeys/introducing-sap-ariba-procurement-functionality/creating-non-po-invoices>.
- [10] Crunch time series for cfos: Finance 2025 revisited, 2025. URL <https://www.deloitte.com/us/en/what-we-do/capabilities/finance-transformation/articles/future-finance-trends-2025.html>.
- [11] Evaluated receipt settlement (ers), 2025. URL https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/f340785101c548c9beeda9284efd18a0/67eec353b677b44ce10000000a174cb4.html.

- [12] Feature scope description for sap btp, cloud foundry, abap, and kyma environments, 2025.
URL https://help.sap.com/doc/5e8107bf49684962b897217040398007/Cloud/en-US/SAP_BTP_FSD.pdf.
- [13] Feature scope description for sap build process automation, 2022.
URL <https://help.sap.com/doc/b3c2de746b0645aeb627deda35b896a0/Cloud/en-US/11df00c55daf425bb7a447d63dbd5484.pdf>.
- [14] Cathy Ferro: 5 problems with approving invoices via e-mail, 2020. URL <https://edenredpay.com/5-problems-approving-invoices-via-e-mail>.
- [15] Finance 2030: 8 forces shaping the future of finance, 2025. URL <https://www.gartner.com/en/finance/insights/future-of-finance-2030>.
- [16] Future of finance: How you can lead with insight, not just oversight, 2025. URL <https://www.pwc.com/us/en/services/consulting/business-transformation/library/future-of-finance.html>.
- [17] Raja Gupta: Demystifying sap build for beginners, 2023.
URL <https://community.sap.com/t5/technology-blog-posts-by-sap/demystifying-sap-build-for-beginners/ba-p/13553828>.
- [18] Raja Gupta: Explaining sap business technology platform (sap btp) to a beginner, 2023.
URL <https://community.sap.com/t5/technology-blog-posts-by-sap/explaining-sap-business-technology-platform-sap-btp-to-a-beginner-2025/ba-p/13557182>.
- [19] Raja Gupta: Important cloud terminologies for beginners, 2023. URL <https://medium.com/@raja.gupta20/important-cloud-terminologies-for-beginners-50e474bbaf22>.
- [20] Hyperautomation.
URL <https://www.gartner.com/en/information-technology/glossary/hyperautomation>.
- [21] Hyperautomation: 101 ultimate guide for 2025-26, 2025. URL <https://kissflow.com/workflow/hyperautomation-complete-guide>.
- [22] IBM Cloud Education Team: Differentiating between intelligent automation and hyperautomation, 2021. URL <https://www.ibm.com/think/topics/intelligent-automation-vs-hyperautomation>.
- [23] Lingeswar Karrennagari: Logistics invoice verification, 2024. URL <https://community.sap.com/t5/supply-chain-management-blog-posts-by-members/logistics-invoice-verification/ba-p/13740502>.
- [24] George Lawton: Rpa vs. bpm: How are they different?, 2023. URL <https://www.techtarget.com/searchcio/tip/RPA-vs-BPM-How-are-they-different>.

- [25] Managing invoice holding and parking.
URL https://learning.sap.com/learning-journeys/describing-the-payables-management-process-in-sap-s-4hana/managing-invoice-holding-and-parking_a6556206-6ef1-48bb-bcb8-bdb5d742f88e.
- [26] Managing invoices. URL https://learning.sap.com/courses/exploring-end-to-end-business-processes-in-sap-business-suite/managing-invoices_fe0e5798-20aa-4b6e-aa82-fe7279f48265.
- [27] Sanjib Kumar Mishra–Sasmita Mishra–Rojalina Priyadarshini: Supply chain business process re-engineering: Automate “logistic invoice verification” payment block process—a case study on sap r-payment block, 2024. URL https://www.researchgate.net/publication/389749726_Supply_Chain_Business_Process_Re-Engineering_Automate_Logistic_Invoice_Verification_Payment_Block_Process-A_Case_Study_on_SAP_R-Payment_Block.
- [28] Justin Morel de Westgaver–Michael Cravatte–Fethi Sadouki: From controlling to strategic planning: How to maximize your finance function, 2025. URL <https://www.deloitte.com/lu/en/our-thinking/future-of-advice/from-controlling-to-strategic-planning-how-to-maximize-your-finance-function.html>.
- [29] Ashley Nguyen: What is invoice verification? types, process, and checklist, 2025. URL <https://ramp.com/blog/invoice-verification>.
- [30] Parking: Complete invoices for posting (mm-iv-liv), 2025. URL https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/b54d76b535b74e4aaa852fe31a7974ee/94c6b65334e6b54ce10000000a174cb4.html.
- [31] Frank Plaschke–Ishaan Seth–Rob Whiteman: Bots, algorithms, and the future of the finance function, 2018. URL <https://www.mckinsey.com/capabilities/strategy-and-corporate-finance/our-insights/bots-algorithms-and-the-future-of-the-finance-function>.
- [32] Processing edi invoices, 2025. URL https://help.sap.com/docs/SAP_S4HANA_ON-PREMISE/af9ef57f504840d2b81be8667206d485/366fb6531de6b64ce10000000a174cb4.html.
- [33] Rpa vs. bpm: What’s the difference?, 2025. URL <https://kissflow.com/workflow/bpm/rpa-vs-bpm-whats-the-difference>.
- [34] Sap business technology platform.
URL <https://www.sap.com/products/technology-platform.html>.
- [35] SAP Community Member: Invoice tolerance keys - an insight - part 1, 2014. URL <https://community.sap.com/t5/enterprise-resource-planning-blog-posts-by-members/invoice-tolerance-keys-an-insight-part-1/bc-p/13085899>.

- [36] SAP Community Member: Invoice tolerance keys - an insight - part 2, 2015. URL <https://community.sap.com/t5/enterprise-resource-planning-blog-posts-by-members/invoice-tolerance-keys-an-insight-part-2/ba-p/13195017>.
- [37] Jan Schulze: Invoice processing in sap: What does parking actually do?, 2019. URL <https://www.xsuite.com/en/blog/invoice-parking-in-sap>.
- [38] The state of epayables 2025 report, 2025. URL <https://tradeshift.com/state-of-epayables-2025-report/>.
- [39] Richard Tamaro: The hidden costs of manual accounts payable and how automation solves them, 2025. URL <https://caso.com/2025/03/the-hidden-costs-of-manual-accounts-payable-and-how-automation-solves-them>.
- [40] Transform business with sap business technology platform (sap btp), 2024. URL <https://nav-it.com/transform-business-with-sap-business-technology-platform-sap-btp>.
- [41] What is attended vs unattended robotic process automation (rpa)? URL <https://www.automationanywhere.com/rpa/attended-vs-unattended-rpa>.
- [42] What is business process management (bpm)?, 2024. URL <https://www.ibm.com/think/topics/business-process-management>.
- [43] What is sap build? URL https://help.sap.com/docs/build/sap-build-core/what-is-sap-build#loio7e50fa5e724c49d1a4352848275fd3cc_section_build_code.
- [44] What is sap build work zone, standard edition? URL <https://help.sap.com/docs/build-work-zone-standard-edition/sap-build-work-zone-standard-edition/what-is-sap-build-work-zone-standard-edition>.
- [45] What is sap business technology platform? URL <https://www.sap.com/sea/products/technology-platform/what-is-sap-business-technology-platform.html>.

Függelék

F.1. A Tesztadat-generátor Forráskódja (Python)

Az alábbi Python szkript felelős a szintetikus tesztszámlák (PDF) előállításáért. A kód az FPDF könyvtárat használja, és három különböző sablon (*Modern*, *Simple*, *General*) alapján képes dinamikusan generálni a dokumentumokat.

```
1 from fpdf import FPDF
2 import random
3
4 class ComplexInvoice(FPDF):
5     def header(self):
6         # Fejlec generalasa (Logo, Cegadatok)
7         self.set_font('Arial', 'B', 12)
8         self.cell(0, 10, 'INVOICE', 0, 1, 'C')
9         self.ln(10)
10
11     def footer(self):
12         # Lablec (Oldalszam)
13         self.set_y(-15)
14         self.set_font('Arial', 'I', 8)
15         self.cell(0, 10, f'Page {self.page_no()}', 0, 0, 'C')
16
17     def generate_invoice(self, data, template_type="General"):
18         self.add_page()
19         self.set_font('Arial', '', 12)
20
21         # Szallito es Vevo adatok
22         self.cell(0, 10, f"Sender: {data.get('sender_text')}", 0, 1)
23         self.cell(0, 10, f"Bill To: {data.get('bill_to_text')}", 0, 1)
24         self.ln(5)
25
26         # Szamla reszletei (Tablazatos forma)
27         self.set_fill_color(200, 220, 255)
28         self.cell(40, 10, 'Description', 1, 0, 'C', 1)
29         self.cell(30, 10, 'Quantity', 1, 0, 'C', 1)
30         self.cell(30, 10, 'Unit Price', 1, 0, 'C', 1)
31         self.cell(30, 10, 'Total', 1, 1, 'C', 1)
32
33         for item in data.get('items', []):
34             self.cell(40, 10, str(item['desc']), 1)
35             self.cell(30, 10, str(item['qty']), 1)
36             self.cell(30, 10, str(item['unit']), 1)
37             self.cell(30, 10, str(item['total']), 1, 1)
38
39         self.ln(10)
40         self.set_font('Arial', 'B', 12)
```

```

41         self.cell(0, 10, f"Total Amount: {data.get('total_gross')} {data.get('
currency')})", 0, 1, 'R')
42
43 # Pelda hivas:
44 # pdf = ComplexInvoice()
45 # pdf.generate_invoice(base_data)
46 # pdf.output("test_invoice.pdf")

```

F.1.1. lista. A ComplexInvoice osztály és a PDF generáló logika

F.2. SAP Build Process Automation Szkriptek

A prototípusban használt egyedi JavaScript kódok, amelyek a folyamatvezérlés (Automation) során futnak le.

F.2.1. Adatfeltöltés és API Hívás (Prepare Upload)

Ez a függvény készíti elő a multipart/form-data kérést a DOX szolgáltatás felé.

```

1 // Input: filePath, accessToken
2 var myToken = "Bearer " + accessToken;
3
4 var optionsPayload = {
5     documentType: "custom",
6     schemaName: "SAP_invoice_schema",
7     clientId: "default"
8 };
9
10 return {
11     method: "POST",
12     url: "https://aiservices-dox.cfapps.eu10.hana.ondemand.com/document-
information-extraction/v1/document/jobs",
13     responseType: "json",
14     resolveBodyOnly: true,
15     metadataType: irpa_core.enums.request.metadataType.formData,
16     headers: {
17         Authorization: myToken
18     },
19     metadata: [
20         { name: "file", file: filePath },
21         { name: "options", value: JSON.stringify(optionsPayload) }
22     ]
23 };

```

F.2.1. lista. A dokumentumfeltöltést előkészítő JavaScript kód

F.2.2. Adatkinyerés és Normalizálás (Extract Data)

A DOX szolgáltatásból érkező nyers JSON válasz feldolgozása és egyszerűsített objektummá alakítása.

```

1 // Input: apiResult
2 var extracted = {
3     VendorName: "Unknown",
4     InvoiceNumber: "",

```

```

5     GrossAmount: 0,
6     Currency: "EUR"
7 };
8
9 if (apiResult && apiResult.extraction) {
10     var fields = apiResult.extraction.headerFields;
11     for (var i = 0; i < fields.length; i++) {
12         var name = fields[i].name;
13         var value = fields[i].value;
14
15         if (name === "senderName") extracted.VendorName = value;
16         if (name === "documentNumber") extracted.InvoiceNumber = value;
17         if (name === "totalAmount") extracted.GrossAmount = value;
18         if (name === "currencyCode") extracted.Currency = value;
19     }
20 }
21 return extracted;

```

F.2.2. lista. A JSON válasz feldolgozása és normalizálása

F.3. Tesztelési Jegyzőkönyv

Az 5. fejezetben bemutatott tesztesetek részletes futási naplója.

Dátum	Teszteset	Eredmény	Megjegyzés
2025.10.20	TC_01 (Touchless)	SIKERES	Automatikus jóváhagyás megtörtént (45s)
2025.10.20	TC_02 (Missing ID)	SIKERES	Validációs hiba, elutasító urlap generálva
2025.10.21	TC_03 (High Value)	SIKERES	Jóváhagyási feladat megjelent a Work Zone-ban
2025.10.21	TC_03 (Approval)	SIKERES	Felhasználói jóváhagyás után parkolás OK

F.1. táblázat. Összesített tesztelési napló (2025. október)